

MTH6150: Introduction to the LINUX operating system

Charalampos Markakis¹ & Vasileios Paschalidis²

¹ *School of Mathematical Sciences, Queen Mary University of London and*

² *Departments of Astronomy & Physics, University of Arizona, Tucson*

This note provides a basic introduction to the Linux operating system, and the basic we will be using for the course. The commands that follow work in a Linux or MacOS terminal.

I. CREATING AND DELETING DIRECTORIES

Open a terminal. One of the most important aspects in keeping your files organized is to learn how to create directories. This can be done with the command

```
mkdir
```

Type

```
mkdir dir1
```

```
mkdir dir2
```

To create directories dir1 and dir2. You can also create in one stroke a directory with subdirectories as follows

```
mkdir -p dir3/subdir/subsubdir
```

The -p flag will create not only the directory subsubdir, but also the parent directories subdir and dir3.

To delete an empty directory you can type

```
rmdir dir1
```

“rm” for remove, or for multiple directories at once

```
rmdir dir1 dir2
```

If the directory “dir1” were not empty rmdir would error out with a message

```
rmdir: failed to remove 'dir1/': Directory not empty
```

To remove a non-empty or empty directory you can type

```
rm -r dir1
```

The command “rm” is the general command for removing files or directories, and the flag “-r” is there to tell “rm” to remove recursively.

II. NAVIGATION

Just like in the Windows operating system, it is important to learn how to navigate through the multiple directories.

- One of the most important commands is “pwd”, which allows you to find your current location. Type

```
pwd
```

You should see

```
/home/netid
```

where “netid” will be your netid. “pwd” stands for “print working directory”.

- Another important command is “ls” which lists the contents under your current location. Type

```
ls
```

You should see dir3 (assuming you did not delete dir3).

- But, we know that dir3 has subdirectories. How can we go into these directories? This is achieved with the command “cd”, which stands for “change directory”. Type

```
cd dir3/subdir/subsubdir
```

and now type again

```
pwd
```

you should see

```
/home/netid/dir3/subdir/subsubdir
```

This is the absolute path to the directory “subsubdir”, which you can use to always get into that directory. If you type

```
cd
```

without any argument, you will always end up to your home directory, which you can test by typing “pwd”. If you type

```
cd dir3/subdir/subsubdir
```

This is the relative path to “dir3/subdir/subsubdir” from your home directory.

- You can also use the “..” operator which is used to go one directory up (the immediate parent directory the working directory), i.e., if you now type

```
cd ../
```

and now type

```
pwd
```

you should get

```
/home/netid/dir3/subdir
```

In other words you are into “subdir”, which is the immediate parent directory of “subsubdir”.

- Now, use the absolute path to the subsubdir to change into “subsubdir”, i.e.,

```
cd /home/netid/dir3/subdir/subsubdir
```

and type again

```
pwd
```

You should now see “/home/netid/dir3/subdir/subsubdir”. In other words when you use the absolute path, it does not matter where you are. You will always go into the directory you want.

- One of the most convenient shortcuts of linux is the “tab-completion”. Which helps in writing all these long paths. Try again changing into the subsubdir using the absolute path

```
cd /home/netid/dir3/subdir/subsubdir
```

but each time you hit a letter hit the “tab” button a couple of times, if the path is not auto-completed enter another letter in the path

III. FILES

Linux is an extensionless system. It doesn't depend on the extension of a file to tell what type of file it is. For example, a pure text file with the extension .txt or .dat or .png or .jpg is treated as a text file in linux.

- Let create our first file using the “touch” command. Type

```
cd /home/netid/dir3/subdir/subsubdir
```

and

```
touch foo.txt foo1.txt foo2.txt
```

This will create 3 empty files in the subsubdir directory. Strictly speaking, touch just updates the timestamp of last access of the files. But, since these files never existed they are created by the system.

Lets also write something in these files. Type

```
echo "First entry into foo file" >> foo.txt
```

```
echo "First entry into foo1 file" >> foo1.txt
```

```
echo "First entry into foo2 file" >> foo2.txt
```

Hint: Remember to always use the up arrow to access your previous commands, so that you don't have to retype everything.

The “echo” command simply prints the text in the quotes, while the append operator “>>” just appends to the files the output from echo, i.e., the text in the quotes.

- We can now use the “file” command to figure out the file type, i.e., enter

```
file foo.txt
```

and the output should be.

```
foo.txt: ASCII text
```

- When choosing file names and directory names, it is more convenient to not use spaces as they require special treatment. A better practice is to use underscores in the place of spaces, i.e.,

```
echo "First entry into foo file" >> my_fist_foo_file.txt
```

- Lets now list the contents of the subsubdir, i.e., type

```
ls
```

We can also use options on ls to obtain more information about the files, i.e.,

```
ls -l
```

which will list the files in alphabetical order and show the permissions of the files (no need to get into this for our purposes), the owner and group of the files, the size of the files in bytes (if you use the flags “-lh” it will print the file size in human readable form), the timestamps for the files and the filenames. We can also use a combination of options. A useful one is

```
ls -ltrh
```

which lists the files in reverse order time, i.e., the item that was accessed last appears at the bottom of the list.

- Earlier we mentioned that Linux has hidden files. We can list these by using the “-a” or “-all” flag. Go to your home directory using `cd`, and type

```
ls -a
```

You should now see a number of files that start with a “.”. All these are hidden files. You can also see your “.bash_history” file (which contains your previous commands), as well as one of the most important files in Linux the “.bashrc”. The “.bashrc” file is loaded every time you log into your linux account and can be used to define useful macros, including aliases, set shell variables (like the `$SHELL` variable we used) and other useful shortcuts.

For most Linux commands there are manual pages which you can access by typing “man” and the command name, e.g.,

```
man ls
```

In the manual pages you can read more about the command and the different options (flags) it has. Of course the world wide web is always your friend.

A. Moving files around

Copying files or moving them is always useful. Create a new directory in your home directory, typing

```
mkdir ~/dir1; cd
```

Notice that we used “~/”, which is a shortcut to the home directory path. We also used the semicolon “;” operator to separate the `mkdir` command from the `cd` command. You can use multiple semicolons on one line to separate multiple commands that you can execute in the order they appear and hit enter only once. Now copy the “my_fist_foo_file.txt” from `subsubdir` to the `dir1` directory. Using the “cp” (copy) command and the paths to the files

```
cp ../dir3/subdir/subsubdir/my_fist_foo_file.txt dir1/
```

The first dot “.” is meant to indicate that `dir3` is in your working directory. The command would be equivalent to

```
cp dir3/subdir/subsubdir/my_fist_foo_file.txt dir1/
```

Remember to always use the tab completion to write the paths, so that you don’t have to type the full path.

The syntax of the copy command is “cp [source] [destination]”. If you are sure you don’t want to copy the file, but move it, i.e., no longer have the file in your source you can use the “mv” command, e.g.,

```
mv ../dir3/subdir/subsubdir/my_fist_foo_file.txt ../dir3/subdir/
```

Now list the contents of the `subsubdir` directory

```
ls ../dir3/subdir/subsubdir/
```

and you should see the file you moved. Also, list the contents of `subdir`

```
ls ../dir3/subdir/
```

and you will see the file was indeed moved. The copy command can be used to copy directories too.

IV. EDITORS

One of the most important tools in building our programs are text editors. Almost all Linux distributions these days have vi, vim, emacs and the nano or pico editors. If you know vi or vim or nano or pico feel free to use them. Here we will see the emacs text editor. Go into the subdir directory and launch emacs

```
emacs my_fist_foo_file.txt
```

You can now start to edit this file. Add a few sentences to it.

- To move fast between the text you can use the “pg up” and pg dn” keys, the “home” and “end” keys, you can keep the “ctrl” key pressed down while using the left, right, up and down arrows to jump between words and text. To move a page down you can also type

```
ctrl+v
```

and to move up

```
Meta+v
```

where the “Meta” key is typically the “alt” key (the key left of the space bar).

To move to the beginning of a line you can also type

```
ctrl+a
```

and to the end of a line you can also type

```
ctrl+e
```

These shortcuts will expedite greatly how quickly you move the cursor through the text. If you want to go directly to the end of the text (buffer) type

```
Meta+<
```

and the beginning of the text (buffer)

```
Meta+>
```

- To save what you wrote type

```
ctrl+x+s
```

- To close emacs type

```
ctrl+x+c
```

- Copying and pasting inside a file is always useful. Open the file with emacs again

```
emacs my_fist_foo_file.txt
```

- Move the cursor to the beginning of the text you would like to copy/cut and hit

```
ctrl+space
```

- This activates the marking area tool. Use the arrows to highlight the area you want to cut/copy. To cut the text type

`ctrl+w`

- To copy the text type

`Meta+w`

- To paste the text, move the cursor where you want to paste the text and type

`ctrl+y`

where “y” stands for yank.

- To remove (kill) an entire line in your text you can type

`ctrl+k`

- To undo a move type

`ctrl+_`

In other words

`ctrl+shift+-`

To redo a move type

`ctrl+shift+space bar+-`

While these may sound like many commands, keep in mind that practice makes perfect!

V. OTHER WAYS TO READ FILES

Linux has a number of tools to read files

- `Cat <file>`

`cat` stands for concatenate. It will print the entire file in one go on your screen. Good for files that are only a few lines long.

- `less <file>`

It will print the file in pages that you can browse through using the up/down arrows or the up/down page keys. Type “Q” to exit.

- `head -nA <file>`

Here A is a integer number. The command print on screen the first A lines of the file

- `tail -nA <file>`

Here A is a integer number. The command print the last A lines of the file.

VI. GREPPING

When debugging we will often need to find certain expressions in a file. Using the linux command `grep` is one of the best ways to find an expression in a text, e.g.,

```
grep "First" ~/dir3/subdir/my_fist_foo_file.txt
```

The syntax is “`grep expression file`”. The above will search and print all the occurrences of “First” in the file `my_fist_foo_file.txt`

VII. MAKEFILES

The Linux utility `make` provides a convenient way for combining jobs/files that depend on other files. To drive `make` we need a `Makefile`. Most often, the `Makefile` tells `make` how to compile and link a program.

A `makefile` consists of “rules” which have the following structure

```
target ... : prerequisites ...
        recipe
        ...
```

A `target` can be the name of a file. Examples of targets are executable or object files.

The `prerequisites` are files used as input to generate the `target`. A `target` can depend on several files.

A `recipe` is an action that `make` executes. **Note: you need to put a tab character at the beginning of every recipe!**

Often a `recipe` is in a `rule` with `prerequisites` and will generate a `target` file if any of the `prerequisites` change. This is extremely useful when compiling a very large code that contains multiple files, because using `make` will recompile only those files that have changed.

However, a `rule` with a `recipe` for a `target` need not have `prerequisites`. For example, a `rule` containing the `rm` command associated with a `target` called “clean” does not have prerequisites.

A simple example `Makefile` would contain the following

```
myprogram : main.o utils.o
        g++ -o myprogram main.o utils.o

main.o : main.C header.h
        g++ -c main.C

utils.o : utils.C
        g++ -c utils.C

clean :
        rm myprogram main.o utils.o
```

If we have the files `main.C`, `utils.C` and `header.h`, and run `make` in the same directory as the `makefile`, the above `makefile` will then build the executable `myprogram`. The flag “-c” is to just compile files, but not link them. If instead you type `make clean`, `make` will then remove the files `myprogram`, `main.o`, `utils.o`.

In the above `makefile`, we have 4 targets that include the executable file `myprogram`, and the object `main.o`, `utils.o` and the target `clean`. The prerequisites for `myprogram` are `main.o` and `utils.o` which have their own prerequisites which are `main.C`, `header.h` and `utils.C`. A `recipe` follows each line that contains a `target` and `prerequisites`, that simply compiles the relevant files.

The target `clean` is not a file, and all `makefiles` typically contain it for convenience.

If we have a code that contains many files, it is far more convenient to use Variables to make our `Makefiles` simpler. Using variables the previous `makefile` can be simplified as follows

```
OBJS = main.o utils.o
cpp=g++
cppflags= -c

myprogram : ${OBJS}
        ${cpp} -o myprogram ${OBJS}

main.o : main.C myheader.h
        ${cpp} ${cppflags} main.C

utils.o : utils.C
        ${cpp} ${cppflags} utils.C
```

```
clean :
    rm myprogram ${OBJS}
```

The variable `OBJS` above contains the object files our program depends on. The variable `cpp` is the compiler, and `cppflags` is the compiler flags we want to use. Everything else is basically the same and to use a variable we always put the variable name within curly brackets that follow a dollar sign: `${}`. Using variables way we can simply change the compiler program, the flags, add more flags, such as compiler optimization flags, as well as add/remove more object files.

A. Compiling multiple programs with one Makefile

We can use one makefile to compile multiple programs. For example lets assume that we have 3 programs. One program is in files `program1.C` `header.h` `c utils.C`, the second program is contained in file `program2.C` and the third in file `program3.C`. The following Makefile can compile these programs

A simple example Makefile would contain the following

```
# Target to compile all
all: program1 program2 program3

# Target for program in program1.C, utils.C and header.C.
program1 : program1.o utils.o
    g++ -o myprogram1 program1.o utils.o

program1.o : program1.C header.h
    g++ -c program1.C

utils.o : utils.C
    g++ -c utils.C

# Target for program in program2.C
program2 : program2.o
    g++ -o program2 program2.o

program2.o : program2.C
    g++ -c program2.C

# Target for program in program3.C
program3 : program3.o
    g++ -o program3 program3.o

program3.o : program3.C
    g++ -c program3.C

clean :
    rm program1 program2 program3 *.o
```

The above Makefile compiles `program1` if you type `make program1`, `program2` if you type `make program2`, `program3` if you type `make program3`. It will compile all programs if you type `make all`. Finally, `make clean` will remove the executables and all object files.

VIII. BASIC PLOTTING WITH GNUPLOT

Gnuplot is linux utility for plotting data files and functions. Gnuplot has many intrinsic functions one can plot. To start plotting data or functions first we need to launch gnuplot. To do so type

gnuplot

Now you will be in the gnuplot command line. To plot the $\sin(x)$ simply type

```
p sin(x)
```

and hit “enter”. To specify a range in the x axis type

```
p [-3.1415:3.1415] sin(x)
```

and hit enter. The last command plots $\sin(x)$ from $-\pi$ to π . You can also set log scales if you desire. For example type

```
p [0,10] x**2
```

and hit enter. The operator “**” is the “to the power of” operator. So, this last command plots the function $f(x) = x^2$. To set the y-axis to be in log scale (i.e., a log-linear plot) just type

```
set log y; p [0.1:10] x**2
```

Here you learned that the semicolon separates different commands. To also set the x-axis in log scale type

```
set log y; set log x; p [0.1:10] x**2
```

and hit enter. You should see a straight line.

We can also label the axes

```
set log y; set log x; set xlabel "x"; set ylabel "f(x)=x*x"; p [0.1:10] x**2
```

Gnuplot has multiple intrinsic functions `log` (natural log) `log10` (base 10 log), `sin(x)`, `cos(x)`, `tan(x)`, `exp(x)` (the exponential function), ...

Gnuplot can make plots not only from analytic functions, but also from data. Put the following data into a file and save it as “data.txt”.

```
0.1      0.00316096 0.00537051
0.05     0.00101441 0.00185594
0.025    0.00025341 0.00048454
0.0125    6.3341e-05 0.00012385
0.00625   1.5834e-05 3.1312e-05
```

As you can see the above data set has 3 columns. To make a plot of column 2 vs column 1, while in the gnuplot command line simply type

```
p "data.txt" u 1:2 w l
```

and hit enter. Here in double quotation marks we place the name of the file “u” stands for “using”, “1:2” stands for column 2 vs column 1, so the first column that appears goes to the x-axis of the plot. The “w l” stands for “with lines”, i.e., it connects the data points. If you want to show the points alone use “w p” (with points), and if you want both points and lines use “w lp” (with line-points).

We can also create a single plot that shows more than one curves, by separating with commas the other curves. We can use, e.g. the third column, as follows

```
p "data.txt" u 1:2 w l, "data.txt" u 1:3 w l
```

The data here are very close to each other, so we need to take log scales, i.e.,

```
set log x; set log y; p "data.txt" u 1:2 w l, "data.txt" u 1:3 w l
```

Let’s also add labels to the axes

```
set log x; set log y; set xlabel "{/Symbol D} x"; set ylabel "Error";
p "data.txt" u 1:2 w l, "data.txt" u 1:3 w l
```

where you can run the first line (i.e., hit enter), and then run the second line (hitting enter again). Here the construct `{/Symbol D}` allows you to write Greek letters, the particular one is the upper case delta, i.e., Δ . We can also add titles to the key (or legend) of the plot, by using the `t` (title) operator as follows

```
set log x; set log y; set xlabel "{/Symbol D} x"; set ylabel "Error";
p "data.txt" u 1:2 w l t "Approximation 1", "data.txt" u 1:3 w l t "Approximation 2"
```

We can also add analytic functions to our plot, e.g.,

```
set log x; set log y; set xlabel "{/Symbol D} x"; set ylabel "Error";
p "data.txt" u 1:2 w l t "Approximation 1", "data.txt" u 1:3 w l t "Approximation 2", 0.01*sin(x) w l
```

Notice that the legend is automatically set to the analytic function name.

Once we are happy with how our plot looks, we can output it to a file. To do this we first decide the file type and set the filename. For example to output a pdf file type

```
set output "myplot.pdf"
```

where in double quotes you enter your filename with the corresponding extension. Then you need to set the terminal to be the corresponding one by typing

```
set term pdf
```

and then finally either type

```
replot
```

if you have already seen the plot in the gnuplot x-terminal, or to be safe just type the whole command again, i.e.,

```
set log x; set log y; set xlabel "{/Symbol D} x"; set ylabel "Error";
p "data.txt" u 1:2 w l t "Approximation 1", "data.txt" u 1:3 w l t "Approximation 2", 0.01*sin(x) w l
```

To exit gnuplot, either hit `ctrl+D` or type `exit` and hit enter. If the Greek letter is not rendered properly on the machine you first have to generate an eps file, and then convert it to pdf. This can be done by using the postscript terminal of gnuplot. To do this at the step where you set the output file name you type

```
set output "myplot.eps"
```

where in double quotes you enter your filename with the corresponding extension. Then you need to set the terminal to be the corresponding one by typing

```
set term post eps enhanced color
```

and finally replot

```
set log x; set log y; set xlabel "{/Symbol D} x"; set ylabel "Error";
p "data.txt" u 1:2 w l t "Approximation 1", "data.txt" u 1:3 w l t "Approximation 2", 0.01*sin(x) w l
```

where you can hit enter after the first line, and then continue onto a second line. All different commands can be in separate lines if you want. Hitting enter after the second line will give the file you want, and you can exit gnuplot. After generating the `myplot.eps` file, you can convert it to pdf by using the linux `ps2pdf` utility, i.e., typing

```
ps2pdf -dEPSCrop myplot.eps
```

To view the plot you can use the `evince` utility, by typing `evince myplot.eps` or `evince myplot.pdf`. You can further convert the pdf plot to png or jpg by using the linux `convert` (imagemagic) utility. To convert to png run

```
convert -density 300 myplot.pdf myplot.png
```

and to convert to jpg run

```
convert -density 300 myplot.pdf myplot.jpg
```

MTH6150: Introduction to the C/C++ programming language

Charalampos Markakis¹ & Vasileios Paschalidis²

¹ *School of Mathematical Sciences, Queen Mary University of London and*

² *Departments of Astronomy & Physics, University of Arizona, Tucson*

This note provides a basic introduction to the C/C++ programming language. Classes and object oriented programming will only be covered briefly, as they are not necessary for numerical analysis. The notes assume that you are running programs on a Linux terminal, but if you are using an IDE such as Visual Studio, Xcode or CLion, you may compile and run the code as described in lectures.

I. THE STRUCTURE OF A PROGRAM

The first program for beginners write is a "Hello World" program, which simply prints "Hello World" to your computer screen. Here is such a program

```
// My first C++ program
#include <iostream>

int main()
{
    std::cout << "Hello World!";
}
```

Line 1: is a comment. Comments in C/C++ follow after a double slash “//” and has no effect on the program. Comments are important to include in order to help the readability of a program.

Line 2: Lines beginning with a hash sign “#” are **directives**, i.e., a set of rules that are read and interpreted by a program called the **preprocessor**. In this case, the directive **#include <iostream>**, tells the preprocessor to include a section of standard C++ code, known as **header iostream**, that allows to perform standard input and output operations.

Line 3: A blank line. Blank lines have no effect on a program. Just like comments they help the readability of a program.

Line 4: **int main ()** This line initiates the declaration of a function. Essentially, a function is a group of program statements which are given a name: in this case, this gives the name "main" to the group of code statements that follow. We will discuss functions later.

The **main** function is a special, in that all C/C++ programs have it because it is the function called when the code is run.

Lines 5 and 7: The open brace ({) at line 5 indicates the beginning of the main function, and the closing brace (}) at line 7, its end. Everything between these braces is the function's body determining what happens when main is called. All functions use braces to indicate the beginning and end of their definitions. Note that the opening and closing braces could have been placed at the end of Line 4 and Line 5, respectively, but the code is more readable the way it is presented. In fact, we could have entered everything in a single line.

Line 6: **std::cout << "Hello World!";** This line is a C++ statement. A statement is an expression that produces an effect. Statements are executed in the same order that they appear within a function's body.

This statement has three parts: First, **std::cout**, which stands for the standard character output device (here this is the computer screen). Second, the insertion operator (<<): what follows after it is inserted into **std::cout**. Finally, a text within quotes ("Hello world!"), is what goes into the standard output.

Notice that the statement ends with a semicolon (;). Semicolons mark the end of statements. All C/C++ statements must end with a semicolon character. One of the most common syntax errors in C/C++ is forgetting to end a statement with a semicolon.

To compile and run the program first using the linux **mkdir** create a directory called **cpp_tutorial**. Go into that directory and follow these steps:

1. Create a file using the emacs editor and call it **main.C** that contains the program above in it. Save and close the file.
2. Type

```
g++ -c main.C
g++ -o myfirstprogram main.o
```

The first command will compile the human readable file `main.C` and produce the object file `main.o` (in machine language) without linking.

The second command will generate the executable `myfirstprogram` by linking any object files we have (which why it is called the linker). Use `ls` to ensure these files were created. Obviously you could have named the executable program `myfirstprogram` anything you desire.

We could have also compiled in one step as follows

```
g++ -o myfirstprogram main.C
```

which produces only the executable. But, the above example separates the compilation process from the linking process, which is typically important to understand when compiling code that is in multiple files.

3. Run the program by typing

```
./myfirstprogram
```

You should see `Hello World!` on your screen.

Lets now add another print statement in the program

```
// My second C++ program
#include <iostream>

int main()
{
    std::cout << "Hello World!";
    std::cout << "This is my second C++ program!";
}
```

Compile and run the program. What do you notice? Are the statements printed on two lines?

To make the statements appear on two lines we need to use the end line operator from the `stdout` library as follows

```
// My second C++ program
#include <iostream>

int main()
{
    std::cout << "Hello World!" << std::endl; // Prints Hello World!
    std::cout << "This is my second C++ program!" << std::endl; // Prints This is my second C++ program!
}
```

Compile and run the program. What do you notice this time? Notice that above we also added comments to the code to explain what that line does.

The code could have also been spread into more lines, e.g.,

```
// My second C++ program
#include <iostream>

int main()
{
    std::cout << "Hello World!"
              << std::endl;
    std::cout << "This is my second C++ program!"
              << std::endl;
}
```

and the result would be exactly the same. Make sure that you comment, indent and spread your codes over lines to make the code more readable.

II. IDENTIFIERS

Identifiers are the names of variables we wish to choose. They are sequences of one or more letters, digits, or underscore characters (_). Spaces, punctuation marks, and symbols cannot be part of an identifier. It is good practice to start identifiers with a letter (although they can start with an underscore). Valid identifiers can never start with a digit, and can never be any of the reserved strings that C++ uses, namely they cannot have names in the following group

`alignas, alignof, and, and_eq, asm, auto, bitand, bitor, bool, break, case, catch, char, char16_t, char32_t, class, compl, const, constexpr, const_cast, continue, decltype, default, delete, do, double, dynamic_cast, else, enum, explicit, export, extern, false, float, for, friend, goto, if, inline, int, long, mutable, namespace, new, noexcept, not, not_eq, nullptr, operator, or, or_eq, private, protected, public, register, reinterpret_cast, return, short, signed, sizeof, static, static_assert, static_cast, struct, switch, template, this, thread_local, throw, true, try, typedef, typeid, typename, union, unsigned, using, virtual, void, volatile, wchar_t, while, xor, xor_eq`

Use different identifiers for different variables. Remember that C/C++ is a case sensitive language, thus the variable identifier `Press` is not the same as the name `press`.

III. DATA TYPES

The values of variables are stored somewhere in the computer memory in machine language (zeros and ones). Our code does not need to know the storage location; it can simply refer to the variable by its identifier. What the code requires is the kind of data stored in the variable. Storing an integer is not the same as storing a letter/string or a floating-point number. The C/C++ fundamental data types are

1. Character types: These represent a **single** character, such as 'C' or '&'. The most basic type is `char`, which is a one-byte character. There are larger size character types for wider characters, and we will also use strings.
2. Integer types: These can store an **integer** number value, such as 3 or 15313. These can either be signed or unsigned, i.e., support or not negative values.
3. Floating-point types: These store **real** values, such as 3.1415 or 0.03. They have different levels of precision, i.e., how many significant digits are accurate.
4. Boolean types: The boolean type, `bool` type in C++, can be either `true` or `false`.

A list of the most common C++ fundamental types is as follows

1. Character types:
 - `char`: 1 byte in size (8 bits)
 - `char16_t`: 16 bits
 - `char32_t`: 32 bits
 - `wchar_t`: Represents the largest supported character set.
2. Integer types (by default signed):
 - `int`: 1 byte in size (16 bits), this integer type will be the most common we will use
 - `long int`: 32 bits
 - `long long int`: 64 bits
3. Integer types (unsigned):
 - `unsigned int`: 1 byte in size (16 bits)
 - `unsigned long int`: 32 bits
 - `unsigned long long int`: 64 bits
4. Floating-point types:

- **float**: single precision (typically 32 bits)
- **double**: double precision (typically 64 bits), this type will be the one we will use
- **long double**: precision no less than double

Increasing the precision makes for slower calculations.

Apart from the above data types we also have `void` types, which are none of the above (no storage). Note that 16-bit arithmetic means we can have $2^{16} = 65536$ representable values, 32-bit arithmetic can have $2^{32} = 4,294,967,296$ representable values, 64-bit arithmetic can have $2^{64} = 18,446,744,073,709,551,616 \sim 1.8 \times 10^{19}$ representable values. For example for signed integers 16-bit arithmetic implies that only the integer numbers in the interval $[-32768, 32767]$ can be represented. On the other 32-bit arithmetic implies that only the integer numbers in the interval $[-2147483648, 2147483647]$ can be represented. For most purposes 32-bit integers will suffice. We will discuss later in the course how floating point numbers are represented.

IV. DECLARATION OF VARIABLES

C++ requires *every* variable to be declared with its type before it is used for the first time. This informs the compiler the size to reserve in memory for the variable and how to interpret its value. Unlike other languages that require a variable to be declared at the beginning of a program, in C/C++ you can introduce (declare) a variable anywhere in the program as long as it is declared before it is used for the first time. Here is a simple program of how we declare the variables.

```
#include <iostream>
using namespace std;

int main ()
{
    // variable declaration
    int a, b;
    int c;

    // Value assignment:
    a = 5;
    b = 2;
    // Subtraction of b from a
    c = a - b;

    // Print the result;
    cout << "Subtracting " << b << " from " << a << " yields " << c << endl;

    /*
    Notice that we did not use ‘std:cout’ because
    of the statement ‘using namespace std;’ at the beginning. Also
    notice that this block is a comment because it is contained within
    "slash asterisk asterisk slash". This is a quick way to add multiline
    comments to your program without having to type // all the time
    */

    // terminate the program:
    return 0;
}
```

Create a file using emacs, type in the above program and compile your file. Then execute the program. What is the output?

Whenever we declare a variable, it is good practice to initialize the value of that variable. C++ offers multiple equivalent ways for initializing variables almost at the same place where they are declared. The following program shows all these ways

```

#include <iostream>
using namespace std;

int main ()
{
    // variable declaration and initialization
    int a=5, b(3);
    int c{0};

    // Subtraction of b from a
    c = a - b;

    // Print the result;
    cout << "Subtracting " << b << " from " << a << " yields " << c << endl;

    // terminate the program:
    return 0;
}

```

Implement the above program, compile it and execute it. Choose different initialization values for a, b, c and confirm that the code yields the right outcome.

V. BASIC STRINGS

Strings are not fundamental data types, but are a compound type that has its own class. As such we need to include the string class in our program if we want to use C++ strings. This is achieved by adding the `#include <string>` at the top of the program as in the following simple program

```

#include <iostream>
#include <string>
using namespace std;

int main ()
{
    string myfirststring;
    myfirststring = "This is my first string\n";
    cout << mystring;
    return 0;
}

```

Notice the new line “\n” operator at the end of the string. The above program has the same outcome as the following

```

#include <iostream>
#include <string>
using namespace std;

int main ()
{
    string myfirststring;
    myfirststring = "This is my first string";
    cout << myfirststring << endl;
    return 0;
}

```

Strings can be initialized and can change their values after they have been initialized exactly as other data types can. Strings can also be added (concatenated is the proper verbiage), as in the following program

```

#include <iostream>
#include <string>
using namespace std;

int main ()
{
    string myfirststring;
    string mysecondstring;
    myfirststring = "This is my first string ";
    mysecondstring = "and this is my second string";
    cout << myfirststring+mysecondstring << endl;
    return 0;
}

```

VI. CONSTANTS

In addition to the previous data types, we can also have constants, i.e., expressions with a fixed value that cannot be changed. These can be achieved either using the `const` type or Preprocessor definitions. Here is an example with `const` type

```

#include <iostream>
using namespace std;

const double PI = 3.14159265359;
const char newline = '\n';

int main ()
{
    double a=3.0;           // radius of disk
    double disk;

    disk = PI * a * a; // area of disk, the asterisk stands for multiplication
    cout << " Disk of radius " << a << " has an area of " << disk;
    cout << newline;
}

```

and an example with preprocessor definitions

```

#include <iostream>
using namespace std;

#define PI 3.14159265359
#define NEWLINE '\n'

int main ()
{
    double a=3.0;           // radius of disk
    double disk;

    disk = PI * a * a;
    cout << disk;
    cout << NEWLINE;
}

```

We can also increase the precision of `out` by using the `iomanip` class as in the following program

```

#include <iostream>

```



```

#include <iomanip>
using namespace std;

#define PI 3.14159265359
#define NEWLINE '\n'

int main ()
{
    double a=3.0;           // radius of disk
    double disk;

    disk = PI * a * a;
    cout << setprecision(12) << disk;
    cout << NEWLINE;
}

```

VII. BASIC OPERATORS

- Assignment operator (=): `a=3` assigns the value of 3 to a. The expression `b=a`, assigns the value of a to b, but never the value of b to a.

In C++ you may see an expression like `y = 3 + (a = 23);`, in which the operation in the parenthesis is performed first, i.e., the value of 23 is assigned to a, and then the 23 and 3 are added and the result assigned to y, so that `y = 26` in the end.

- Arithmetic operators: `+` is the addition operator, `-` the subtraction operator, `*` the multiplication operator, `/` the division operator, and `%` the modulo operator.)
- Compound assignment operators (`+=`, `-=`, `/=`, `*=`, `%=`) are better explained with their equivalent statements

```

x+=2      <=>    x = x + 2
x-=3      <=>    x = x - 3
x*=10     <=>    x = 10*x
x%=2      <=>    x = x%2
x*=y+1    <=>    x = x*(y+1)

```

- Increment/decrement operators (`++`, `-`): the following statements are all equivalent

```

++x;
x+=1;
x=x+1;

```

Similarly for the decrement operator.

Exercise: Form groups of 2-4 people and think about the outcome of the following operations. What are the values of x and y after the operations are completed in either case?

```

x=10;
y=++x;

```

and

```

x=10;
y=x++;

```

Write a program and compute x and y in both cases. Are your initial expectations met?

- Comparison operators (==, !=, >, <, >=, <=):

```

==    equal to
!=    not equal to
>     greater than
<     less than
>=    greater than or equal to
<=    less than or equal to

```

- Logical operators (!, &&, ||):

```

!      Boolean NOT
&&     Boolean AND
||     Boolean OR

```

For example the following is false

```
( (7 == 7) && (1 > 19) )
```

because the first is true but the second is false. While the following evaluates to true

```
( (7 == 7) || (1 > 19) )
```

because of the OR operator, we need only one of the two relations to be true.

- Conditional ternary operator (?): this has the following structure
`condition ? result1 : result2`, i.e., if the `condition` is true then you obtain `result1`, else you obtain `result2`. Look at the following program and figure out its output. Discuss the program with your group and try to reason the outcome. Then code it up and try it yourself.

```

#include <iostream>
using namespace std;

int main ()
{
    int x,y,z;

    x=2;
    y=7;
    z = (x>y) ? x : y;

    cout << z << '\n';
}

```

- Comma operator (,): The comma operator is used to separate two or more expressions when only one expression is expected. To see the result implement, compile and execute the following program

```

#include <iostream>
using namespace std;

int main ()
{
    int a,b;

    a = (b=7, b+8);
    cout << 'a=' << a << '\n';
}

```

What is the output, i.e., the value of `a`? Discuss within your group what the comma operator does. What operation is performed first?

- Precedence of operators: 1) postfix increment / decrement is from left to right, 2) prefix increment / decrement is from right to left, 3) multiply, divide, modulo are always from left to right, 4) addition, subtraction from left to right. It is not trivial to remember the precedence of operators especially since C++ has many operators. Always test your expressions for the precedence of operators, and use parentheses, because expressions in parentheses are evaluated first.

VIII. INPUT/OUTPUT FROM STANDARD IN/OUT

- Input of numerical values:

We have already seen the `cout` operator. There exists an input operator `cin` that reads values from standard input (which is your keyboard). This is demonstrated with the following example

```
#include <iostream>
using namespace std;

int main ()
{
    double a;
    cout << "Please enter a real number: ";
    cin >> a;
    cout << "The value you entered is " << a;
    cout << " and its square is " << a*a << endl;
    return 0;
}
```

Implement the above program, compile it and execute it. Change the precision of `cout`, and see how the output is affected. Do you need to set the precision in both `cout` operations?

To force scientific notation you can use `cout << setprecision(15) << scientific << "The value you entered is " << a;`. Do you need to enter to set `scientific` in both `cout` operations?

- Input of strings:

The standard header `<sstream>` defines a data type called `stringstream`. `stringstream` allows the extraction or insertion operations from/to strings in the same way as they are performed on `cin` and `cout`. It is extremely useful for converting strings to numerical values and vice versa. The following code demonstrates this.

```
#include <iostream>
#include <string>    // We have added <string>
#include <sstream>   // and <sstream>
using namespace std;

int main ()
{
    string mystr;
    double length=0,width=0;
    double area=0;

    cout << "Enter rectangle length in ft: ";
    getline (cin,mystr);
    stringstream(mystr) >> length;
    cout << "Enter rectangle width in ft: ";
    getline (cin,mystr);
    stringstream(mystr) >> width; // Notice that we reused mystr

    cout << "Rectangle area: " << length*width << " ft^2" << endl;
```

```
    return 0;
}
```

Implement the above program, and write a program that reads also the height and computes the volume of a room.

IX. INPUT/OUTPUT WITH FILES

C++ has 3 classes to perform input/output to/from files:

1. ofstream: To write on files
2. ifstream: To read from files
3. fstream: To both read and write from/to files.

The following program first writes a text and then reads it and prints it on the screen.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main () {
    ofstream myfl ("mytext.txt"); // We open (i.e. create) the file and name it mytext.txt
    if (myfl.is_open()) // This if statement tests if the file exists
    {
        // Now we write 3 lines to the file
        myfl << "Hello my first name is John.\n";
        myfl << "My middle name initial is J.\n";
        myfl << "My last name initial is Smith\n";
        myfl.close(); // Here we close the file
    }
    else cout << "Unable to open file";

    // Now lets read the file we wrote
    string line;
    ifstream myfile ("mytext.txt");
    if (myfile.is_open()) // Test if the file exists
    {
        while ( getline (myfile,line) ) // the while structure here reads all lines, getline reads these lines
        {
            cout << line << '\n'; // We print on screen line by line
        }
        myfile.close();
    }

    else cout << "Unable to open file";

    return 0;
}
```

Implement the previous program. Compile it and run it. Does it produce the file mytext.txt? Does the file have the expected content? Does the screen output agree with the file content? Use the same structure to introduce six doubles a1=11, a2=21, a3=31, b1=21, b2=22, b3=33 and output them in two columns to file which has the following structure

```
a1 b1
a2 b2
a3 b3
```

Read the file and print it on screen. Discuss within your group how to implement this program, and then each one of you work independently to implement it. Ensure the screen output is consistent with the file.

X. FLOW CONTROL AND LOOPS

When we write code in any language some of the most important statements are those controlling the flow and those generating loops. We will demonstrate each of these with specific examples.

• Flow control

1. *if and else*: the syntax is `if (condition1){ statement1} else if (condition2){ statement2}... else {statement n}`. The following program demonstrates the use

```
// if else, testing for positive or negative?
#include <iostream>
using namespace std;

int main ()
{
    int n;
    cout << "Please enter an integer number: ";
    cin >> n;

    if (n>0){
        cout << "You entered a positive integer\n";
    }
    else if (n<0){
        cout << "You entered a negative integer\n";
    }
    else{
        cout << "You entered 0\n";
    }
}
```

Implement, compile and execute the previous program. Run the program entering positive, negative and a 0 value to make sure the program works as expected.

2. *switch*: This is used to check for specific values. The syntax is

```
switch (expression)
{
    case constant1:
        group-of-statements-1;
        break;
    case constant2:
        group-of-statements-2;
        break;
    .
    .
    .
    default:
        default-group-of-statements
}
```

Here is a program demonstrating the use of `switch`

```
// switch
#include <iostream>
using namespace std;
```

```

int main ()
{
    int n;
    cout << "Please enter an integer number: ";
    cin >> n;

    switch (n)
    {
        case 1:
            cout << "You entered the integer 1\n";
            break;
        case 0:
            cout << "You entered zero\n";
            break;
        default:
            cout << "You entered a number I do not understand\n";
    }
}

```

Implement, compile and execute the previous program.

Of course switch in the previous program could have been replaced with if statements as follows

```

// if else instead of switch
#include <iostream>
using namespace std;

int main ()
{
    int n;
    cout << "Please enter an integer number: ";
    cin >> n;

    if(n==1){
        cout << "You entered the integer 1\n";
    }
    else if (n==0){
        cout << "You entered zero\n";
    }
    else{
        cout << "You entered a number I do not understand\n";
    }
}

```

• Loops and flow control

1. *The for loop*: this iterates over integers. The syntax is `for (initialization; condition; increase/decrease) statement`; For example

```

#include <iostream>
using namespace std;

int main ()
{
    for (int i=1; i<=10; i++) { // Notice that we can declare i in the loop, also ++i would be the same
        if (i<10)
        {
            cout << i << ", ";
        }
        else
        {

```

```

        cout << i << ". ";
    }
}
cout << "\n";
}

```

Exercise: Based on the above program, write a program that also prints the value of *i* after the loop ends. Does the program compile? Why do you think it does not? Discuss with your group what the issue may be. Once you fix the issue and compile the code run it. Is the value of *i* after the loop 10 or 11?

Now, write a program that uses a for loop to sum the first 10 positive integer numbers, i.e., the sum of numbers 1,2,3,...,10. First compute the result analytically, then implement a program that achieves the goal and compare your program to your analytic answer.

Note that we can have nested loops as in the following program

```

#include <iostream>
using namespace std;

int main ()
{
    for (int i=1; i<=10; i++) {
        for (int j=1; j<=10; j++) {

            if (i==j)
            {
                cout << "i=j= " << i << endl;
            }
        }
    }
}

```

Jump statements: we will sometimes need to exit loops. This can be achieved with jump statements. In C++, these are the **break** and **continue** statements. The following example codes demonstrate how these work

```

// break loop example
#include <iostream>
using namespace std;

int main ()
{
    for (int i=1; i<11; i++){
        for (int j=1; j<11; j++){

            cout << i << ", " << j << endl;
            if (i==3)
            {
                cout << "Can you fight!";
                break;
            }
        }
    }
}

```

The continue statement is demonstrated with the following code

```

// continue loop example
#include <iostream>
using namespace std;

int main ()
{

```

```

for (int i=1; i<10; i++) {
    if (i==3){
        cout << "\nI am skipping one number\n";
        continue;
    }
    cout << i << ", ";
}
cout << "Done counting!\n";
}

```

C++ has a `goto` statement, but I do not encourage its use because it does not make for easily readable code.

2. *The while loop:* The while loop has simple syntax: **while (expression) statement** as demonstrated by this code

```

// countdown using while
#include <iostream>
using namespace std;

int main ()
{
    int i = 10;

    while (i>0) {
        cout << i << ", ";
        --i;
    }

    cout << i << "!\n";
}

```

Notice what the value of `i` is after the end of the while statement.

3. *The do-while loop:* the do while loops has the following syntax **do statement while (condition);**. It behaves like a while-loop, but the condition is probed *after* the execution of **statement** and not *before*. The following simple code demonstrates its use

```

// echo machine
#include <iostream>
#include <string>
using namespace std;

int main ()
{
    string str;
    do {
        cout << "Enter something: ";
        getline (cin,str);
        cout << "You entered: " << str << '\n';
    } while (str != "Hello");
}

```

This code will keep asking you to enter something, until you enter “Hello”. Try it out. If you enter “hello” does the program terminate?

XI. FUNCTIONS

Functions are part of a code that perform a certain task. They help make codes more modular, remove repetitive pieces of code and make code more transparent and readable. The typical syntax for a function is **type name (argument1, argument2, ...) statements**. Some of the function arguments can be input, some can be output (depending on the function type). But, a function can have pretty much as many arguments as you wish. There are functions that have a type, e.g., `int`, `float`, `double`, and functions with no type.

A. Functions with type

Functions which have a type will return a value. For example, a function of type `int`, `float`, or `double` will return an integer, float or double value, respectively. Lets look at the following simple example code

```
// my first function
#include <iostream>
using namespace std;

int multiplication (int a, int b)
{
    int result;
    result=a*b;
    return result;
}

int main ()
{
    int product;
    product = multiplication (15,5);
    cout << "The product is " << product << endl;
    int a=3,b=8;
    product = multiplication (a,b);
    cout << "The new product is " << product << endl;
}
```

Notice that a function can be called multiple times and you can also pass variable to it. Also notice that the variables `a`, `b` in the multiplication function are *local* to that routine alone. They are basically dummy variables, and completely separate from the variables `a`, `b` in the main routine, which are global variables.

Moreover, notice that functions **must** be **declared** *before* the main function. However, they do not have to be defined before. For example, the previous piece of code can be written as follows

```
// my first function
#include <iostream>
using namespace std;

int multiplication (int a, int b);

int main ()
{
    int product;
    product = multiplication (15,5);
    cout << "The product is " << product << endl;
}

int multiplication (int a, int b)
{
    int result;
    result=a*b;
    return result;
}
```

Compile and execute both programs above. Finally, the functions do not even have to be in the same file. Lets now learn how to build programs written over different files, and how to compile them.

First, create a file called, e.g. `main.C` and in it add the following code

```
// my first code in multiple files
#include <iostream>
#include "header.h"
```

```
using namespace std;

int main ()
{
    int product;
    product = multiplication (15,5);
    cout << "The product is " << product << endl;
}
```

save and close the main.C file. Now create a file called, e.g., functions.C (the file that will contain your functions) and inside paste the following code

```
int multiplication (int a, int b)
{
    int result;
    result=a*b;
    return result;
}
```

save and close the function.C file. Finally, create a file called header.h (this file will contain the declarations of all your functions) and inside paste the following line

```
int multiplication (int a, int b);
```

Note! This could also be

```
int multiplication (int, int);
```

So, now your code is in main.C, functions.C and header.h. Place all these files in the same directory. To compile the code type

```
g++ -c functions.C main.C
g++ -o function functions.o main.o
```

This will generate the executable `function` that you can run and obtain the same result as the previous program. Try it out!

Exercise: Code a routine for adding two numbers and add to the file functions.C. Then declare the routine in your header.h file and call the routine in the main.C file. Compile the code and run it outputting both the product and the sum of the two integers a and b.

Now that you know how to compile a code that is spread over multiple files. Go back to the last section of the introduction to Linux tutorial and read about Makefiles. Follow the instructions in that section and prepare your own makefile that compiles the code in your main.C, header.h and functions.C file.

B. Functions without type

There are also functions that have no type, and do not return a value. These are the `void` functions. An example is given here

```
// void function example
#include <iostream>
using namespace std;

void myvoidfunction ()
{
    cout << "This is a void function with no argument!";
}
```

```

int main ()
{
    myvoidfunction ();
}

// void function example2
#include <iostream>
using namespace std;

void addition (int a, int b, int &c)
{
    c=a+b; // This function adds a, b and assigns the result to c.
}

int main ()
{
    int a,b,c(0); // c is initialized to 0.
    a=3,b=5;
    cout << "The value of c before addition is " << c << endl;
    addition (a,b,c);
    cout << "The new value of c is " << c << endl;
}

```

In this example you see for the first time that in the definition/declaration of the `addition` function we have `&c` and not `c`. We will discuss this further below. For now implement the above program and run it. Then define the routine without the `&`, i.e., `addition (int a, int b, int c)` and run the code. **Does it work as intended?**

Note: You may have noticed that the type of `main` is `int`, but in many of our examples we did not actually include a return statement for `main`. This is because the compiler assumes the function ends with an implicit return statement: `return 0;`.

C. Passing arguments by value and by reference

Variables can be passed by value or by reference. If they are passed by value the value cannot be changed in the routine. If the value is passed by reference, the value of the variable can change. The following code passes variables to the function by reference (using the `&` operator)

```

// passing variables by reference
#include <iostream>
using namespace std;

void triple (int& a, int& b, int& c)// The variables are passed by reference
{
    a*=3;
    b*=3;
    c*=3;
}

int main ()
{
    int i=1, j=3, k=7;
    triple (i, j, k);
    cout << "i=" << i << ", j=" << j << ", k=" << k;
}

```

Had the definition been `void triple (int a, int b, int c)`, the variables would be passed by value and the `cout` statement would print the same values as before the call to `triple`. When passing by value essentially the

variables are treated as parameters and we cannot change their values in the function. In these cases we can also pass the variables with default values, e.g.,

Note: Functions can be of string type, e.g.,

```
string cat (string& a, string& b)
{
    return a+b; // concatenates two strings
}
```

Add the above function in a program, pass two strings to it, and call it to see the result.

D. Recursive functions

These are functions that can call themselves as in this example

```
// sum calculator
#include <iostream>
using namespace std;

int sum (int i)
{
    if (i > 1)
        return (i + sum (i-1));
}

int main ()
{
    int j = 10;
    cout << "sum of first " << j << " integers greater than 0 is "<< sum (j);
    cout << "\n";
    return 0;
}
```

XII. ARRAYS

Arrays are series of values of the same type placed in chunks of memory that can be individually accessed by adding an index to a unique identifier. You can think of them as multidimensional mathematical vectors where the index accesses the vector components. The following example code demonstrates how one declares arrays, initializes them and accesses their values.

```
// My first array code
#include <iostream>
using namespace std;
// Here the size of the array arr1 is 5, and is dictated by the initialization
int arr1 [] = {6, 6, -7, 4, -1};
int n, result=0;
// Notice that the fact we have defined the array arr1, the integers n and result before
// the main routine, implies that they are global variables accessible by
// all functions in our program without having to pass them as arguments to
// functions

int main ()
{
    for ( n=0 ; n<5 ; ++n )
    {
        result += arr1[n]; // notice that arr1[n] accesses the value of the array
```

```

}
cout << result << endl;
// Below we declare the array arr2 to have size 10
// we do not access the value of the 10th component
int arr2[10];

for (int i=0 ; i<10 ; i++ )
{
    arr2[i] = 3*i*i-5*i-2; // notice that here we initialize arr1
}

// Lets sum the components of arr2 this time
result = 0;
for (int i=0 ; i<10 ; i++ )
{
    result += arr2[i];
}
cout << result << endl;
return 0;
}

```

Notice that the first value of the array `arr1` is accessed with index 0, i.e., `arr1[0]` and the last one `arr1[4]` and not `arr1[5]`. If you attempted to access `arr1[5]` you would try to access an element in the memory that does not exist.

Implement and run the above program. Discuss within your group why `result` was set to 0 before summing up the elements of `arr2`. What would the code output the second time if we did not set `result=0`? Test your expectation.

Apart from 1D arrays we can also have multidimensional arrays. For example, the following code shows a 2D arrays, which you can think of as a matrix

```

#include <iostream>
using namespace std;
#include <math.h> // Library for intrinsic C++ functions like cos, sin, log, pow etc.
#define cols 1000
#define rows 1000

int main ()
{
    double mat[rows][cols];
    int i,j;
    double sum=0;
    for (i=0; i<rows; i++)
        for (j=0; j<cols; j++)
        {
            mat[i][j]=(i+1.)*(j+1.); // here we assign values to the 2D array elements
        }

    // Lets sum up the sin^2 and cos^2 of the matrix elements to the power 4
    for (i=0; i<rows; i++)
        for (j=0; j<cols; j++)
        {
            sum+=cos(pow(mat[i][j],4))*cos(pow(mat[i][j],4))+sin(pow(mat[i][j],4))*sin(pow(mat[i][j],4));
        }

    cout << sum << endl;
}

```

Here you see that C++ has intrinsic functions, like `cos`, `sin`, `log`, etc. To use this you need the `#include <math.h>` directive. Also you see for the first time the `pow` function which essentially exponentiates a value. The `pow` function

is computationally demanding and for efficiency it is best to be used for non-integer powers only. Implement and execute the program above.

Like variables, arrays can be passed as arguments to functions. However, we cannot pass the entire set of values. What we can pass is the address of the array in the memory (we practically pass pointers which we will discuss below). The following example demonstrates how we can pass arrays as arguments in functions.

```
// arrays as arguments in functions
#include <iostream>
#include <math.h>
using namespace std;

void sumarray (int arr[], int length) {
    int sum=0;
    for (int i=0; i<length; ++i){
        sum+=arr[i];
    }
    cout << sum ;
    cout << '\n';
}

int main ()
{
    int n1=10,n2=20;
    int arr1[n1], arr2[n2];

    for (int i=0;i<n1;i++){
        arr1[i] = i*i;
    }
    for (int i=0;i<n2;i++){
        arr2[i] = pow(2,i);
    }

    sumarray (arr1,n1);
    sumarray (arr2,n2);
}
```

In a function declaration, it is also possible to include multidimensional arrays. The format for a 3D array

```
type name[][size2][size3]
```

For example, a function with a multidimensional array as argument could be

```
void myfunction (int threeDarray[][10][20])
```

Notice that the first brackets [] are empty, while the following ones specify the respective dimension size. This syntax is necessary. But, we can do better with pointers that we discuss next.

XIII. POINTERS

In a C++ program, you can treat the memory of a computer as a succession of memory cells and each with a *unique* address, such that each cell can be located in the memory by use of its unique address. For instance, the cell with address 2034 is always after cell with address 2033 and before the cell with address 2035, it is also 2000 cells after the cell with address 34. When we declare a variable in C++, the memory needed to store its value is assigned a specific address. The quantity that carries the information of what the address is is called a *pointer*: it points to the address.

- **Address-of operator (&):** The address of a variable can be obtained by use of the address-of operator, which is invoked by prepending an ampersand symbol (&) to the variable, i.e., `addrvar = &var`; This operation assigns the address of variable `var` to `addrvar`. Hence, `addrvar` is not assigned the value of the variable `var`. Let consider the following snippet of code

```
var = 3;
addrvar = &var;
newvar = myvar;
```

If we assume that the address of `var` is 2034, then the value of `addrvar` is 2034, while the value of `newvar` is 3.

- **Dereference operator (*):** pointers can be used to access the value of the variable they point to. This is achieved by the dereference operator (*), which can be read as “value pointed to by”. In our previous example, the value of `*addrvar` would be 3.

A. Declaring pointers

A pointer is generally declared as `type * name`; The following program demonstrates the declaration of pointers and their use.

```
// pointers
#include <iostream>
using namespace std;

int main ()
{
    int value1 = 5, value2 = 15;
    int * p1, * p2;

    p1 = &value1; // p1 = address of value1
    p2 = &value2; // p2 = address of value2
    *p1 = 10;      // value pointed to by p1 = 10, this will also be the value of value1
    *p2 = *p1;      // value pointed to by p2 = value pointed to by p1, thus 10 is the value of value2
    cout << "value1 is " << value1 << '\n';
    cout << "value2 is " << value2 << '\n';

    p1 = p2;        // p1 = p2 (value of pointer is copied), these are the addresses, now p1 points to address of p2
    *p1 = 20;        // value pointed to by p1 = 20, thus the value of value2 is 20.

    cout << "value1 is " << value1 << '\n';
    cout << "value2 is " << value2 << '\n';
    return 0;
}
```

The above example demonstrates the use of the address-of and dereference operators. Notice that the values of pointers can be assigned to other pointers. Discuss the above program with your group. Make sure you understand it. Note that a declaration of the form `int * p1, p2`; would mean that `p1` is a pointer, but `p2` is not a pointer, but a regular integer.

B. Pointers and arrays

Arrays are related to pointers, work like pointers to their first elements, and, an array can always be implicitly converted to a pointer of the same type. For instance, consider the following declarations:

```
int array [3];
int * p;
```

The assignment: `p = array;` is completely valid. In other words array is a pointer, which represents 3 addresses in the memory. Lets consider now the following code

```
// pointers to arrays
#include <iostream>
using namespace std;

int main ()
{
    int array[5];
    int * p;
    p = array; *p = 10; // Here p points to element 1 of array, then the value pointed to by
                        // p, i.e., array[0] is set to 10.
    p++; *p = 20;      // we now increment the address of p, i.e., it takes on the value
                        // of the address of array[1], hence we set array[1]=20
    p = &array[2]; *p = 30; // now p is the address of array[2], thus we set array[2]=30
    p = array + 3; *p = 40; // now p is the address of array[0]+3, thus we set array[3]=40
    p = array; *(p+4) = 50; // now p is the address of array, thus we set the value pointed by
                        // the address of (p+4), i.e., array[4]=50

    for (int n=0; n<5; n++)
        cout << array[n] << ", ";
    return 0;
}
```

Thus, the name of an array is a pointer to the first element of the array.

The main difference between pointers and arrays is that pointers can be assigned new addresses, but arrays cannot.

We previously discussed that square brackets (`[]`) specify the index of an element of the array. Here we learn that these brackets are in fact a dereferencing operator known as *offset* operator. They dereference the variable they follow just as `*` does to a pointer, but they also add the number between brackets to the address being dereferenced. For example:

```
b[4] = 0;      // b [offset of 4] = 0
*(b+4) = 0;    // pointed to by (b+4) = 0
```

The above two expressions are equivalent.

C. Pointers to functions

C++ allows pointers to functions. We use these to pass a function as an argument to another function which makes for very modular codes. Pointers to functions are declared with the same syntax as a regular function declaration, except that the name of the function is enclosed between parentheses (`()`) and an asterisk (`*`) precedes the name. The following example code shows how this is done

```
// pointer to functions
#include <iostream>
using namespace std;

int addition (int a, int b) // addition function
{ return (a+b); }

int subtraction (int a, int b) // subtraction function
{ return (a-b); }

int operation (int x, int y, int (*func)(int,int)) // general function to which we pass another func.
{
    int g;
    g = (*func)(x,y);
    return (g);
}
```



```

}

int main ()
{
    int m,n;
    int (*minus)(int,int) = subtraction;// pointer to subtraction

    m = operation (8, 3, addition);
    n = operation (33, m, minus);
    cout <<n << "\n";
    n = operation (33, m, subtraction);
    cout <<n << "\n";
    return 0;
}

```

In the example above, `minus` is a pointer to a function that has two parameters of type `int`, and here it is initialized to point to the function `subtraction`: `int (* minus)(int,int) = subtraction;`. Then in the last line we pass the function `subtraction` directly to `operation`.

XIV. DATA STRUCTURES

A data structure consists of data elements grouped together under one name. These data elements, known as members, can have different types and different lengths. Data structures can be declared in C++ using the following syntax:

```

struct type_name {
    member_type1 member_name1;
    member_type2 member_name2;
    member_type3 member_name3;
    .
    .
} object_name;

```

The `object_name` is optional. Typically one can define a struct as e.g.

```

struct product {
    int weight;
    double price;
}

```

and then declare data structs as

```
product kiwi, banana, pineapple;
```

then each of the `kiwi`, `banana`, `pineapple` struct will have `weight` and `price` members that can be accessed using the “.” operator, i.e.,

```

kiwi.weight
kiwi.price
banana.weight
banana.price
pineapple.weight
pineapple.price

```

Data structures are extremely useful in writing modular code, as well as in passing multiple variables as arguments to a function by using only one name! For example, consider a group of parameters necessary for computing the gravitational and electric force between two charges: `charge1`, `charge2`, `mass1`, `mass2`, `distance`.

```

// using structures
#include <iostream>
using namespace std;

struct params { // Here we declare the struct
    double q1,q2;
    double m1,m2;
    double dis;
};

void gravitational_force(params sys,double &Fg, double G){
    Fg=-G*sys.m1*sys.m2/(sys.dis*sys.dis);
}

void electrostatic_force(params sys,double &Fe, double k){
    Fe=k*sys.q1*sys.q2/(sys.dis*sys.dis);
}

int main ()
{
    double k=8.85418782e-12; // Coulomb's law constant in SI
    double G=6.674e-11; // Newton's constant in SI.
    params ep; // declaration of params struct for an electron-proton (ep) system

    ep.q1=-1.60217662e-19; // e charge in Cb
    ep.q2=1.60217662e-19; // proton charge in Cb
    ep.m1=9.10938356e-31; // e mass in kg
    ep.m2=1.6726219e-27; // p mass in kg
    ep.dis=0.01; // 1cm=0.01m

    double Fg, Fe;

    gravitational_force(ep,Fg,G);
    electrostatic_force(ep,Fe,k);

    cout << "The ratio of grav. to elec. force between a proton and electron is " << Fg/Fe << endl;

    return 0;
}

```

XV. EXERCISE

Approximating $\ln(2)$: We know that $\ln(2)$ is the limit of the infinite series:

$$\ln(2) = \sum_{n=1}^{\infty} \frac{(-1)^{(n+1)}}{n} = \left(1 + \frac{1}{3} + \frac{1}{5} + \frac{1}{7} + \dots\right) - \left(\frac{1}{2} + \frac{1}{4} + \frac{1}{6} + \dots\right) \quad (1)$$

Write a program that approximates $\ln(2)$ to within an accuracy of 1 part in 10^6 . How many terms in the series do you need to achieve this accuracy?

Here is a flow chart/set of steps:

1. Introduce two estimates for $\ln(2)$, call them `double ln2a, ln2b`. Also introduce an integer `n`. Also introduce a tolerance variable to check the accuracy of the estimate for $\ln(2)$, let's call it `double TOL=1.e-6`.
2. Initialize `ln2a` and `ln2b` to 0.

3. Create a loop over n (can be a while or a for loop), with n starting at value 1. If you choose a for loop you need to consider the maximum number of iterations in the loop. Notice that the series is a slowly converging one because each term is order $1/n$. Thus, for an accuracy of $1e-6$, you would need more than 10^6 steps.
4. **Inside** the loop set (form groups of 2 and reason with your classmates why we are doing the following)


```
ln2a=ln2a + pow(-1,n+1)/n
ln2b=ln2a + pow(-1,n+2)/(n+1)
```

Now you have two estimates for the $\ln(2)$ from two consecutive terms in the series.

5. **Inside** the loop check for the accuracy of the estimate. Given that the term with $n+1$ has a smaller contribution to the sum than the term n (why is that?), this implies that $\ln2b$ is a more accurate estimate for $\ln(2)$ than $\ln2a$. Thus, check for the relative error between these two estimates and exit the loop when the relative error is smaller than the desired tolerance, e.g.

```
if(((abs(ln2b-ln2a))/ln2b < TOL){ break; }
```

Note: To use the C++ `abs` function you need to include the `cmath` library `# include <cmath>`

6. **Outside** the loop, output $\ln2b$ and the number of terms in the series that you used to achieve the desired accuracy in calculating $\ln(2)$.

How many terms in the expansion do you think you would need to achieve an accuracy of 10^{-16} ?

Another representation for $\ln(2)$ is the limit of the infinite series:

$$\ln(2) = \sum_{n=1}^{\infty} \frac{1}{2^n n} \quad (2)$$

How many terms in the expansion do you need to achieve an accuracy of $1e-16$? How does this compare with the previous estimator? Why do you think that is?

MTH6150: Roundoff Error

Charalampos Markakis¹ & Vasileios Paschalidis²

¹ *School of Mathematical Sciences, Queen Mary University of London and*

² *Departments of Astronomy & Physics, University of Arizona, Tucson*

This note provides a basic understanding of roundoff error.

I. REPRESENTATION OF REAL NUMBERS ON THE COMPUTER

Real numbers are not fully represented on computers. The IEEE standard for represented 64-bit floating point numbers (a double in C/C++) is

$$(-1)^s 2^{(c-1023)} (1 + f), \quad (1)$$

where s is called the sign, c the characteristic and f the mantissa. The representation of a real number is

$$\begin{array}{c} \text{sign} \quad \text{characteristic} \quad \text{mantissa} \\ \hline \boxed{1\text{bit}} \quad \boxed{11\text{bits}} \quad \boxed{52\text{bits}} \end{array} \quad (2)$$

Of course s can be 0 or 1. When it is 0 we represent positive real numbers, when it is 1 we represent negative real numbers. For example, we can have the number (in binary)

$$|0|10000000011|101110010001 \overbrace{0000000000000000 \dots 0}^{40 \times}| \quad (3)$$

which has

$$\begin{aligned} s &= 0 \\ c &= 1 \times 2^{10} + 0 \times 2^9 + 0 \times 2^8 + \dots + 1 \times 2^1 + 1 \times 2^0 = 1027, \\ f &= 1 \times \left(\frac{1}{2}\right)^1 + 0 \times \left(\frac{1}{2}\right)^2 + 1 \times \left(\frac{1}{2}\right)^3 + 1 \times \left(\frac{1}{2}\right)^4 + 1 \times \left(\frac{1}{2}\right)^5 + 0 \times \left(\frac{1}{2}\right)^6 + 0 \times \left(\frac{1}{2}\right)^7 \\ &\quad + 1 \times \left(\frac{1}{2}\right)^8 + 0 + 0 + 0 + 1 \times \left(\frac{1}{2}\right)^{12} + 0 + \dots + 0 \times \left(\frac{1}{2}\right)^{52} = 0.722900390625 \end{aligned} \quad (4)$$

and hence the real number represented is

$$N_0 = (-1)^0 2^{(1027-1023)} (1 + 0.722900390625) = 27.56640625 \quad (5)$$

Now, we can start to see the limitations of floating point arithmetic.

The next largest real number that can be represented is

$$|0|10000000011|101110010001 \overbrace{0000000000000000 \dots 01}^{39 \times}| \quad (6)$$

This number has the same s , the same c , f which is larger by $1/2^{52} = 2.220446e-16$, i.e., $f = 0.72290039062500022204$, and hence the number represented is

$$N_1 = (-1)^0 2^{(1027-1023)} (1 + 0.72290039062500022204) = 27.56640625000000355264 \quad (7)$$

The next smallest real number that can be represented is

$$|0|10000000011|101110010000 \overbrace{1111111111111111 \dots 1}^{40 \times}| \quad (8)$$

This number, let's call it N_{-1} , has the same s , the same c , and f which is smaller by $-1/2^{52} + \sum_{n=13}^{52} 1/2^n = 1/2^{52}$. Thus, N_0 represents half of the numbers between N_0 and N_1 and half of the numbers between N_{-1} and N_0 . These examples explicitly show that there is a finite number of reals that is represented.

A. Overflow/Underflow

In addition, there is a finite range of numbers that can be represented. In particular, the largest possible positive real number corresponds (by convention) to $c = 2046$, $f = 1 - 1/2^{52}$, thus

$$N_{\max} = 2^{1023}(2 - 1/2^{52}) \approx 0.17977 \times 10^{309} \quad (9)$$

The smallest possible positive real number corresponds to $c = 1$, $f = 0$, thus

$$N_{\min} = 2^{-1022}(1 + 0) \approx 0.22251 \times 10^{-307} \quad (10)$$

If during a computation a number becomes greater than $N_{\max}$, we say we have overflow, and if a number becomes smaller than $N_{\min}$, we say we have underflow.

B. Decimal Machine Numbers

Let the machine numbers be represented in the normalized decimal floating-point form

$$\pm 0.d_1 d_2 \dots d_k \times 10^n, \quad 1 \leq d_1 \leq 9, 0 \leq d_i \leq 9, \quad i = 2, \dots, k \quad (11)$$

Numbers of this form are called k-digit decimal machine numbers.

Any positive real number

$$y = 0.d_1 d_2 \dots d_k d_{k+1} d_{k+2} \dots \times 10^n \quad (12)$$

will have a floating-point representation by terminating the mantissa at k decimal digits either by chopping or rounding.

Chopping means remove $d_k d_{k+1} d_{k+2} \dots$ and produces the floating point k-digit number

$$fl(y) = 0.d_1 d_2 \dots d_k \dots \times 10^n \quad (13)$$

Rounding adds $5 \times 10^{n-(k+1)}$ and chops the result to obtain a number $fl(y) = 0.\delta_1 \delta_2 \dots \delta_k$:

- If $d_{k+1} \geq 5$ then $d_k \rightarrow d_k + 1$ (round up).
- If $d_{k+1} < 5$ then chop off all but first k digits (round down).

When rounding down $\delta_i = d_i$. When rounding up the digits can change.

Exercise: consider $\pi = 3.14159265 = 0.314159265 \times 10^1$. What is the number that is represented after 5-digit rounding or chopping?

Answer: Using 5-digit chopping $fl(\pi) = 0.31415 \times 10^1 = 3.1415$. Using 5-digit rounding $fl(\pi) = \text{chop}[(0.314159265 + 0.00001) \times 10^1] = \text{chop}[(0.314160265) \times 10^1] = 0.31416 \times 10^1 = 3.1416$.

Definition: If $\tilde{p}$ represents an approximation to a real number p , the relative error

$$\frac{|p - \tilde{p}|}{p} \equiv \text{Roundoff error.} \quad (14)$$

In the previous exercise, what is the roundoff error after chopping? What is the roundoff error after rounding?

Exercise: Think about the way real numbers are represented in a computer, i.e., Eqs. (1)-(5). Based on this, what is the roundoff error in 64-bit floating point arithmetic?

Answer: The smallest possible change to any real number comes from the mantissa term $1/2^{52} = 2.2204 \times 10^{-16}$. Thus, this term represents approximately the roundoff error of 64-bit floating point arithmetic, i.e., 64-bit floating real numbers are accurate to about 16 digits.

MTH6150: Solving non-linear algebraic equations

Charalampos Markakis¹ & Vasileios Paschalidis²

¹ School of Mathematical Sciences, Queen Mary University of London and

² Departments of Astronomy & Physics, University of Arizona, Tucson

In this note we discuss methods for solving non-linear algebraic equations in one or more variables.

I. SOLVING EQUATIONS IN ONE VARIABLE

In this section we consider one of the most basic mathematical problems: root-finding in one variable. In other words find the root for an equation of the form $f(x) = 0$.

A. Bisection

The first method we will consider is called bisection and is based on the Intermediate Value Theorem.

Intermediate value theorem: Let f be a continuous function in the interval $[a, b]$ with $f(a)f(b) < 0$, in other words $f(a)$ and $f(b)$ have different opposite signs. Then there exists $x_0 \in (a, b)$ with $f(x_0) = 0$.

Although bisection works when there multiple roots in the interval (a, b) , we assume for simplicity that the root in this interval is unique. The bisection method then simply repeatedly halves (or bisects) subintervals of $[a, b]$ and, at each step, locates that half of the subintervale which contains x_0 based on the intermediate value theorem.

Flowchart of the algorithm:

- INPUT: endpoints a, b ; tolerance TOL; maximum number of iterations $N, f(x)$
- OUTPUT approximate solution x_0 or message of failure.
- Step 1: set $fa = f(a), fb = f(b)$ Check if $fa = 0$ or $fb = 0$, and exit saying that you have found roots $x = a$ or $x = b$.
If $fa \neq 0$ **and** $fb \neq 0$, then
Check if $f(a)f(b) \geq 0$.
If the condition is true exit with a message stating that “if $f(x)$ is continuous, bisection will not be able to find a root in this interval.”

Else go to Step 2.
- Step 2: Set $n = 1$; // The counter of the number of iterations
- Step 3: While $n \leq N$ do Steps 4-7
- Step 4: Set $x0 = (a + b)/2$; Set $fx0 = f(x0)$;
- Step 5: If $fx0 = 0$ or $(b - a)/b < \text{TOL}$ then
return $x0$;
- Step 6: Set $n+ = 1$;
- Step 7: If $fa * fx0 > 0$, then set $a = x0$
else set $b = x0$
- OUTPUT “Could not find a root: Maximum number of iterations reached. Either increase the number of iterations or check your tolerance or your condition for exiting the loop.”
RETURN

Other stopping procedures can be applied in Step 5. For example you can use the value of the function $f(x0)$ itself, i.e., stop if $fx0 < \text{TOL}$.

Another is to stop if $|b - a|/b < \text{TOL}$. Another is to consider two consecutive estimates for x_0 , let's call them x_{0N} , x_{0N-1} and stop when $|x_{0N} - x_{0N-1}|/x_{0N} < \text{TOL}$. The last two stopping conditions while they are very good because they test for the relative error, they will fail if $x_0 = 0$. In that case you can consider as safety to add a small number to the denominator, i.e., $|b - a|/(b + \epsilon) < \text{TOL}$ and $|x_{0N} - x_{0N-1}|/(x_{0N} + \epsilon) < \text{TOL}$, with ϵ something on the order of machine precision, e.g., $\epsilon = 1e-14$. In such cases you may need to experiment with the tolerance and ϵ , and it is always best to have multiple checks for when you have found a root.

Exercise: Write a code that implements the bisection algorithm and solve $x^2 + 4x^{4/3} - 10 = 0$, which has one real root. Think of a convenient interval that brackets the root. Determine the root with an accuracy of 1 part in 10^{10} . Can you think of a way to determine how many iterations you may need to achieve such accuracy?

Make sure you introduce a function $f(x) = x^2 + 4x^{4/3} - 10$. This way your code will be modular, i.e., function independent. Therefore, if you want to solve another equation you only need to change the function. Better, yet, create a file called, e.g., `functions.C` which contains the bisection function and the `fofx` function. Then add a header file and call the bisection function from the main C++ routine.

Note: To exit a C++ program before reaching return you can use `exit(0);`, for which you will need `#include <cstdlib>`.

B. Newton-Raphson

One of the most powerful methods for solving non-linear equations in one or more variables is the Newton-Raphson, which also has a nice geometric interpretation. Let's derive the algorithm.

Let f be a C^2 function in $[a, b]$. Let $\bar{x}_0$ be an approximation to a root of f in $[a, b]$. Since, f is C^2 we can Taylor expand it about $\bar{x}_0$, i.e.,

$$f(x) = f(\bar{x}_0) + (x - \bar{x}_0)f'(\bar{x}_0) + \frac{(x - \bar{x}_0)^2}{2}f''(c), \quad (1)$$

where the above equation is exact for some $c \in [a, b]$, and the primes indicate differentiation. If we evaluate the above equation at the true root, let's call it x_0 , then $f(x_0) = 0$ and we obtain

$$f(\bar{x}_0) + (x_0 - \bar{x}_0)f'(\bar{x}_0) + \frac{(x_0 - \bar{x}_0)^2}{2}f''(c) = 0. \quad (2)$$

Given that we assumed $\bar{x}_0$ is a good approximation to x_0 the term $(x_0 - \bar{x}_0)^2$ is much smaller than the first two terms, so we drop that quadratic term to arrive at

$$f(\bar{x}_0) + (x_0 - \bar{x}_0)f'(\bar{x}_0) = 0. \quad (3)$$

which we can solve with x_0 - the true root of f , i.e.,

$$x_0 = \bar{x}_0 - \frac{f(\bar{x}_0)}{f'(\bar{x}_0)} \quad (4)$$

This equation is the heart of the Newton-Raphson algorithm as it serves to generate an iterative scheme that converges to the true root, as follows

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, n \geq 0. \quad (5)$$

Starting with an initial guess x_0 for the root, we can obtain a better estimate by means of the above iterative scheme as

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} \quad (6)$$

FIG. 1. The geometric interpretation of the Newton-Raphson method for solving equations. Given the guess at x_0 we draw the tangent to $f(x)$ at x_0 . The next guess for the root (x_1) is where **tangent1** intersects the x-axis. Then we draw the tangent to $f(x)$ at x_1 , and the next guess for the root (x_2) is where **tangent2** intersects the x-axis. Next we draw the tangent to $f(x)$ at x_2 , and the next guess for the root (x_3) is where **tangent3** intersects the x-axis. Next we draw the tangent to $f(x)$ at x_3 , and the next guess for the root (x_4) is where **tangent4** intersects the x-axis, which is very close to the true root of $f(x)$.

Then x_1 can be used to get an even better estimate

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)} \quad (7)$$

And continue until the scheme converges. Given two consecutive approximations to the root, we have converged when $|x_{n+1} - x_n|/|x_{n+1}| < \text{TOL}$. The other criteria for convergence we discussed for bisection can be applied here, too.

Notice that one of the greatest pitfalls of the Newton-Raphson method is when the derivative of the function becomes 0. When deriving Eq. (4) we implicitly assumed that $f'(\bar{x}_0) \neq 0$. If during the iteration the derivative becomes 0, then obviously dividing by 0 will be a bad thing. Another pitfall of the algorithm is that if the function has multiple roots, then during the iteration the algorithm may leave the domain of the root we are looking for. Usually, changing the initial guess helps overcome these problems, but it is not always guaranteed.

The algorithm has a nice geometric interpretation. Essentially at each guess for the solution we approximate the function by its tangent at the guess point, and the point where the tangent intersects the x axis becomes the new guess. This is demonstrated in Fig. 1.

A set of steps for implementing the Newton-Raphson algorithm are as follows:

INPUT initial approximation $\bar{x}_0$; tolerance TOL; maximum number of iterations N .

OUTPUT approximate solution x_0 or message of failure.

- Step 1: Set $n = 1$.
- Step 2: While $n \leq N$ do Steps 3-6.
- Step 3: Set $x = x_0 - f(x_0)/f'(x_0)$ (Compute x_{n+1})
- Step 4: If $|x - x_0|/|x| < \text{TOL}$ then return x ; (The procedure was successful.)
- Step 5: Set $n+ = 1$. (Next iteration);
- Step 6: Set $x_0 = x$ (Update x_0);
- Step 7: OUTPUT ("The method failed after N iterations");
(The procedure was unsuccessful.) EXIT.

C. Secant method

A weakness of the Newton-Raphson method is that we need to know the derivative of $f(x)$ at each iteration. For non-linear equations computing the derivative analytically can be difficult or it can be a very long expression that requires many algebraic computations that render the algorithm slow. To overcome this issue the computation of the derivative can be performed numerically using its definition, i.e.,

$$f'(x_0) = \lim_{x \rightarrow x_0} \frac{f(x) - f(x_0)}{x - x_0}. \quad (8)$$

If we assume that x and x_0 are sufficiently close then we can approximate

$$f'(x_0) \approx \frac{f(x) - f(x_0)}{x - x_0}. \quad (9)$$

In terms of the iterative Newton-Raphson scheme that needs the derivative, this can be written as

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}. \quad (10)$$

If we plug in Eq. (10) in Eq. (5), we obtain

$$x_{n+1} = x_n - \frac{f(x_n)(x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}, n \geq 0. \quad (11)$$

Equation (11) is the essence of the secant method. A set of steps for implementing the secant algorithm are as follows:

INPUT initial approximations x_0, x_1 ; tolerance TOL; maximum number of iterations N .

OUTPUT approximate solution x_1 or message of failure.

- Step 1: Set $n = 2$. $f x_0 = f(x_0)$, $f x_1 = f(x_1)$
- Step 2: While $n \leq N$ do Steps 3-6.
- Step 3: Set $x = x_1 - f x_1(x_1 - x_0)/(f x_1 - f x_0)$ (Compute x_{n+1})
- Step 4: If $|x - x_1|/|x| < \text{TOL}$ then return x; (The procedure was successful.)
EXIT.
- Step 5: Set $n+ = 1$. (Next iteration);
- Step 6: Set $x_0 = x_1$, $f x_0 = f x_1$, $x_1 = x$, $f x_1 = f(x)$ (Update $x_0, f x_0, x_1, f x_1$);
- Step 7: OUTPUT (“The method failed after N iterations”);
(The procedure was unsuccessful.) EXIT.

Note: It is also possible to create hybrid methods. For example bisection can be combined with the secant method to accelerate convergence. The resulting method is called Brent’s method, but we will not be going into greater details during this course. We only mention this here for the interested reader.

II. SOLVING SYSTEMS OF ALGEBRAIC EQUATIONS

So far we discussed methods for solving non-linear equations in one variable. However, in Physical sciences we often face the challenge of having to solve a system of N non-linear algebraic equations for N unknown variables. Here we will discuss an extension of the Newton-Raphson method for systems of equations.

A. Newton-Raphson method for systems of non-linear equations

Lets consider a system of N equations f_i with N unknown variables (x_i)

$$\begin{aligned} f_1(x_1, x_2, x_3, \dots, x_N) &= 0 \\ f_2(x_1, x_2, x_3, \dots, x_N) &= 0 \\ &\vdots \\ f_N(x_1, x_2, x_3, \dots, x_N) &= 0 \end{aligned} \quad (12)$$

We can also introduce the vector of all equations, i.e.,

$$F = \begin{pmatrix} f_1 \\ f_2 \\ \vdots \\ f_N \end{pmatrix} \quad (13)$$

We can now follow the same path as in the Newton-Raphson method for one single variable, i.e., use Taylor series. Only here we need to use multivariate calculus, to perform a first-order Taylor series for F about $x = x_0$, i.e.,

$$f_i(x) = f_i(x_0) + \sum_{j=1}^N \frac{\partial f_i}{\partial x_j}(x_j - x_0), \quad i = 1, \dots, N, \quad (14)$$

where $x = (x_1, x_2, \dots, x_N)$ are the true roots of f_i , and $x_0 = (x_{1,0}, x_{2,0}, \dots, x_{N,0})$ are the approximate roots. Since x are the exact roots, we have $f_i(x) = 0$. Thus, the previous equation becomes

$$\sum_{j=1}^N \frac{\partial f_i}{\partial x_j}(x_j - x_0) = -f_i(x_0), \quad i = 1, \dots, N, \quad (15)$$

Note that $\frac{\partial f_i}{\partial x_j}$ is evaluated at x_0 and is the i, j element of the Jacobian J matrix of the system of equations (20). This is easier to see if we write this last equation in the following matrix format

$$\begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \dots & \frac{\partial f_1}{\partial x_N} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \dots & \frac{\partial f_2}{\partial x_N} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_N}{\partial x_1} & \frac{\partial f_N}{\partial x_2} & \dots & \frac{\partial f_N}{\partial x_N} \end{pmatrix} \begin{pmatrix} x_1 - x_{1,0} \\ x_2 - x_{2,0} \\ \vdots \\ x_N - x_{N,0} \end{pmatrix} = - \begin{pmatrix} f_1(x_0) \\ f_2(x_0) \\ \vdots \\ f_N(x_0) \end{pmatrix} \quad (16)$$

or the following shorter matrix notation

$$J \cdot (x - x_0) = -F(x_0). \quad (17)$$

The formal solution of this last equation is

$$x = x_0 - J^{-1}F(x_0), \quad (18)$$

where J^{-1} is the inverse of the Jacobian evaluated at x_0 and F the column vector of equations we introduced earlier. As in the original Newton-Raphson method, this last equation forms the basis of the following iterative scheme

$$x_{n+1} = x_n - J^{-1}(x_n)F(x_n), \quad (19)$$

i.e., starting from an initial guess we compute the vector of variables x_n , the residual of equations $F(x_n)$, and the Jacobian $J(x_n)$, we then invert the Jacobian and use Eq. (19) to obtain the new approximation to the solution.

Systems of 2 equations with 2 unknowns: Lets apply the above to the simple case of a 2×2 , system.

$$\begin{aligned} f_1(x_1, x_2) &= 0 \\ f_2(x_1, x_2) &= 0 \end{aligned} \quad (20)$$

With the vector of all equations, i.e.,

$$F = \begin{pmatrix} f_1 \\ f_2 \end{pmatrix} \quad (21)$$

is easier to see if we write this last equation in the following matrix format

The Newton-Raphson method

$$x_{n+1} = x_n - J^{-1}(x_n)F(x_n), \quad (22)$$

in the 2×2 case translates to the following matrix equations

$$\begin{pmatrix} x_{1,n+1} \\ x_{2,n+1} \end{pmatrix} = \begin{pmatrix} x_{1,n} \\ x_{2,n} \end{pmatrix} - \begin{pmatrix} \frac{\partial f_1}{\partial x_1} \big|_{x_{1,n}, x_{2,n}} & \frac{\partial f_1}{\partial x_2} \big|_{x_{1,n}, x_{2,n}} \\ \frac{\partial f_2}{\partial x_1} \big|_{x_{1,n}, x_{2,n}} & \frac{\partial f_2}{\partial x_2} \big|_{x_{1,n}, x_{2,n}} \end{pmatrix}^{-1} \begin{pmatrix} f_1(x_{1,n}, x_{2,n}) \\ f_2(x_{1,n}, x_{2,n}) \end{pmatrix} \quad (23)$$

Notice that the 2×2 matrix above is the inverse of the matrix shown. Recall that the inverse of a 2×2 matrix is given by

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad - cb} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} \quad (24)$$

Note: the Newton-Raphson method for systems of equations has similar pitfalls as the original method for an equation in one variable. Namely, if the inverse of the Jacobian does not exist the scheme will fail. From linear algebra you know that if the determinant of a matrix is 0, then it cannot be inverted. Thus, before using the Newton-Raphson method for systems of equations one first checks whether the determinant of the Jacobian matrix is 0 or not.

A set of steps for implementing the Newton-Raphson algorithm for a set of equation are as follows:

INPUT initial approximation to all variables $\bar{x}_0$; tolerance TOL; maximum number of iterations N .

OUTPUT approximate solution x_0 or message of failure.

- Step 1: Set $n = 1$.
- Step 2: While $n \leq N$ do Steps 3-7.
- Step 3: Compute the determinant of the Jacobian matrix of the system for x_0 . If the determinant is 0 exit stating that the Jacobian cannot be inverted. Else, continue to Step 4.
- Step 4: Compute inverse of Jacobian. Set $x = x_0 - J^{-1}(x_0)F(x_0)$ (Compute the vector x_{n+1})
- Step 5: If $|x - x_0|/|x| < \text{TOL}$ (for each of the unknown variables separately) then return x; (The procedure was successful.)
EXIT.
- Step 5: Set $n+ = 1$. (Next iteration);
- Step 6: Set $x_0 = x$ (Update x_0);
- Step 7: OUTPUT ("The method failed after N iterations");
(The procedure was unsuccessful.) EXIT.

MTH6150: Interpolation

Charalampos Markakis¹ & Vasileios Paschalidis²

¹ *School of Mathematical Sciences, Queen Mary University of London and*

² *Departments of Astronomy & Physics, University of Arizona, Tucson*

In this note we discuss methods for performing interpolation in one or more variables.

I. INTERPOLATION IN ONE VARIABLE

In many occasions we do not have analytic functions to work with, but we have only numerical data, and we need the value of a function at a point where we do not have data. This is shown in Fig. 1. The procedure for “predicting” $f(x_0)$ is called *interpolation* if x_0 is within the range of x values for which we have data. If x_0 is outside the range of x values for which we have data, then the procedure is called *extrapolation* and should be avoided.

An important theorem on which interpolation is based on is the following:

Weierstrass Approximation Theorem: Let $f(x)$ be defined and continuous for $x \in [a, b]$. For each $\epsilon > 0$, there exists a polynomial $P(x)$ such that $|f(x) - P(x)| < \epsilon$, $\forall x \in [a, b]$.

In other words any continuous function can be approximated by a suitable polynomial. You may think that we can always achieve this with Taylor polynomials. Lets consider e^x , for which the Taylor polynomials at $x = 0$ are

$$P_0(x) = 1, P_1(x) = 1 + x, P_2(x) = 1 + x + \frac{x^2}{2}, P_3(x) = 1 + x + \frac{x^2}{2} + \frac{x^3}{6}, \dots \quad (1)$$

As shown in Fig. 2, these polynomials converge to the correct function as the order of the polynomial increases. However, this is not the case for all functions. Lets consider $f(x) = 1/x$ for which the Taylor series at $x = 1$ is

$$P_n(x) = \sum_{k=0}^n (-1)^k (x - 1)^k \quad (2)$$

As shown in Fig. 3, these polynomials do not converge to the correct function as the order of the polynomial increases. More specifically, for $x \gtrsim 2$ the polynomials diverge!

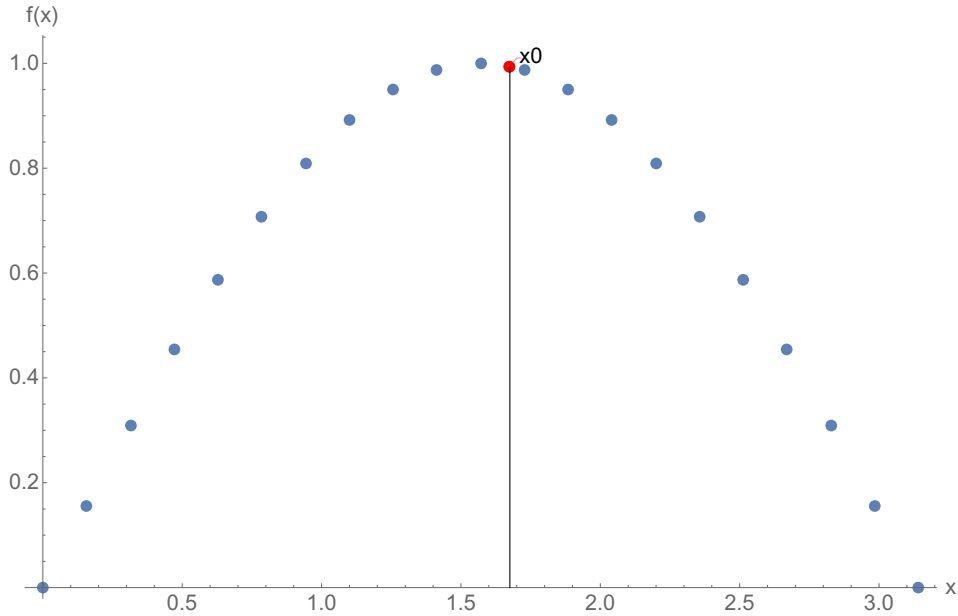


FIG. 1. The function $f(x)$ shown here has only numerical data (blue points) and we do not know its analytic form. However, we would like to find the value of f at $x = x_0$ (red point). The procedure for “predicting” $f(x_0)$ is called interpolation if x_0 is within the range of x values for which we have data as is the case with the red point shown here.

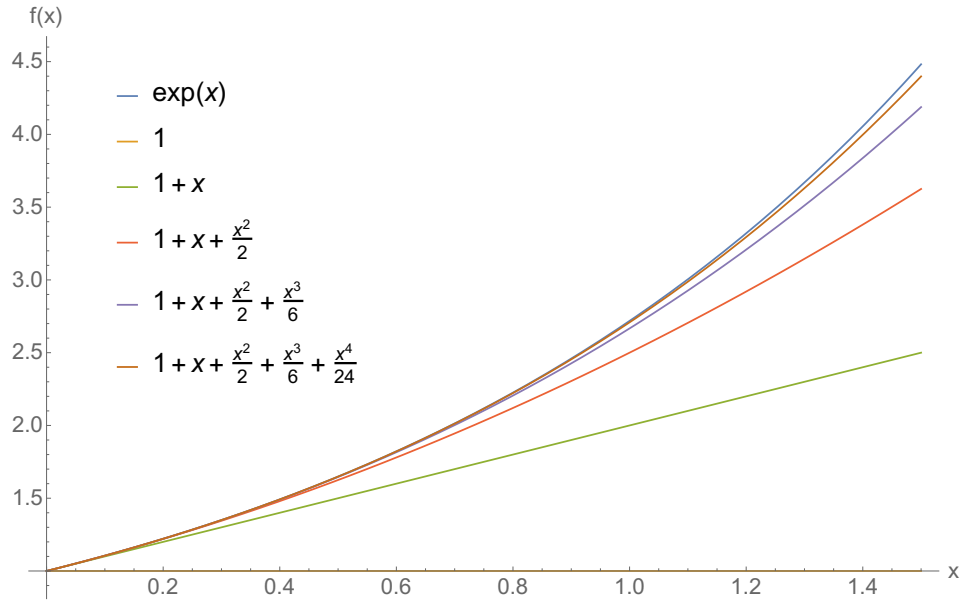


FIG. 2. The function $f(x) = e^x$ and its first 5 Taylor polynomials. Notice that as the order of the polynomial increases, we obtain a better and better approximation to $f(x) = e^x$, i.e., we converge to the correct values.

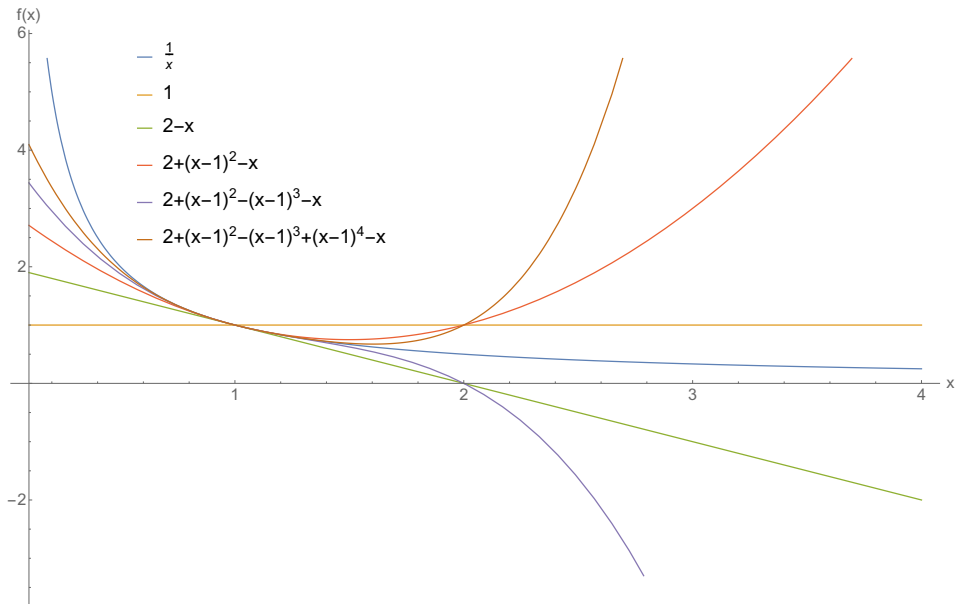


FIG. 3. The function $f(x) = 1/x$ and its first 5 Taylor polynomials at $x = 1$. Notice that as the order of the polynomial increases, in no sense do the polynomials converge to $f(x) = 1/x$, e.g., at $x = 3$.

The reason why Taylor polynomials do not always work is because these are very local. Once we move away from the expansion point, we can easily lose convergence. To correctly find interpolating values, we need to use more global information. This is what is achieved by the *Lagrange Interpolating Polynomials* that we discuss next.

A. Lagrange Interpolating Polynomials

Lagrange polynomials agree exactly with a function $f(x)$ at N distinct points. For simplicity let's consider two points: (x_0, y_0) and (x_1, y_1) , with $y_i = f(x_i)$.

Let

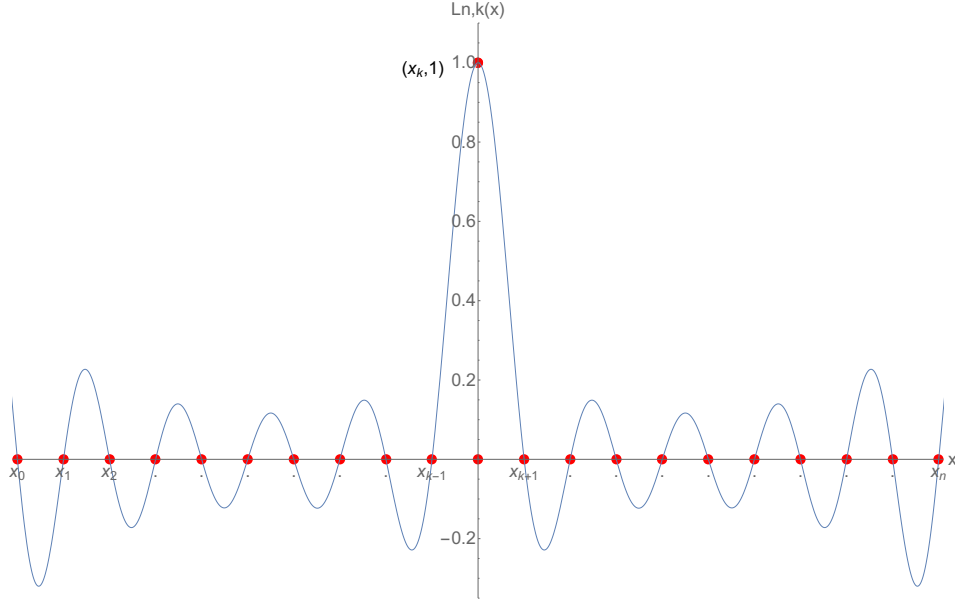


FIG. 4. The function $L_{n,k}(x)$ which is equal to zero at all x_i , and equal to 1 at $x_i = x_k$.

$$L_0(x) = \frac{x - x_1}{x_0 - x_1}, \quad L_1(x) = \frac{x - x_0}{x_1 - x_0}. \quad (3)$$

The linear Lagrange interpolating polynomial through (x_0, y_0) and (x_1, y_1) is defined as

$$P(x) = L_0(x)f(x_0) + L_1(x)f(x_1) = \frac{x - x_1}{x_0 - x_1}f(x_0) + \frac{x - x_0}{x_1 - x_0}f(x_1) \quad (4)$$

Note that $L_0(x_0) = 1$, $L_0(x_1) = 0$, $L_1(x_0) = 0$, $L_1(x_1) = 1$, which imply $P(x_0) = f(x_0)$ and $P(x_1) = f(x_1)$. Most importantly P is unique.

Generalization to degree n : For any polynomial of the form $P_n(x) = \sum_{i=0}^n a_i x^i$, there are $(n+1)$ a_i coefficients. Thus, to completely determine all coefficients we need $n+1$ conditions. For the Lagrange polynomial these conditions are $P_n(x_k) = f(x_k)$, $k = 0, 1, \dots, n$. Thus, a degree n Lagrange polynomial requires $n+1$ points at which we know the values of a function. Consider that we have the set of points

$$(x_0, f(x_0)), (x_1, f(x_1)), \dots, (x_n, f(x_n)). \quad (5)$$

To construct the degree n Lagrange polynomial, first $\forall k = 0, 1, \dots, n$ construct a function $L_{n,k}(x)$ such that $L_{n,k}(x_i) = 0$, $i \neq k$. To satisfy $L_{n,k}(x_i) = 0$, $i \neq k$, the function $L_{n,k}(x)$ must be

$$L_{n,k}(x) \propto (x - x_0)(x - x_1) \dots (x - x_{k-1})(x - x_{k+1}) \dots (x - x_n) \quad (6)$$

We also want $L_{n,k}(x_k) = 1$. Given Eq. (6), this can be achieved if we divide Eq.(6) by $(x_k - x_0)(x_k - x_1) \dots (x_k - x_{k-1})(x_k - x_{k+1}) \dots (x_k - x_n)$. Then, we arrive at the final form

$$L_{n,k}(x) = \frac{(x - x_0)(x - x_1) \dots (x - x_{k-1})(x - x_{k+1}) \dots (x - x_n)}{(x_k - x_0)(x_k - x_1) \dots (x_k - x_{k-1})(x_k - x_{k+1}) \dots (x_k - x_n)} = \prod_{i=0, i \neq k}^n \frac{x - x_i}{x_k - x_i}. \quad (7)$$

A graphical illustration of this function is shown in Fig. 4, where you can see that $L_{n,k}(x_i) = 0$, $i \neq k$, and $L_{n,k}(x_k) = 1$.

Once we have constructed the function $L_{n,k}(x)$ then the Lagrange interpolant of degree n is given by

$$P_n(x) = \sum_{k=0}^{k=n} L_{n,k}(x)f(x_k) \quad (8)$$

Exercise 1: Consider the function $f(x) = 1/x$, its 2nd-order Taylor expansion at $x_0 = 1$ ($T_2(x)$), and the quadratic Lagrange interpolant ($P_2(x)$) using $x_0 = 1$ and $x_1 = 2$, and $x_2 = 3$. Compare the value $f(1.5)$ with $T_2(1.5)$ and with $P_2(1.5)$. Is $T_2(1.5)$ or $P_2(1.5)$ a better approximation to $f(1.5)$?

Error theorem: Let $n + 1$ distinct values $x_0, x_1, \dots, x_n \in [a, b]$, and $f \in C^{n+1}[a, b]$. Then for each $x \in [a, b]$ there exists a number $c \in (a, b)$ and between x_i , such that

$$f(x) = P_n(x) + \frac{f^{(n+1)}(c)}{(n+1)!} (x - x_0)(x - x_1) \dots (x - x_n), \quad (9)$$

where $P_n(x)$ is the Lagrange interpolating polynomial and $f^{(n+1)}$ the $n + 1$ -th derivative of f . The second term on the right-hand-side of Eq. (9) is the *error* of the interpolation.

Thus, the max. error in $[a, b]$ will be given by

$$\max \left[\left| \frac{f^{(n+1)}(c)}{(n+1)!} \prod_{i=0}^n (x - x_i) \right| \right]. \quad (10)$$

This expression can be used to determine the tolerance within which we desire the interpolation. Note that if the analytic form of the function is not known, the above expression cannot be applied. This is the case when only numerical data are provided. In such case, one can estimate an error for $P_n(x)$ in $[a, b]$ by computing $P_{n+1}(x)$ in $[a, b]$ and then in $[a, b]$ computing

$$\max \left[\left| P_{n+1}(x) - P_n(x) \right| \right]. \quad (11)$$

Exercise 2:a) Consider $f(x) = 1/x$ given in $[2, 4]$. Write a code that splits the interval $[a, b] = [2, 4]$ in n equidistant intervals, i.e. $n + 1$ points, where you will define $f(x_i)$. This can be achieved if you define the grid points $x_i = a + i * (b - a) / n, i = 0, 1, 2, \dots, n$. Thus, the grid consists of $n + 1$ grid points. What is the highest degree Lagrange interpolant you can construct with $n + 1$ points? Follow these steps:

- Declare an array (**xi**) of size $n + 1$, construct a loop and save the coordinate points x_i into the array **xi**.
- Declare an array (**yi**) of size $n + 1$, construct a loop and save the value $f(x_i)$ at the coordinate points x_i into the array **yi**. For convenience introduce a function $f(x) = 1/x$ to do these computations. This way your code will be modular as you can change $f(x)$ if you wish to.
- Construct the unique n -th degree Lagrange polynomial given xi, yi and use it to interpolate at $x = 3.127$ for various values of n . It may be convenient to construct a C++ function $L_{n,k}(x)$ which takes **n**, **k**, **xi**, **x** as arguments and returns the value of $L_{n,k}(x)$. If you have the function $L_{n,k}(x)$, you can easily construct the Lagrange interpolating polynomial through Eq. (8), which will be a C++ function that takes **xi**, **yi**, **x** as inputs calls $L_{n,k}(x)$ and uses Eq. (8) to output $P_n(x)$.
- Run your code for $n = 3, 5, 9, 17$ and output the Degree of the polynomial and the relative error $|P_n(3.127) - f(3.127)| / |f(3.127)|$?

Does the relative error $|P_n(3.127) - f(3.127)| / |f(3.127)|$ decrease as we increase the degree of the Lagrange polynomial? Make a plot of the relative error vs the degree of the Lagrange polynomial and determine the trend. Is the trend a power law, exponential, something else? Think about what kind of plot you need to make: log-linear, log-log to determine whether the trend is a power law or exponential.

Note: when interpolating a function that is known at $N + 1$ points, we don't have to introduce an Lagrange interpolant that is N -th order. We can always use a smaller number of points, say 3 or 4 (or whatever number you desire) that are the nearest ones to and contain the point at which we want to interpolate the function, and then perform 2nd, 3rd or other degree Lagrange interpolation. This saves computational cost and memory. The disadvantage of this approach is that the interpolating functions in general will not match the slope of the function being interpolated, in other words the interpolants at different points will *not* match smoothly one onto another. This issue can be resolved by spline interpolation that we will not cover here, but you can look up online or in numerical methods books.

B. Introduction to Spectral Interpolation

Lets assume we have a complete basis functions $b_n(x)$. Then any smooth function f can be expanded as

$$f(x) = \sum_{n=0}^{\infty} a_n b_n(x) \quad (12)$$

For periodic functions a Fourier basis is always the preferred choice. In cases where there is no periodicity and f is highly smooth (especially when it is C^∞), the preferred basis is formed by the Chebyshev polynomials. These polynomials are defined for $|x| \leq 1$, and are given by

$$T_n(x) = \cos(n \arccos(x)) \text{ or } T_n(\cos(x)) = \cos(nx) \quad (13)$$

Thus, an N -th order approximation to $f \in [a, b]$ is

$$f(x) = \sum_{n=0}^N a_n T_n(x), \quad (14)$$

but first we need a map from $[a, b]$ to $[-1, 1]$ where the Chebyshev polynomials are defined. The simplest possible map is a linear one, i.e.,

$$x = Au + B \quad (15)$$

and to determine the A, B coefficients for the new variable $u \in [-1, 1]$ we require $x = a, u = -1$, $x = b, u = 1$ which yields the following system of equations

$$a = -A + B, \quad b = A + B \quad (16)$$

whose solution gives

$$A = \frac{b-a}{2}, \quad B = \frac{a+b}{2}. \quad (17)$$

Thus, the map is

$$x = \frac{b-a}{2}u + \frac{a+b}{2}, \quad (18)$$

or

$$u = \frac{2x - a - b}{b - a} \quad (19)$$

Thus, for every $x \in [a, b]$ there exists a unique $u \in [-1, 1]$, so we can now write

$$f(x(u)) = \sum_{n=0}^N a_n T_n(u), \quad (20)$$

How do we compute the coefficients a_n ? There is a standard procedure using:

- the so-called Gauss-Lobatto collocation points, which are the zeroes of $(1 - u^2)T'_k(u)$, and are given by

$$u_n = \cos\left(\frac{\pi n}{k}\right), n = 0, 1, \dots, k \quad (21)$$

- the Chebushev orthogonality property

$$(T_k, T_\ell)_W = \int_{-1}^1 T_k T_\ell w dx = \frac{\pi}{2} c_k \delta_{k\ell}, \quad (22)$$

where $\delta_{k\ell}$ is the Kronecker delta, $c_k = 2$ for $k = 0$ while $c_k = 1$ for $k \geq 1$, and the weightfunction $w = (1 - x^2)^{-1/2}$. The quantity $(T_k, T_\ell)_W$ is a weighted inner product in the space of integrable functions.

Then using Eq. (20) we can compute the inner product of f with a Chebyshev polynomial as

$$(f, T_\ell)_w = \int_{-1}^1 \sum_n a_n T_n T_\ell w dx = \sum_n a_n \int_{-1}^1 T_n T_\ell w dx = \frac{\pi}{2} \sum_n a_n c_n \delta_{n\ell} = \frac{\pi}{2} a_\ell c_\ell. \quad (23)$$

This last expression serves to determine the coefficient a_ℓ , through

$$a_\ell = \frac{2}{\pi c_\ell} (f, T_\ell)_w. \quad (24)$$

Therefore, to determine the expansion coefficients, we need to compute the integral $\frac{2}{\pi c_\ell} (f, T_\ell)_w$. This is performed by using Gaussian quadratures that we will discuss later in the course. For now, we can just give the straight answer that for any function $p(x)$ we have

$$\int_{-1}^1 p(u) w dx \simeq \frac{\pi}{N} \sum_{i=0}^N \frac{p(u_i)}{\bar{c}_i}, \quad (25)$$

where

$$\bar{c}_k = 2, \text{ if } k = 0, \quad \bar{c}_k = 1, \text{ if } 1 \leq k \leq N-1, \quad \bar{c}_k = 2, \text{ if } k = N, \quad (26)$$

and where again the Gauss-Lobatto points are

$$u_i = \cos\left(\frac{\pi i}{N}\right), \quad i = 0, 1, \dots, N. \quad (27)$$

Thus, to summarize a function can be approximated as

$$f(x(u)) = \sum_{\ell=0}^N a_\ell T_\ell(u), \quad (28)$$

with

$$a_\ell = \frac{2}{\pi c_\ell} \int_{-1}^1 f T_\ell w dx \simeq \frac{2}{N c_\ell} \sum_{i=0}^N \frac{f(x(u_i)) T_\ell(u_i)}{\bar{c}_i}. \quad (29)$$

If $f(x)$ is a polynomial, the above expression is **exact!** The above procedure is called Spectral interpolation and if f is C^∞ the expansion is exponentially convergent, i.e., the relative error converges to 0 exponentially.

MTH6150: Differentiation

Charalampos Markakis¹ & Vasileios Paschalidis²

¹ *School of Mathematical Sciences, Queen Mary University of London and*

² *Departments of Astronomy & Physics, University of Arizona, Tucson*

In this note we discuss methods for computing numerical derivatives.

I. NUMERICAL DIFFERENTIATION

In many occasions we do not have analytic functions to work with (for example when solving differential equations), but we need to compute the derivative of a function with respect to a variable given the values of the function at a finite number of points. Computing derivatives of functions known at a finite number of grid points is called numerical differentiation. There are multiple ways one can compute a numerical derivative and we will review here some of these methods.

A. Finite differences: first order derivatives

We have already seen a numerical approximation to the derivative of a function $f(x)$ at a point x in the secant method for solving non-linear equations. We used the definition of the derivative to achieve this, i.e.,

$$f'(x_0) = \lim_{x \rightarrow x_0} \frac{f(x) - f(x_0)}{x - x_0}. \quad (1)$$

If we define $\Delta x = x - x_0$, the above is also written as

$$f'(x_0) = \lim_{\Delta x \rightarrow 0} \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x}, \quad (2)$$

which gives us the following numerical approximation

$$f'(x_0) \approx \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x}. \quad (3)$$

This approximation does not allow us to understand the level of error we are making (or how quickly the approximation converges to the correct value as the difference $\Delta x = x - x_0$ goes to 0), when we compute the derivative with this simple formula. To understand this we need to find more accurate ways for computing the numerical derivative.

To simplify the analysis in this subsection we will assume that the function $f(x)$ is known on a grid of points with fixed grid spacing Δx . In other words, we will assume that we know the function at points $x_i = x_0 + i \cdot \Delta x$, where x_0 is the point we want to compute the derivative at, and i is an integer. Next we will Taylor expand the function at $x_0 + \Delta x$, i.e.,

$$f(x_0 + \Delta x) = f(x_0) + f'(x_0)\Delta x + \frac{1}{2}f''(x_0)\Delta x^2 + \frac{1}{3!}f'''(x_0)\Delta x^3 + \dots, \quad (4)$$

where the ellipsis correspond to the higher order terms which we will neglect for now. We can now use this last equation to form the right-hand-side of Eq. (3), which gives

$$\frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x} = f'(x_0) + \frac{1}{2}f''(x_0)\Delta x + \frac{1}{3!}f'''(x_0)\Delta x^2 + \dots \quad (5)$$

and solving with respect to (wrt) $f'(x_0)$ we obtain

$$f'(x_0) = \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x} - \underbrace{\frac{1}{2}f''(x_0)\Delta x - \frac{1}{3!}f'''(x_0)\Delta x^2 + \dots}_{\text{leading order truncation error}} \quad (6)$$

Assuming Δx is sufficiently small, the term proportional to Δx^2 is small compared to the term proportional to Δx , thus the leading order truncation error is $(1/2)f''(x_0)\Delta x$ (it is called truncation error because we truncate the series).

In other words the error is first order in Δx which we write as $O(\Delta x)$. Thus, the approximation of Eq. (3) is only first order accurate

$$f'(x_0) = \frac{f(x_0 + \Delta x) - f(x_0)}{\Delta x} + O(\Delta x), \quad (7)$$

which that as we decrease Δx the numerical derivative approaches the correct value at a rate proportional to Δx . In most applications, the first-order approximation is not good enough to obtain a good approximation to numerical derivatives. More accurate expressions are desirable, especially those that converge to the correct answer at a faster rate than first order. To achieve this we need to use either more information or choose the information in a smarter way. Notice that the expression Eq. (3) uses the values of $f(x)$ at two grid points, i.e., $f(x_0)$ and $f(x_0 + \Delta x)$. To derive a second-order accurate expression for $f'(x_0)$ in fact we do not need more than 2 grid points as we will show now.

Let us consider again the Taylor expansion of $f(x_0 + \Delta x)$,

$$f(x_0 + \Delta x) = f(x_0) + f'(x_0)\Delta x + \frac{1}{2}f''(x_0)\Delta x^2 + \frac{1}{3!}f'''(x_0)\Delta x^3 + \dots \quad (8)$$

but also consider the Taylor expansion of $f(x_0 - \Delta x)$,

$$f(x_0 - \Delta x) = f(x_0) - f'(x_0)\Delta x + \frac{1}{2}f''(x_0)\Delta x^2 - \frac{1}{3!}f'''(x_0)\Delta x^3 + \dots \quad (9)$$

We can now subtract Eq. (9) from Eq. (8) to obtain

$$f(x_0 + \Delta x) - f(x_0 - \Delta x) = 2f'(x_0)\Delta x + \frac{1}{3}f'''(x_0)\Delta x^3 + \dots \quad (10)$$

Solving the last equation for $f'(x_0)$ we obtain

$$f'(x_0) = \frac{f(x_0 + \Delta x) - f(x_0 - \Delta x)}{2\Delta x} - \frac{1}{6}f'''(x_0)\Delta x^2 + \dots \quad (11)$$

Notice now that if we approximate the derivative as

$$f'(x_0) \approx \frac{f(x_0 + \Delta x) - f(x_0 - \Delta x)}{2\Delta x} \quad (12)$$

The error is $O(\Delta x^2)$. Notice also we are again using 2 grid points of information, but now the derivative is more accurate as the leading order error term decreases quadratically with Δx !

The numerical derivative in Eq. (12) is known as a second-order accurate centered finite difference (it uses equal number of points to the left and right of x_0), while the expression (3) is known as a first-order accurate forward difference approximation (it uses one point to the right of x_0 and no points to the left of x_0). Using Eq. (9) we can also derive the first-order accurate backward difference approximation, i.e.,

$$f'(x_0) \approx \frac{f(x_0) - f(x_0 - \Delta x)}{\Delta x}. \quad (13)$$

For many practical purposes using second-order accurate expressions suffice for a decent approximation to the first-order derivative. However, if we have a function in an interval $[a, b]$, and we need the derivative at $x = a$ or $x = b$ we cannot use the centered finite difference approximation since there is no point left of $x = a$ and no point right of $x = b$. At $x = a$ we need a forward finite difference approximation, and at $x = b$ we need a backward finite difference.

Can we obtain second-order accurate forward/backward difference approximations? The answer is yes, but it comes at a price. Let us see how this can be achieved for the forward difference. Consider the Taylor expansion Eq. (8) multiplied by 4, i.e.

$$4f(x_0 + \Delta x) = 4f(x_0) + 4f'(x_0)\Delta x + 2f''(x_0)\Delta x^2 + \frac{2}{3}f'''(x_0)\Delta x^3 + \dots \quad (14)$$

and the expansion about $x_0 + 2\Delta x$

$$f(x_0 + 2\Delta x) = f(x_0) + 2f'(x_0)\Delta x + 2f''(x_0)\Delta x^2 + \frac{4}{3}f'''(x_0)\Delta x^3 + \dots \quad (15)$$

Let us subtract these last two equations to obtain

$$4f(x_0 + \Delta x) - f(x_0 + 2\Delta x) = 3f(x_0) + 2f'(x_0)\Delta x - \frac{2}{3}f'''(x_0)\Delta x^3 + \dots \quad (16)$$

and solve with respect to $f'(x_0)$

$$f'(x_0) \approx \frac{-3f(x_0) + 4f(x_0 + \Delta x) - f(x_0 + 2\Delta x)}{2\Delta x} + O(\Delta x^2) \quad (17)$$

Thus, we arrived at the second-order accurate forward difference approximation. Notice, however, that to achieve this we need to use the values of $f(x)$ at 3 points, i.e., $f(x_0)$, $f(x_0 + \Delta x)$, and $f(x_0 + 2\Delta x)$.

Similarly, we can derive the second-order accurate backward difference, which is given by

$$f'(x_0) \approx \frac{3f(x_0) - 4f(x_0 - \Delta x) + f(x_0 - 2\Delta x)}{2\Delta x} + O(\Delta x^2) \quad (18)$$

Exercise: Derive the last expression.

B. Finite differences: second order derivatives

In addition to first order derivatives, second order derivatives often appear in the physical sciences. We can use Taylor expansions to also derive numerical approximations to second order derivatives. Lets see how we can achieve this using centered differences.

Consider the Taylor expansions at $x_0 + \Delta x$ and $x_0 - \Delta x$, i.e.,

$$f(x_0 + \Delta x) = f(x_0) + f'(x_0)\Delta x + \frac{1}{2}f''(x_0)\Delta x^2 + \frac{1}{3!}f'''(x_0)\Delta x^3 + \frac{1}{4!}f^{(4)}(x_0)\Delta x^4 + \dots \quad (19)$$

$$f(x_0 - \Delta x) = f(x_0) - f'(x_0)\Delta x + \frac{1}{2}f''(x_0)\Delta x^2 - \frac{1}{3!}f'''(x_0)\Delta x^3 + \frac{1}{4!}f^{(4)}(x_0)\Delta x^4 \dots, \quad (20)$$

where we included the term proportional to Δx^4 as well. If instead of subtracting these expressions, we add them up we obtain

$$f(x_0 + \Delta x) + f(x_0 - \Delta x) = 2f(x_0) + f''(x_0)\Delta x^2 + \frac{1}{3!}f^{(4)}(x_0)\Delta x^4 + \dots \quad (21)$$

Solving the last expression wrt $f''(x_0)$ we find

$$f''(x_0) = \frac{f(x_0 + \Delta x) + f(x_0 - \Delta x) - 2f(x_0)}{\Delta x^2} - \frac{1}{3!}f^{(4)}(x_0)\Delta x^2 + \dots \quad (22)$$

In other words the approximation to the second order derivative

$$f''(x_0) \approx \frac{f(x_0 + \Delta x) - 2f(x_0) + f(x_0 - \Delta x)}{\Delta x^2} + O(\Delta x^2) \quad (23)$$

is second-order accurate, i.e., the leading order truncation error decays as Δx^2 . Using Taylor expansions one can derive second-order accurate forward and backward difference approximations for the second derivative.

C. Convenient notation for computations

Let us introduce the notation that

$$f_i = f(x_i), \quad (24)$$

then for uniformly spaced grids

$$f_{i+1} = f(x_{i+1}) = f(x_i + \Delta x), \quad f_{i+2} = f(x_{i+2}) = f(x_i + 2\Delta x), \text{ etc.} \quad (25)$$

Thus, we can write the expressions for the derivative at each point x_i of the grid as follows:

- Second-order accurate centered difference for first order derivative:

$$f'_i = f'(x_i) = \frac{f_{i+1} - f_{i-1}}{2\Delta x} \quad (26)$$

- Second-order accurate forward difference for first order derivative:

$$f'_i = \frac{-3f_i + 4f_{i+1} - f_{i+2}}{2\Delta x} \quad (27)$$

- Second-order accurate backward difference for first order derivative:

$$f'_i = \frac{f_{i-2} - 4f_{i-1} + 3f_i}{2\Delta x} \quad (28)$$

- Second-order accurate centered difference for second order derivative:

$$f''_i = \frac{f_{i-1} - 2f_i + f_{i+1}}{\Delta x^2} \quad (29)$$

Exercise: α) Consider $f(x) = \ln(x)$ given in $[1, 2]$. Compute the derivative of the function at $x = 1.5$: i) using the exact expression, and ii) assuming $\Delta x = 0.1$ using

- the first-order accurate forward difference and its relative error with respect to the exact value
- the second-order accurate forward difference and its relative error with respect to the exact value
- the second-order accurate centered difference and its relative error with respect to the exact value

Which numerical approximation has the smallest relative error?

β) Use your code from the Lagrange interpolation as template to write a code that splits the interval $[a, b] = [1, 2]$ in n equidistant intervals, i.e. $n + 1$ points, where you will define $f(x_i)$. This can be achieved if you define the grid points $x_i = a + i * (b - a)/n, i = 0, 1, 2, \dots, n$. Thus, the grid consists of $n + 1$ grid points. The particular form of the grid points implies that the uniform grid spacing is $\Delta x = x_{i+1} - x_i = (b - a)/n$. Follow these steps:

- Declare an array (**xi**) of size $n + 1$, construct a loop and save the coordinate points x_i into the array **xi**.
- Declare an array (**yi**) of size $n + 1$, construct a loop and save the value $f(x_i)$ at the coordinate points x_i into the array **yi**. For convenience introduce a function $f(x) = \ln(x)$ to do these computations. This way your code will be modular as you can change $f(x)$ if you wish to.
- Compute $f'(x) \forall x \in (a, b)$ using i) second-order accurate centered differences, ii) forward differences and iii) backward differences. It may be convenient to introduce 3 functions that take the arrays **xi**, **yi**, and n as inputs, and you pass to them an array (pointer) **dfdx** which will be assigned the values of the approximate derivative. You can introduce 3 different **dfdx** arrays of size $n-1$, e.g., **dfdx1**, **dfdx2**, **dfdx3**, which will store the approximate derivatives computed with each method separately.
- Compute the L_2 norm of the errors for each of the 3 approximations. For a quantity F_i approximated by $\tilde{F}_i$, the L_2 of the errors is given by

$$L_2(\delta F) = \sqrt{\sum_{i=1} \left(F_i - \tilde{F}_i \right)^2}, \quad (30)$$

where here the sum is over the some grid points ($x \in (a, b)$), and obviously in our problem $F_i = f'_i$. It may also be convenient to introduce a function for computing the analytic derivative of the function so that your code is modular if it is provided $f(x)$ and df/dx .

- Run your code for $n = 10, 20, 40, 80, 160, 320$, and plot the L_2 norm of the errors keeping the same points only (independently of n) vs Δx . Since we are using fwd and bwd difference the calculation computing the derivative at points that are the same for all approximations requires that the first two grid points ($x[0]$, $x[1]$), and the last two grid points ($x[n-2]$, $x[n-1]$) be excluded from the computation. The L_2 must be computed at the **same** grid points (i.e. the same coordinates) for all values of n . Given that $n = 10$ has the fewest grid points, only the 6 grid points at $n = 10$ should be used in the computation of L_2 norms. This can be achieved if you have a loop over the grid that looks like this

```

for(int i=2;i<n-1;i++){ // excluding here i=0,i=1, and i=n-2 and i=n-1
    if (i % (n/10)==0){ // the "%" symbol is the modulo function
        "here you will do the L2 norm computation
    }
}

```

Does the error decay as Δx^2 for each of the 3 methods. After running your code create a file with 4 columns: "dx", "L2 norm centered", "L2 norm forward", "L2 norm backward". Then make a plot the 3 different L2 norms vs dx. What kind of plot do you need to make to show that the errors decay as Δx^2 .

D. General formula

Consider the set of $n+1$ points $\{x_0, x_1, \dots, x_n\}$ in some interval I , where $f \in C^\infty$, then from Lagrange interpolation we know

$$f(x) = \sum_{k=0}^n f(x_k) L_k(x) + \frac{(x-x_0) \dots (x-x_n)}{(n+1)!} f^{(n+1)}(\xi(x)), \quad (31)$$

where

$$L_k = \frac{(x-x_0)(x-x_1) \dots (x-x_{k-1})(x-x_{k+1}) \dots (x-x_n)}{(x_k-x_0)(x_k-x_1) \dots (x_k-x_{k-1})(x_k-x_{k+1}) \dots (x_k-x_n)} \quad (32)$$

Using the Lagrange approximant we can compute the derivative, i.e.,

$$f'(x) = \sum_{k=0}^n f(x_k) L'_k(x) + D_x \left[\frac{(x-x_0) \dots (x-x_n)}{(n+1)!} \right] f^{(n+1)}(\xi(x)) + \frac{(x-x_0) \dots (x-x_n)}{(n+1)!} D_x \left(f^{(n+1)}(\xi(x)) \right), \quad (33)$$

If we evaluate the last expression at any $x = x_j$ the last term vanishes and we are left with

$$f'(x_j) = \sum_{k=0}^n f(x_k) L'_k(x_j) + \frac{f^{(n+1)}(\xi(x_j))}{(n+1)!} \prod_{k=0, k \neq j}^n (x_j - x_k) \quad (34)$$

Equation (34) is called the $(n+1)$ -point formula approximating $f'(x_j)$. In practice using large n is very expensive. The power of approximations to derivatives obtained with this method is that the approximants can be applied to non-uniformly spaced grids.

Let us now obtain derivative expressions for 3-point formulas, for which we have

$$\begin{aligned} L_0(x) &= \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}, \quad L_1(x) = \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}, \quad L_2(x) = \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}, \\ L'_0(x) &= \frac{2x-x_1-x_2}{(x_0-x_1)(x_0-x_2)}, \quad L'_1(x) = \frac{2x-x_0-x_2}{(x_1-x_0)(x_1-x_2)}, \quad L'_2(x) = \frac{2x-x_0-x_1}{(x_2-x_0)(x_2-x_1)} \end{aligned} \quad (35)$$

Thus, applying Eq. (34) to obtain the 3-point formulae for the first order derivative we have

$$f'(x_j) = \sum_{k=0}^2 f(x_k) L'_k(x_j) + \frac{f^{(3)}(\xi(x_j))}{3!} \prod_{k=0, k \neq j}^2 (x_j - x_k) \quad (36)$$

which if we expand becomes

$$f'(x_j) = f(x_0)L'_0(x_j) + f(x_1)L'_1(x_j) + f(x_2)L'_2(x_j) + \frac{f^{(3)}(\xi(x_j))}{3!} \prod_{k=0, k \neq j}^2 (x_j - x_k) \quad (37)$$

Notice that this formula applies even if the points are not equidistant, i.e., $x_2 - x_1 \neq x_1 - x_0$. For uniformly spaced grids we have $x_2 = x_1 + \Delta x = x_0 + 2\Delta x$, and $x_1 = x_0 + \Delta x$. Then if we set $x_j = x_0$, $x_1 = x_0 + \Delta x$, $x_2 = x_0 + 2\Delta x$ the expressions (37) simplify to

$$\begin{aligned} f'(x_0) &= \frac{1}{\Delta x} \left[-\frac{3}{2}f(x_0) + 2f(x_1) - \frac{1}{2}f(x_2) \right] + \frac{\Delta x^2}{3} f'''(\xi_0) \\ f'(x_1) &= \frac{1}{\Delta x} \left[-\frac{1}{2}f(x_0) + \frac{1}{2}f(x_2) \right] + \frac{\Delta x^2}{6} f'''(\xi_1) \\ f'(x_2) &= \frac{1}{\Delta x} \left[\frac{1}{2}f(x_0) - 2f(x_1) + \frac{3}{2}f(x_2) \right] + \frac{\Delta x^2}{3} f'''(\xi_2) \end{aligned} \quad (38)$$

Since $x_1 = x_0 + \Delta x$, $x_2 = x_0 + 2\Delta x$ the last can be re-written as

$$\begin{aligned} f'(x_0) &= \frac{1}{2\Delta x} [-3f(x_0) + 4f(x_0 + \Delta x) - f(x_0 + 2\Delta x)] + O(\Delta x^2) \\ f'(x_0 + \Delta x) &= \frac{1}{\Delta x} \left[-\frac{1}{2}f(x_0) + \frac{1}{2}f(x_0 + 2\Delta x) \right] + O(\Delta x^2) \\ f'(x_0 + 2\Delta x) &= \frac{1}{\Delta x} \left[\frac{1}{2}f(x_0) - 2f(x_0 + \Delta x) + \frac{3}{2}f(x_0 + 2\Delta x) \right] + O(\Delta x^2) \end{aligned} \quad (39)$$

The first equation in Eq. (39) is the familiar second-order accurate forward difference we derived earlier. The other two equations can be recast in the centered and backward differences we derived as follows. For the second equation we introduce a new variable $x_0 + \Delta x = \bar{x}_0$, and then the equation can be written as

$$f'(\bar{x}_0) = \frac{1}{2\Delta x} [-f(\bar{x}_0 - \Delta x) + f(\bar{x}_0 + \Delta x)] + O(\Delta x^2) \quad (40)$$

which is the second-order accurate centered finite difference formula. For the third equation similarly we introduce a new variable $x_0 + 2\Delta x = \tilde{x}_0$, and then we obtain

$$f'(\tilde{x}_0) = \frac{1}{2\Delta x} [f(\tilde{x}_0 - 2\Delta x) - 4f(\tilde{x}_0 - \Delta x) + 3f(\tilde{x}_0)] + O(\Delta x^2), \quad (41)$$

which is the second-order accurate backward finite difference formula. Obviously the 3-point formulas are 2nd-order accurate.

E. Higher-order approximations

Similarly to the 3-point formulae, one can derive the 5-point formulae, which will be 4th-order accurate. For example, the centered difference is given by

$$f'(x_0) = \frac{1}{12\Delta x} [f(x_0 - 2\Delta x) - 8f(x_0 - \Delta x) + 8f(x_0 + \Delta x) - f(x_0 + 2\Delta x)] + O(\Delta x^4), \quad (42)$$

and the forward difference five-point formula is

$$f'(x_0) = \frac{1}{12\Delta x} [-25f(x_0) + 48f(x_0 + \Delta x) - 36f(x_0 + 2\Delta x) + 16f(x_0 + 3\Delta x) - 3f(x_0 + 4\Delta x)] + O(\Delta x^4). \quad (43)$$

II. ROUND-OFF ERROR INSTABILITY

Consider the 2nd-order accurate centered difference approximation for the first order derivative

$$f'(x_0) = \frac{f(x_0 + \Delta x) - f(x_0 - \Delta x)}{2\Delta x} - \frac{\Delta x^2}{6} f'''(\xi) \quad (44)$$

In a computer the expressions $f(x_0 \pm \Delta x)$ have a round-off error $e(x_0 \pm \Delta x)$, i.e., what we input is $\tilde{f}(x_0 \pm \Delta x) = f(x_0 \pm \Delta x) + e(x_0 \pm \Delta x)$. This implies that what we compute is

$$f'(x_0) = \frac{\tilde{f}(x_0 + \Delta x) - \tilde{f}(x_0 - \Delta x)}{2\Delta x} - \frac{\Delta x^2}{6} f'''(\xi) \quad (45)$$

or

$$f'(x_0) - \frac{f(x_0 + \Delta x) - f(x_0 - \Delta x)}{2\Delta x} = \frac{e(x_0 + \Delta x) - e(x_0 - \Delta x)}{2\Delta x} - \frac{\Delta x^2}{6} f'''(\xi) \quad (46)$$

If $|e(x_0 \pm \Delta x)| \leq \epsilon$, and $|f'''(\xi)| \leq M$, then

$$\left| f'(x_0) - \frac{f(x_0 + \Delta x) - f(x_0 - \Delta x)}{2\Delta x} \right| \leq \frac{\epsilon}{\Delta x} + \frac{\Delta x^2}{6} M \quad (47)$$

This last expression implies that if $\Delta x \downarrow$, $\Delta x^2 \downarrow$ so the truncation error $\downarrow$. But, $\frac{\epsilon}{\Delta x} \uparrow$. Hence, we cannot expect the error to always go down as $\Delta x \downarrow$. Thus, when decreasing Δx from a large value the total error goes down and at some point it will begin to increase. The turnover point occurs when $\frac{\epsilon}{\Delta x} \simeq \frac{\Delta x^2}{6} M$. Thus, if we take the function $\ln x$ in the interval $[1, 2]$, $f'''(x) = 2/x^3$, which is $f'''(x) \leq 2$ for $x \in [1, 2]$. Assuming 64-bit floating point arithmetic $\epsilon \simeq 10^{-16}$. Thus, solving $\frac{\epsilon}{\Delta x} \simeq \frac{\Delta x^2}{6} M$ for Δx , yields $\Delta x = (6\epsilon/M)^{1/3}$, i.e., $\Delta x = (3 \times 10^{-16})^{1/3} \approx \times 10^{-6}$. Thus, for $\Delta x \lesssim 10^{-6}$, we would expect the error in evaluating the derivative to begin to increase. You will test this expectation in the 3rd homework set.

III. RICHARDSON EXTRAPOLATION

The Richardson extrapolation can be applied when an approximation method has a predictable truncation error term, that depends on a single parameter (usually the grid spacing or step size δx). When we are able to apply it, Richardson extrapolation provides a way to obtain more accurate expressions by using less accurate results. As a result, Richardson extrapolation (RE) can be used to also obtain error estimates.

Let $N_1(\Delta x)$ approximate a constant M , i.e.,

$$N - N_1(\Delta x) = k_1 \Delta x + k_2 \Delta x^2 + k_3 \Delta x^3, \quad (48)$$

for unknown k_1, k_2, k_3 . The leading order error in the last expression is $O(\Delta x)$, i.e.,

$$M - N_1(\Delta x) \approx k_1 \Delta x. \quad (49)$$

RE combines the “inaccurate” $O(\Delta x)$ approximation $N_1(\Delta x)$ to produce more accurate approximations. Let us consider

$$\begin{aligned} M &= N_1(\Delta x) + k_1 \Delta x + k_2 \Delta x^2 + k_3 \Delta x^3 + \dots \\ M &= N_1\left(\frac{\Delta x}{2}\right) + k_1 \frac{\Delta x}{2} + k_2 \frac{\Delta x^2}{4} + k_3 \frac{\Delta x^3}{8} + \dots \end{aligned} \quad (50)$$

If we subtract the former equation from twice the latter equation we obtain

$$M = N_1\left(\frac{\Delta x}{2}\right) + [N_1\left(\frac{\Delta x}{2}\right) - N_1(\Delta x)] + k_2 \left[\frac{\Delta x^2}{2} - \Delta x^2\right] + k_3 \left[\frac{\Delta x^3}{4} - \Delta x^3\right] \dots \quad (51)$$

Now, let

$$N_2(\Delta x) = N_1\left(\frac{\Delta x}{2}\right) + [N_1\left(\frac{\Delta x}{2}\right) - N_1(\Delta x)]. \quad (52)$$

Then

$$M = N_2(\Delta x) - k_2\Delta x^2 - \frac{3k_3}{4}\Delta x^3 \dots \quad (53)$$

is $O(\Delta x^2)$!

RE can be applied whenever the truncation error has the form $\sum_{j=1}^{m-1} k_j \Delta x^{a_j} + O(\Delta x^{a_m})$, for some k_j , with $a_1 < a_2 < \dots < a_m$.

Many formulae will have only even powers of Δx in the truncation error, e.g., centered finite difference formula for $f'(x)$,

$$\begin{aligned} M &= N_1(\Delta x) + k_1\Delta x^2 + k_2\Delta x^4 + k_3\Delta x^6 + \dots \\ M &= N_1\left(\frac{\Delta x}{2}\right) + k_1\frac{\Delta x^2}{4} + k_2\frac{\Delta x^4}{16} + k_3\frac{\Delta x^6}{64} + \dots \end{aligned} \quad (54)$$

Multiply the last equation by 4, and subtract the two equations to obtain

$$M = \frac{1}{3} \left[4N_1\left(\frac{\Delta x}{2}\right) - N_1(\Delta x) \right] - \frac{k_2}{4}\Delta x^4 - \frac{5k_3}{16}\Delta x^6 + \dots \quad (55)$$

Now let

$$N_2(\Delta x) = \frac{1}{3} \left[4N_1\left(\frac{\Delta x}{2}\right) - N_1(\Delta x) \right] \quad (56)$$

Then the approximation

$$M = N_2(\Delta x) - \frac{k_2}{4}\Delta x^4 + \dots \quad (57)$$

In other words, having 2 different resolutions at $O(\Delta x^2)$ accuracy with the N_1 operator we can combine them to obtain an $O(\Delta x^4)$ approximation!

Now, consider the N_2 approximant and take $h \rightarrow h/2$. We have

$$M = N_2\left(\frac{h}{2}\right) - \frac{k_2}{64}\Delta x^4 - k'_3\Delta x^6 \dots \quad (58)$$

It is straightfoward to show that the approximant

$$N_3(\Delta x) = \frac{1}{15} \left[16N_2\left(\frac{\Delta x}{2}\right) - N_2(\Delta x) \right] \quad (59)$$

is

$$M = N_3(\Delta x) + O(\Delta x^6) \dots \quad (60)$$

In other words, if we have 3 resolutions using the N_1 approximant at 2nd-order accuracy, we can combine them to achieve an approximation that is 6-th order accurate!!!!!!!!!!!!!! This procedure can be done iteratively to obtain higher-accuracy representations, but in practice we rarely have more than 3 different resolutions. Thus, going up to N_3 is most common.

MTH6150: Integration

Charalampos Markakis¹ & Vasileios Paschalidis²

¹ *School of Mathematical Sciences, Queen Mary University of London and*

² *Departments of Astronomy & Physics, University of Arizona, Tucson*

In this note we discuss methods for computing numerical integrals.

I. NUMERICAL INTEGRATION

Just as in the case of differentiation, we often need to integrate data without having analytic functions to work with. Computing integrals of functions/data known at a finite number of grid points is called numerical integration. There are multiple ways one can approximate an integral numerically, and we will review here some of these methods including the trapezoidal, Simpson's 3-point, Gaussian quadrature, and Monte-Carlo techniques. We will also consider multiple integrals.

II. NUMERICAL INTEGRATION IN ONE DIMENSION

An approximation to the integral

$$\int_a^b f(x)dx \quad (1)$$

is called a numerical quadrature, and uses a weighted sum of f at a finite number of points in $[a, b]$, i.e.,

$$\int_a^b f(x)dx = \sum_i a_i f(x_i). \quad (2)$$

So, we select a set of distinct nodes (points) $\{x_0, x_1, \dots, x_n\} \in [a, b]$, and then base our methods on the Lagrange interpolation polynomial

$$f(x) \approx P_n(x) = \sum_{i=0}^n f(x_i) L_i(x) + \prod_{i=0}^n (x - x_i) \frac{f^{(n+1)}(\xi(x))}{(n+1)!} \quad (3)$$

Substituting Eq. (3) in Eq. (1) we obtain

$$\int_a^b f(x)dx = \int_a^b \sum_{i=0}^n f(x_i) L_i(x) dx + \frac{1}{(n+1)!} \int_a^b \prod_{i=0}^n (x - x_i) f^{(n+1)}(\xi(x)) dx \quad (4)$$

or

$$\int_a^b f(x)dx = \sum_{i=0}^n a_i f(x_i) + \frac{1}{(n+1)!} \int_a^b \prod_{i=0}^n (x - x_i) f^{(n+1)}(\xi) dx, \quad (5)$$

where

$$a_i = \int_a^b L_i(x) dx, \quad i = 0, \dots, n \quad (6)$$

A. Basic rules for uniformly spaced grids

1. Trapezoidal method

We will assume that $x_0 = a, x_1 = b$ and define $h = b - a$. Through 2 points we can fit a first-order Lagrange polynomial, i.e., a straight line

$$P_1(x) = \frac{x - x_1}{x_0 - x_1} f(x_0) + \frac{x - x_0}{x_1 - x_0} f(x_1) + \frac{1}{2} (x - x_0)(x - x_1) f''(\xi(x)) \quad (7)$$

Then the quadrature of f is

$$\int_a^b f(x)dx = \int_{x_0}^{x_1} \left[\frac{x-x_1}{x_0-x_1} f(x_0) + \frac{x-x_0}{x_1-x_0} f(x_1) \right] dx + \frac{1}{2} \int_{x_0}^{x_1} (x-x_0)(x-x_1) f''(\xi(x)) dx \quad (8)$$

At this point it is instructive to review the Weighted Mean Value theorem for integrals: *Let f be continuous on $[a, b]$ and g an integrable function on $[a, b]$ that does not change sign on $[a, b]$, then*

$$\int_a^b f(x)g(x)dx = f(c) \int_a^b g(x)dx, \quad (9)$$

for some c in (a, b) .

In the second term on the right-hand-side of Eq. (8) the function $g(x) = (x-x_0)(x-x_1)$ does not change sign on $[a, b]$. Thus, based on the Weighted Mean Value theorem for integrals

$$\frac{1}{2} \int_{x_0}^{x_1} (x-x_0)(x-x_1) f''(\xi(x)) dx = f''(\xi) \frac{1}{2} \int_{x_0}^{x_1} (x-x_0)(x-x_1) dx = f''(\xi) \frac{(x_0-x_1)^3}{12} = -f''(\xi) \frac{h^3}{12}, \quad (10)$$

for some $\xi \in (a, b)$. By use of Eq. (10), Eq. (8) after integrating the first term yields

$$\int_a^b f(x)dx = \frac{x_1-x_0}{2} [f(x_0) + f(x_1)] - f''(\xi) \frac{h^3}{12}, \quad (11)$$

So, we arrive at the trapezoidal rule for uniformly spaced grids

$$\int_a^b f(x)dx = h \left(\frac{f(x_0) + f(x_1)}{2} \right) - f''(\xi) \frac{h^3}{12}, \quad (12)$$

Notice that the leading order truncation error term is $O(h^3)$.

The geometric interpretation of the trapezoidal rule is shown in Fig. 1

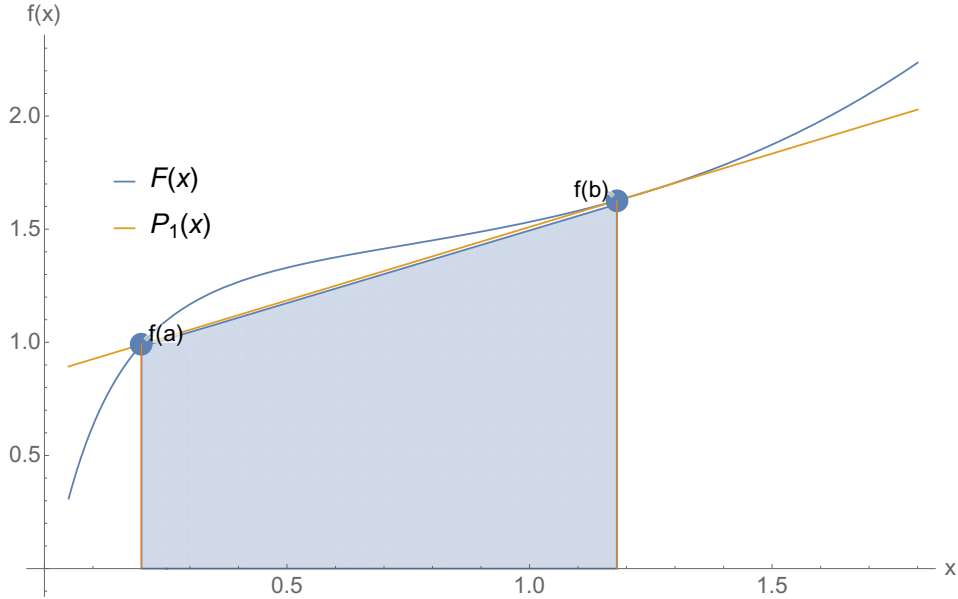


FIG. 1. Trapezoidal approximation to a numerical quadrature. We fit a first-order Lagrange polynomial, i.e, a line, and approximate the integral (area under $f(x)$) from $x = a$ to $x = b$ by the trapezoidal shaded area, whose area simply is $[f(a) + f(b)](b-a)/2$ in agreement with the expression (12).

2. Simpson's 3-point formula

We will now consider three points. Through 3 points we can fit a second-order Lagrange polynomial, i.e.,

$$P_2(x) = \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}f(x_0) + \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}f(x_1) + \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}f(x_2) + \frac{1}{6}(x-x_0)(x-x_1)(x-x_2)f'''(\xi(x)) \quad (13)$$

The quadrature of f then becomes

$$\begin{aligned} \int_a^b f(x)dx &= \int_{x_0}^{x_2} \left[\frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}f(x_0) + \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}f(x_1) + \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}f(x_2) \right] dx \\ &\quad + \frac{1}{6} \int_{x_0}^{x_2} (x-x_0)(x-x_1)(x-x_2)f'''(\xi(x))dx \end{aligned} \quad (14)$$

The first integral in Eq. (14) yields

$$\begin{aligned} \int_{x_0}^{x_2} \left[\frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}f(x_0) + \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}f(x_1) + \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}f(x_2) \right] dx &= \\ \frac{(x_2-x_0)(2x_0-3x_1+x_2)}{6(x_0-x_1)}f(x_0) + \frac{(x_2-x_0)^3}{6(x_1-x_0)(x_1-x_2)}f(x_1) + \frac{(x_2-x_0)(x_0-3x_1+2x_2)}{6(x_2-x_1)}f(x_2) \end{aligned} \quad (15)$$

Equation (15) is Simpson's general 3-point formula that applies to both uniformly spaced and non-uniformly-spaced grids. If we assume that $x_0 = a, x_1 = a + h, x_2 = x_1 + h$ with $h = (b - a)/2$, the formula becomes

$$\int_a^b f(x)dx \approx \frac{h}{3} [f(a) + 4f(\frac{a+b}{2}) + f(b)] \quad (16)$$

Exercise 1: Consider the fact that $f(a)$, $f(b)$, $f((b+c)/2)$ all have a round-off error that is bounded, and assume the highest order derivative in the leading truncation error is also bounded. Is there a round-off error instability in numerical quadratures computed with the trapezoidal or Simpson's rules? Explain.

Now, obtaining the error with this approach will yield an $O(h^4)$ leading truncation error, but the method is actually $O(h^5)$. To demonstrate this we will provide an alternative derivation of Simpson's 3-point formula which reveals the true leading order truncation error.

Let's consider the Taylor expansion up to 4th-order of f at $x = x_1$

$$f(x) = f(x_1) + f'(x_1)(x-x_1) + \frac{1}{2}f''(x_1)(x-x_1)^2 + \frac{1}{6}f'''(x_1)(x-x_1)^3 + \frac{1}{24}f^{(4)}(\xi(x))(x-x_1)^4 \quad (17)$$

We can then substitute Eq. (17) in $\int_{x_0}^{x_2} f(x)dx$ and perform the integration to obtain

$$\begin{aligned} \int_{x_0}^{x_2} f(x)dx &= \left[f(x_1)(x-x_1) + \frac{1}{2}f'(x_1)(x-x_1)^2 + \frac{1}{6}f''(x_1)(x-x_1)^3 + \frac{1}{24}f'''(\xi(x_1))(x-x_1)^4 \right]_{x_0}^{x_2} \\ &\quad + \frac{1}{24} \int_{x_0}^{x_2} f^{(4)}(\xi(x))(x-x_1)^4 dx \end{aligned} \quad (18)$$

Note that the integral of $f(x_1)$ is just $f(x_1)x$ and not $f(x_1)(x-x_1)$. However, since this is a definite integral we can add a constant $c = -f(x_1)x_1$ and the result of the integration remains unaffected. The integral on the right-hand-side of Eq. (18) is already in a form that allows us to use the Weighted Mean Value Theorem for integrals, because the function $g(x) = (x-x_1)^4$ does not change sign in $[x_0, x_2]$. Thus,

$$\frac{1}{24} \int_{x_0}^{x_2} f^{(4)}(\xi(x))(x-x_1)^4 dx = \frac{1}{24} f^{(4)}(\xi_1) \int_{x_0}^{x_2} (x-x_1)^4 dx = \frac{1}{120} f^{(4)}(\xi_1)(x-x_1)^5 \Big|_{x_0}^{x_2} = \frac{4}{15} f^{(4)}(\xi_1)h^5. \quad (19)$$

The first term on the right-hand-side of Eq. (18) yields

$$\int_{x_0}^{x_2} f(x)dx = 2hf(x_1) + \frac{h^3}{3}f''(x_1), \quad (20)$$

but the 2nd-order accurate centered difference for the second derivative is

$$f''(x_1) = \frac{1}{h^2} [f(x_1 - h) - 2f(x_1) + f(x_1 + h)] + O(h^2) = \frac{1}{h^2} [f(x_0) - 2f(x_1) + f(x_2)] + O(h^2) \quad (21)$$

Hence the last two equations and Eq. (19) imply that the Simpson 3-point rule is

$$\int_{x_0}^{x_2} f(x)dx = \frac{h}{3} [f(x_0) + 4f(x_1) + f(x_2)] + O(h^5), \quad (22)$$

where the $O(h^5)$ leading order truncation error is now evident, and again this applies to uniformly spaced grids: $x_1 = x_0 + h$, $x_2 = x_1 + h = x_0 + 2h$. Notice that Eq. (22) is the same as Eq. (16) because $x_1 = (x_0 + x_2)/2$.

The geometric interpretation of Simpson's 3-point rule is shown in Fig. 1

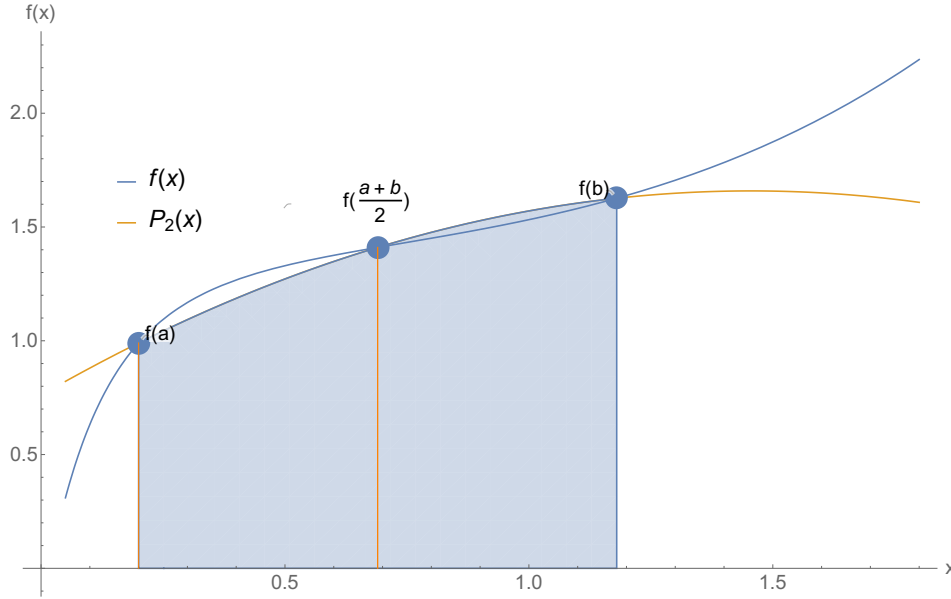


FIG. 2. Simpson's 3-point rule approximation to a numerical quadrature. We approximate the integral (area under $f(x)$) from $x = a$ to $x = b$ by the area under the unique quadratic function that passes through points $(a, f(a))$, $((a+b)/2, f((a+b)/2))$, $(b, f(b))$.

For all practical purposes Simpson's 3-point rule suffices to achieve a very accurate result, because it is 5th-order convergent.

FIG. 3. Simpson's 3-point rule in the case of a large integration domain. Notice how the area of the shaded region does not match the area under the function $f(x)$ because the spacing between the 3 points is large.

Exercise 2: Compute $\int_0^h \exp(x)dx$: a) exactly, b) using the trapezoidal rule, and c) using Simpson's 3-point rule. Use your results to calculate the absolute error (difference) of the Simpson result with respect to the exact result. Make a plot of the relative error vs h . How fast does the absolute error decay with h ? Note that in gnuplot you can define functions by simply typing $f(x)=\exp(x)$.

B. Composite Integration

When using either the trapezoidal or Simpson's 3-point rule, achieving an accurate result requires a sufficiently small h . But, the domain of integration is typically going to be too large for, e.g., 3 points to yield an accurate result as shown in Fig. 3. In this case we will break the interval into multiple small intervals and use either the trapezoidal rule or Simpson's 3-point rule. Geometrically this is shown in Fig. 4

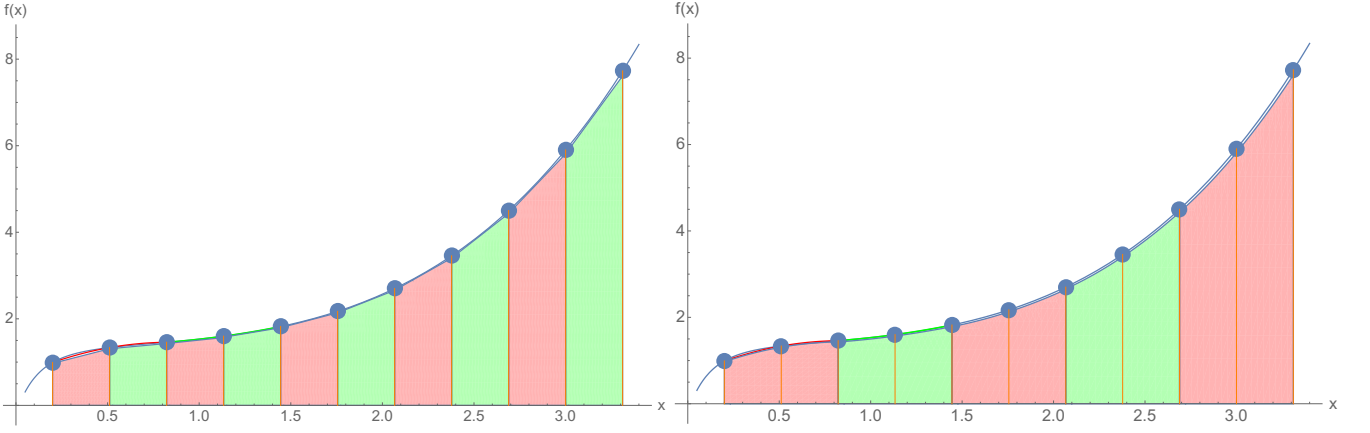


FIG. 4. Geometric representation of composite integration. On the left is the composite trapezoidal rule. The alternating colored shaded regions each of size h show the intervals that each separate trapezoidal rule is applied. Same as on the left but for the composite Simpson's 3-point rule. Notice that the shaded regions now have a size of $2h$ because we need 3-points to apply Simpson's 3-point rule.

1. Composite Simpson's 3-point rule

For convenience we re-write Simpson's 3-point rule

$$\int_{x_0}^{x_2} f(x)dx = \frac{h}{3} [f(x_0) + 4f(x_0 + h) + f(x_0 + 2h)]. \quad (23)$$

Our goal is to have an accurate representation of the integral $\int_a^b f(x)dx$. First we choose an even number n that will divide the interval $[a, b]$ into n subintervals. Then we will apply Simpson's rule on each **pair** of subintervals. We will define the grid spacing

$$h = \frac{b - a}{n} \quad (24)$$

and the coordinate points of the subintervals

$$x_i = a + j \cdot h, \quad j = 0, 1, \dots, n \quad (25)$$

The integral can be analytically rewritten as

$$\int_a^b f(x)dx = \int_{x_0}^{x_2} f(x)dx + \int_{x_2}^{x_4} f(x)dx + \int_{x_4}^{x_6} f(x)dx + \dots + \int_{x_{n-2}}^{x_n} f(x)dx = \sum_{i=1}^{n/2} \int_{x_{2i-2}}^{x_{2i}} f(x)dx \quad (26)$$

By use of Eq. (23) the last equation becomes

$$\int_a^b f(x)dx = \sum_{i=1}^{n/2} \int_{x_{2i-2}}^{x_{2i}} f(x)dx = \frac{h}{3} \sum_{i=1}^{n/2} [f(x_{2i-2}) + 4f(x_{2i-1}) + f(x_{2i})] = \frac{h}{3} \left[\sum_{i=1}^{n/2} f(x_{2i-2}) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}) + \sum_{i=1}^{n/2} f(x_{2i}) \right] \quad (27)$$

The sum $\sum_{i=1}^{n/2} f(x_{2i-2})$ can be recast in a more convenient form by introducing a new counter $k = i - 1$. For $i = 1$ then $k = 0$ and for $i = n/2$, $k = n/2 - 1$. Thus, the sum can be transformed to $\sum_{k=0}^{n/2-1} f(x_{2k})$. Given that k is a dummy variable we can rename it back to i , hence

$$\sum_{i=1}^{n/2} f(x_{2i-2}) = \sum_{i=0}^{n/2-1} f(x_{2i}) = f(x_0) + \sum_{i=1}^{n/2-1} f(x_{2i}) \quad (28)$$

The last sum in Eq. (27) can also be rewritten as follows

$$\sum_{i=1}^{n/2} f(x_{2i}) = f(x_n) + \sum_{i=1}^{n/2-1} f(x_{2i}) \quad (29)$$

By substituting Eqs. (28) and (29) in (27) yields the final form of the composite Simpson's 3-point rule

$$\int_a^b f(x)dx = \frac{h}{3} \left[f(x_0) + f(x_n) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}) + 2 \sum_{i=1}^{n/2-1} f(x_{2i}) \right] + O(h^4) \quad (30)$$

Notice that the leading truncation error is $O(h^4)$ and not $O(h^5)$. This can be shown, but we will not show it here. Despite the fact that the composite method is $O(h^4)$ it is still more accurate than the standard 3-point method because h is much smaller in the composite method.

Algorithm for Simpson's composite rule: Goal to approximate $I = \int_a^b f(x)dx$

1. INPUT: $f(x)$, end points a, b and an even positive integer n .
OUTPUT: approximation to I .
2. Set $h = (b - a)/n$.
3. Set $xI = f(a) + f(b)$; $xI1 = 0$ (sum of $f(x_{2i-1})$), $xI2 = 0$ (sum of $f(x_{2i})$)
4. For $i = 1, \dots, n-1$ do steps 5 and 6.
5. Set $x = a + ih$
6. If (i is even) set $xI2 = xI2 + f(x)$
else set $xI1 = xI1 + f(x)$
7. Set $xI = h(xI + 2 \cdot xI2 + 4 \cdot xI1)/3$.
8. Return (xI)

2. Composite trapezoidal rule

Following a procedure similar to the one we followed for the composite Simpson's rule, we can derive the composite trapezoidal rule which is

$$\int_a^b f(x)dx = \frac{h}{2} \left[f(a) + 2 \sum_{i=1}^{n-1} f(x_i) + f(b) \right] + O(h^2), \quad (31)$$

where again $x_i = a + jh$, $j = 0, 1, \dots, n$. Note that for the trapezoidal rule n can be either even or odd.

Algorithm for composite trapezoidal rule: Goal to approximate $I = \int_a^b f(x)dx$

1. INPUT: $f(x)$, end points a, b and a positive integer n .
OUTPUT: approximation to I .
2. Set $h = (b - a)/n$.
3. Set $xI = f(a) + f(b)$; $xI1 = 0$ (sum of $f(x_i)$)
4. For $i = 1, \dots, n-1$ do steps 5 and 6.
5. Set $x = a + i \cdot h$
6. Set $xI1 = xI1 + f(x)$
7. Set $xI = h(xI + 2 \cdot xI1)/2$.
8. Return (xI)

Exercise 3: Use the trapezoidal rule to compute the integral $I = \int_{-1}^1 \cos(x)dx$. The exact value of the integral is $I = 2 \sin(1) = 1.682941969615793$. Run your code for $n = 50, 100, 200, 400$. Plot the relative error as a function of n . What value of n does your plot predict that you need to compute I to within an accuracy of 1 part in 10^7 ? Run your code with your predicted value for n and measure the relative error. Was your prediction correct?

C. Gaussian Quadrature

So far we focused on uniformly spaced grid points (nodes). Gaussian Quadrature is an algorithm that chooses the nodes such that in the approximation

$$\int_a^b f(x)dx = \sum_{i=1}^n c_i f(x_i) \quad (32)$$

the error is minimized in the sense that the result is **exact** for the class of polynomials of degree $2n - 1$. This is because Gaussian Quadrature treats both c_i and x_i as parameters, i.e., there are $2n$ parameters which is the number of parameters in a polynomial of degree $2n - 1$.

For example, let's consider the case $n = 2$ for $x \in [-1, 1]$. We wish to determine c_1, c_2, x_1, x_2 such that

$$\int_{-1}^1 f(x)dx = \sum_{i=1}^n c_i f(x_i) = c_1 f(x_1) + c_2 f(x_2) \quad (33)$$

is exact whenever $f(x)$ is a polynomial of degree $2n - 1 = 2 \cdot 2 - 1 = 3$ or less, i.e.,

$$f(x) = a_0 + a_1 x + a_2 x^2 + a_3 x^3. \quad (34)$$

To achieve this we will first integrate this polynomial

$$\int_{-1}^1 f(x)dx = a_0 \int_{-1}^1 dx + a_1 \int_{-1}^1 x dx + a_2 \int_{-1}^1 x^2 dx + a_3 \int_{-1}^1 x^3 dx. \quad (35)$$

We can now consider the function $f(x)$ as a linear combination of the 4 functions $f_0(x) = 1$, $f_1(x) = x$, $f_2(x) = x^2$, $f_3(x) = x^3$, i.e., $f(x) = a_0 f_0(x) + a_1 f_1(x) + a_2 f_2(x) + a_3 f_3(x) = \sum_{i=0}^3 a_i f_i(x)$. Thus, if for each $f_i(x)$, $i = 0, 1, 2, 3$ we satisfy $\int_{-1}^1 f_i(x)dx = c_1 f_i(x_1) + c_2 f_i(x_2)$, then we will achieve Eq. (33) because

$$\begin{aligned} \int_{-1}^1 f(x)dx &= \int_{-1}^1 \sum_i a_i f_i(x)dx = \sum_i a_i \int_{-1}^1 f_i(x)dx = \sum_i a_i (c_1 f_i(x_1) + c_2 f_i(x_2)) = \sum_i a_i c_1 f_i(x_1) + \sum_i a_i c_2 f_i(x_2) \\ &= c_1 \sum_i a_i f_i(x_1) + c_2 \sum_i a_i f_i(x_2) = c_1 f(x_1) + c_2 f(x_2). \end{aligned} \quad (36)$$

Thus, for the 4 $f_i(x)$ polynomials we need to satisfy $\int_{-1}^1 f_i(x)dx = c_1 f_i(x_1) + c_2 f_i(x_2)$, which will provide 4 equations for the 4 parameters c_1, c_2, x_1, x_2 :

$$\begin{aligned} c_1 f_0(x_1) + c_2 f_0(x_2) &= c_1 \cdot 1 + c_2 \cdot 1 = \int_{-1}^1 1 dx = 2 \\ c_1 f_1(x_1) + c_2 f_1(x_2) &= c_1 \cdot x_1 + c_2 \cdot x_2 = \int_{-1}^1 x dx = 0 \\ c_1 f_2(x_1) + c_2 f_2(x_2) &= c_1 \cdot x_1^2 + c_2 \cdot x_2^2 = \int_{-1}^1 x^2 dx = \frac{2}{3} \\ c_1 f_3(x_1) + c_2 f_3(x_2) &= c_1 \cdot x_1^3 + c_2 \cdot x_2^3 = \int_{-1}^1 x^3 dx = 0. \end{aligned} \quad (37)$$

This system can be solved easily by eliminating c_1 and c_2 from the first two equations, e.g., solve the first w.r.t c_1 and then replace the result in the second equation to express c_2 in terms of x_1 and x_2 . Then substitute c_1 and c_2 in terms of x_1 and x_2 in the last two equations that will provide a system of 2 equations for the 2 unknowns x_1 and x_2 . The solution of the system is $c_1 = 1$, $c_2 = 1$, $x_1 = -\sqrt{3}/3 = -x_2$. Therefore, the $n = 2$ Gaussian quadrature is

$$\int_{-1}^1 f(x)dx = f\left(-\frac{\sqrt{3}}{3}\right) + f\left(\frac{\sqrt{3}}{3}\right), \quad (38)$$

which yields an **exact** result for any $f(x)$ that is a polynomial up to degree $n = 3$!

The same approach can yield results for degree $2n - 1$ polynomials, but this turns out to be more involved. By using Legendre polynomials, the process can be expedited. So, let's introduce the Legendre polynomials first.

Legendre polynomials ($P_n(x)$) are orthogonal and form a complete basis. Some of their properties are:

1. For each n , $P_n(x)$ is a monic polynomial of degree n , i.e., the coefficient of the smallest power of x that appears in the polynomial is 1.
2. $\int_{-1}^1 P(x)P_n(x)dx = 0$, when $P(x)$ is a polynomial of degree $< n$.
3. The first 5 Legendre polynomials are: $P_0(x)=1$, $P_1(x)=x$, $P_2(x)=x^2-1/3$, $P_3(x)=x^3-3x/5$, $P_4(x)=x^4-6x^2/7+3/35$.

4. Theorem

Assume $x_1, x_2, \dots, x_n$ are the roots of the n -th Legendre polynomial $P_n(x)$ and for each $i = 1, 2, \dots, n$, consider the numbers

$$c_i = \int_{-1}^1 \prod_{j=1, j \neq i}^n \frac{x - x_j}{x_i - x_j} dx \quad (39)$$

If $P(x)$ is any polynomial of degree less than $2n - 1$, then

$$\int_{-1}^1 P(x)dx = \sum_{i=1}^n c_i P(x_i). \quad (40)$$

Let's consider the integral

$$I = \int_{-1}^1 e^x \cos(x) dx = 1.933421496200713, \quad (41)$$

and let's approximate it using Gaussian quadrature with $n = 3$, i.e.,

$$\tilde{I} = c_1 f(x_1) + c_2 f(x_2) + c_3 f(x_3). \quad (42)$$

where x_1, x_2, x_3 are the roots of the Legendre polynomial $P_3(x)$, i.e., $x^3 - 3x/5 = 0 \Rightarrow x = 0$ or $x^2 - 3/5 = 0 \Rightarrow x = \pm\sqrt{3/5}$. Thus, $x_1 = -\sqrt{3/5}$, $x_2 = 0$, and $x_3 = \sqrt{3/5}$. Then

$$\begin{aligned} c_1 &= \int_{-1}^1 \prod_{j=1, j \neq 1}^n \frac{x - x_j}{x_1 - x_j} dx = \int_{-1}^1 \frac{x - x_2}{x_1 - x_2} \frac{x - x_3}{x_1 - x_3} dx = \frac{5}{9} \\ c_2 &= \int_{-1}^1 \prod_{j=1, j \neq 2}^n \frac{x - x_j}{x_2 - x_j} dx = \int_{-1}^1 \frac{x - x_1}{x_2 - x_1} \frac{x - x_3}{x_2 - x_3} dx = \frac{8}{9} \\ c_3 &= \int_{-1}^1 \prod_{j=1, j \neq 3}^n \frac{x - x_j}{x_3 - x_j} dx = \int_{-1}^1 \frac{x - x_1}{x_3 - x_1} \frac{x - x_2}{x_3 - x_2} dx = \frac{5}{9} \end{aligned} \quad (43)$$

Thus,

$$\tilde{I} = \frac{5}{9} f\left(-\sqrt{\frac{3}{5}}\right) + \frac{8}{9} f(0) + \frac{5}{9} f\left(\sqrt{\frac{3}{5}}\right) = 1.933390469264298. \quad (44)$$

Thus, the relative error is $|(\tilde{I} - I)/I| = 1.6 \times 10^{-5}!!!$ Notice that Eq. (44) applies to any function, and is **exact** for any polynomial of degree 5 or less.

Arbitrary intervals: The general procedure for Gaussian quadrature outlined earlier applies to the interval $[-1, 1]$. However, we will often have general integration domains $[a, b]$. We can still apply Eq. (40) by a simple transformation of the integration variable. An integral of the form

$$\int_a^b f(x) dx \quad (45)$$

can easily be transformed to the form

$$\int_{-1}^1 f(t) dt \quad (46)$$

if we let

$$t = \frac{2x - a - b}{b - a}, \quad (47)$$

which implies

$$x = \frac{1}{2}[(b - a)t + a + b]. \quad (48)$$

Thus, when $x = a$, $t = -1$, and when $x = b$, $t = 1$. We then have $dx = \frac{b-a}{2}dt$, and hence

$$\int_a^b f(x)dx = \frac{b-a}{2} \int_{-1}^1 f\left(\frac{(b-a)t + (b+a)}{2}\right) dt. \quad (49)$$

The last expression allows the application of Eq. (40) to any function in any integration interval.

Exercise 4: Use the $n = 3$ Gaussian quadrature method to compute the integral $\mathbf{I} = \int_0^\pi \mathbf{x} \cos(\mathbf{x})d\mathbf{x}$. The exact value of the integral is $\mathbf{I} = -2$. What is the relative error of the $n = 3$ Gaussian quadrature? **Answer: 0.00406**

III. NUMERICAL INTEGRATION IN MULTIPLE DIMENSIONS: MULTIPLE INTEGRALS

We will consider double integrals because the extension to three dimensions is straightforward. We use to approximate the integral

$$\int \int f(x, y) dx dy \quad (50)$$

We will use properties of multiple integrals to write this as

$$\int \int f(x, y) dx dy = \int_a^b \left(\int_c^d f(x, y) dy \right) dx \quad (51)$$

In other words first we will approximate the integral in the parenthesis $\int_c^d f(x, y) dy$ and then we will do the integral over dx .

A. Trapezoidal rule

Using the trapezoidal rule the integral $\int_c^d f(x, y) dy$ becomes

$$\int_c^d f(x, y) dy = \frac{f(x, c) + f(x, d)}{2} (d - c) + O([d - c]^2) \quad (52)$$

Then we have to integrate

$$\begin{aligned} \int \int f(x, y) dx dy &= \int_a^b \left(\int_c^d f(x, y) dy \right) dx = \int_a^b \left(\frac{f(x, c) + f(x, d)}{2} (d - c) + O([d - c]^2) \right) dx \\ &= \frac{d - c}{2} \left(\int_a^b f(x, c) dx + \int_a^b f(x, d) dx \right) + \int_a^b O([d - c]^2) dx \end{aligned} \quad (53)$$

For which we can use the trapezoidal formula again to obtain

$$\int \int f(x, y) dx dy = \frac{d - c}{2} \left(\frac{f(a, c) + f(b, c)}{2} (b - a) + \frac{f(a, d) + f(b, d)}{2} (b - a) + O[(b - a)^2] \right) + O[(d - c)^2 (b - a)] \quad (54)$$

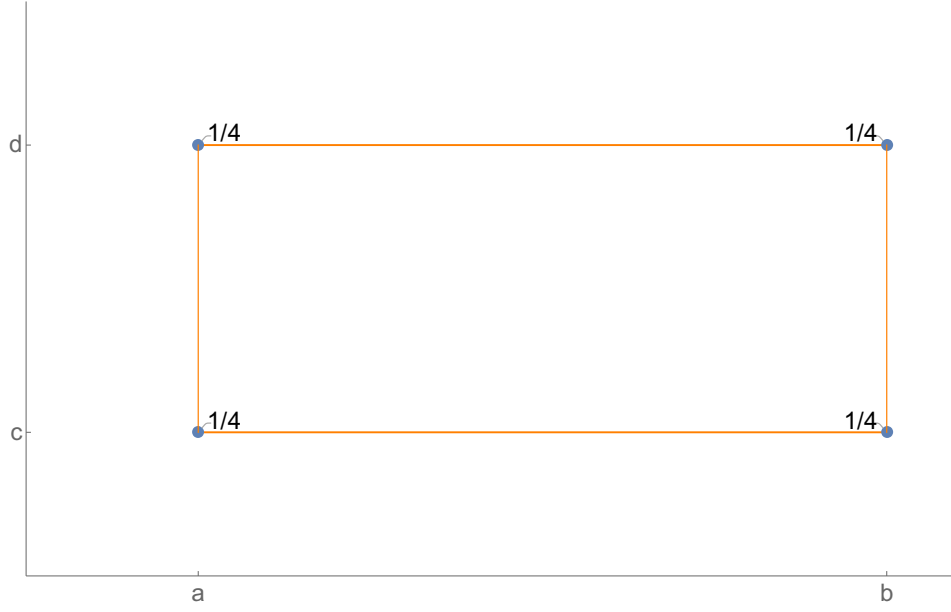


FIG. 5. Stencil coefficients for 2D trapezoidal integration. Each vertex value is multiplied by the weight $1/4$, then the values summed over and multiplied by the area of the rectangle.

or

$$\iint f(x, y) dx dy = \frac{f(a, c) + f(b, c) + f(a, d) + f(b, d)}{4} (d - c)(b - a) + O[(b - a)^2(d - c) + (d - c)^2(b - a)] \quad (55)$$

In other words, the trapezoidal rule for the integral is equal to the average value of $f(x, y)$ within the rectangle $x \in [a, b]$, $y \in [c, d]$ times the area of the rectangle $(d - c)(b - a)$. This is depicted in Fig. 5

We can also apply the composite trapezoidal rule in the rectangle $[a, b] [c, d]$, using as midpoints the coordinates $(a + b)/2$ and $(c + d)/2$. Then it is straightforward to show

$$\begin{aligned} \iint f(x, y) dx dy &= \frac{(d - c)(b - a)}{16} [f(a, c) + f(b, c) + f(a, d) + f(b, d) \\ &\quad + 2 \left(f\left(\frac{a + b}{2}, c\right) + f\left(\frac{a + b}{2}, d\right) + f\left(a, \frac{c + d}{2}\right) + f\left(b, \frac{c + d}{2}\right) \right) + 4f\left(\frac{a + b}{2}, \frac{c + d}{2}\right)] \\ &\quad + O((b - a)(d - c)[(d - c)^2 + (b - a)^2]) \end{aligned} \quad (56)$$

which is depicted in Fig. 6

B. Simpson's rule

We can also perform the integration using Simpson's rule

$$\int_c^d f(x, y) dy = \frac{f(x, c) + f(x, d) + 4f(x, \frac{c+d}{2})}{2} (d - c) + O([d - c]^5) \quad (57)$$

and again we have

$$\begin{aligned} \iint f(x, y) dx dy &= \int_a^b \left(\int_c^d f(x, y) dy \right) dx = \int_a^b \left(\frac{f(x, c) + f(x, d) + 4f(x, \frac{c+d}{2})}{2} (d - c) + O[(d - c)^5] \right) dx \\ &= \frac{d - c}{2} \left(\int_a^b f(x, c) dx + \int_a^b f(x, d) dx + 4 \int_a^b f(x, \frac{c+d}{2}) dx \right) + \int_a^b O[(d - c)^5] dx \end{aligned} \quad (58)$$

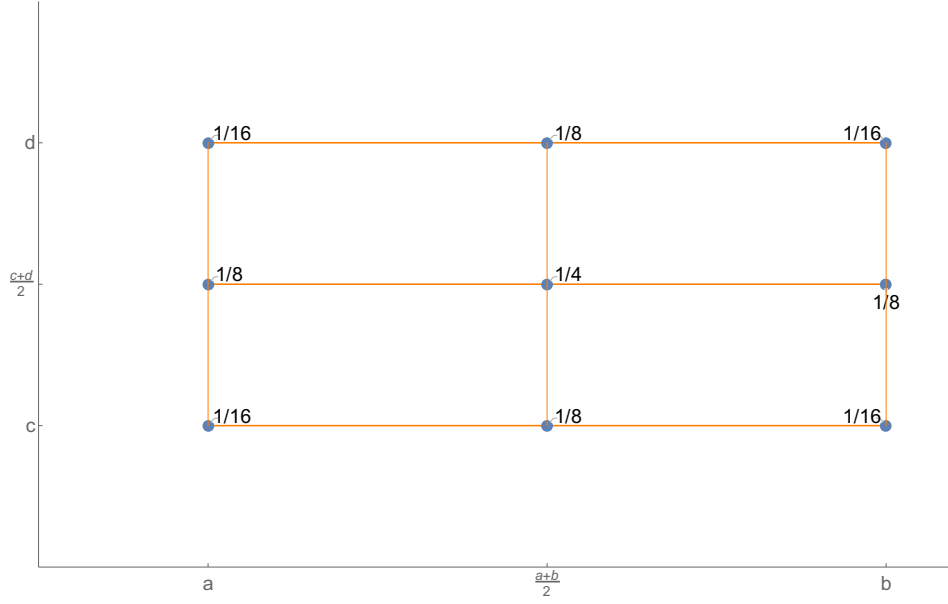


FIG. 6. Stencil coefficients for 2D composite trapezoidal integration. Each vertex value is multiplied by the weight shown, then the values summed over and multiplied by the area of the rectangle.

for which we can use Simpson's 3-point rule again to obtain

$$\begin{aligned} \iint f(x, y) dx dy &= \frac{d-c}{2} \left[\left(f(a, c) + f(b, c) + 4f\left(\frac{a+b}{2}, c\right) \right) \frac{b-a}{2} + O[(b-a)^5] \right. \\ &\quad + \left(f(a, d) + f(b, d) + 4f\left(\frac{a+b}{2}, d\right) \right) \frac{b-a}{2} + O[(b-a)^5] \\ &\quad + 4 \left(f(a, \frac{c+d}{2}) + f(b, \frac{c+d}{2}) + 4f\left(\frac{a+b}{2}, \frac{c+d}{2}\right) \right) \frac{b-a}{2} + O[(b-a)^5] \Big] \\ &\quad + O[(d-c)^5(b-a)] \end{aligned} \quad (59)$$

which simplifies to

$$\begin{aligned} \iint f(x, y) dx dy &= \frac{(b-a)(d-c)}{4} \left[\left(f(a, c) + f(b, c) + f(a, d) + f(b, d) \right) \right. \\ &\quad + 4 \left(f\left(\frac{a+b}{2}, c\right) + f\left(\frac{a+b}{2}, d\right) + f(a, \frac{c+d}{2}) + f(b, \frac{c+d}{2}) \right) \\ &\quad \left. + 16f\left(\frac{a+b}{2}, \frac{c+d}{2}\right) \right] + O[(d-c)(b-a)^5] + (d-c)^5(b-a) \end{aligned} \quad (60)$$

This stencil is depicted in Fig. 7

C. Composite Simpson's rule

Let's now apply the composite Simpson's rule in double integrals. We will again use the property

$$\iint f(x, y) dx dy = \int_a^b \left(\int_c^d f(x, y) dy \right) dx \quad (61)$$

First let $y_j = c + j \cdot h_y$, $j = 0, 1, \dots, m$ with $h_y = (d-c)/m$ and m is an even number. Then

$$\left(\int_c^d f(x, y) dy \right) = \frac{h_y}{3} \left[f(x, y_0) + 2 \sum_{j=1}^{(m/2)-1} f(x, y_{2j}) + 4 \sum_{j=1}^{m/2} f(x, y_{2j-1}) + f(x, y_m) \right] \quad (62)$$

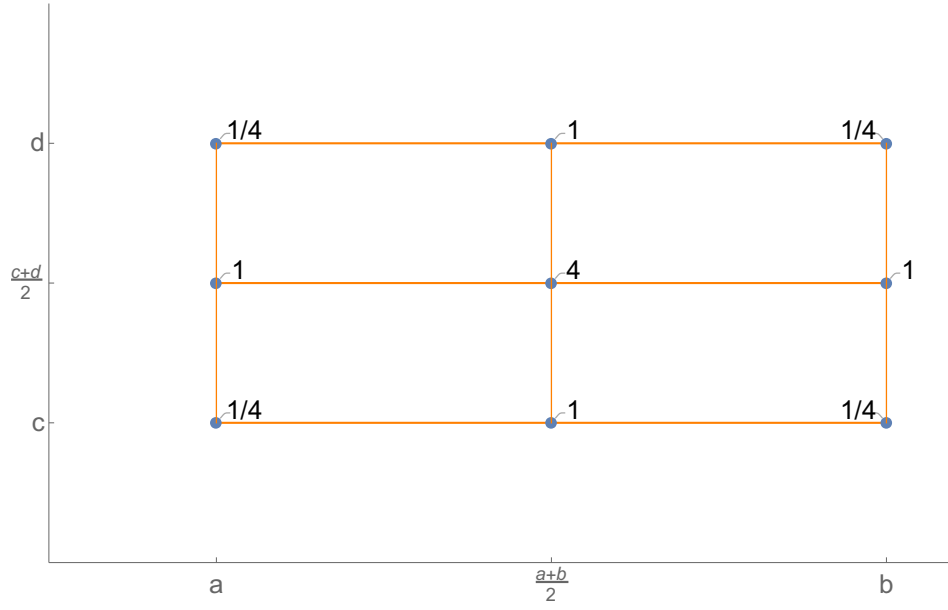


FIG. 7. Stencil coefficients for 2D Simpson's integration. Each vertex value is multiplied by the weight shown, then the values summed over and multiplied by the area of the rectangle.

Then Eq. (61) becomes

$$\begin{aligned} \iint f(x, y) dx dy &= \int_a^b \left(\frac{h_y}{3} \left[f(x, y_0) + 2 \sum_{j=1}^{(m/2)-1} f(x, y_{2j}) + 4 \sum_{j=1}^{m/2} f(x, y_{2j-1}) + f(x, y_m) \right] \right) dx \\ &= \frac{h_y}{3} \left[\int_a^b f(x, y_0) dx + 2 \sum_{j=1}^{(m/2)-1} \int_a^b f(x, y_{2j}) dx + 4 \sum_{j=1}^{m/2} \int_a^b f(x, y_{2j-1}) dx + \int_a^b f(x, y_m) dx \right] \end{aligned} \quad (63)$$

Now, we need to define a discrete grid in the x direction, so we let $x_i = a + i \cdot h_x$, $i = 0, 1, \dots, n$ with $h_x = (b - a)/n$, where again n must be an even number. We will next use the composite Simpson's rule to treat each of the 4 integrals appearing in Eq. (63) separately:

$$\int_a^b f(x, y_0) dx = \frac{h_x}{3} \left[f(x_0, y_0) + 2 \sum_{i=1}^{(n/2)-1} f(x_{2i}, y_0) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}, y_0) + f(x_n, y_0) \right] \quad (64)$$

$$\int_a^b f(x, y_m) dx = \frac{h_x}{3} \left[f(x_0, y_m) + 2 \sum_{i=1}^{(n/2)-1} f(x_{2i}, y_m) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}, y_m) + f(x_n, y_m) \right] \quad (65)$$

$$\int_a^b f(x, y_{2j}) dx = \frac{h_x}{3} \left[f(x_0, y_{2j}) + 2 \sum_{i=1}^{(n/2)-1} f(x_{2i}, y_{2j}) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}, y_{2j}) + f(x_n, y_{2j}) \right] \quad (66)$$

$$\int_a^b f(x, y_{2j-1}) dx = \frac{h_x}{3} \left[f(x_0, y_{2j-1}) + 2 \sum_{i=1}^{(n/2)-1} f(x_{2i}, y_{2j-1}) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}, y_{2j-1}) + f(x_n, y_{2j-1}) \right] \quad (67)$$

Let's now treat the last two integrals because these are further summed over j in (63). In particular, we have the

term

$$2 \sum_{j=1}^{(m/2)-1} \int_a^b f(x, y_{2j}) dx = \frac{h_x}{3} \left[2 \sum_{j=1}^{(m/2)-1} f(x_0, y_{2j}) + 4 \sum_{j=1}^{(m/2)-1} \sum_{i=1}^{(n/2)-1} f(x_{2i}, y_{2j}) + 8 \sum_{j=1}^{(m/2)-1} \sum_{i=1}^{n/2} f(x_{2i-1}, y_{2j}) + 2 \sum_{j=1}^{(m/2)-1} f(x_n, y_{2j}) \right] \quad (68)$$

and the term

$$4 \sum_{j=1}^{m/2} \int_a^b f(x, y_{2j-1}) dx = \frac{h_x}{3} \left[4 \sum_{j=1}^{m/2} f(x_0, y_{2j-1}) + 8 \sum_{j=1}^{m/2} \sum_{i=1}^{(n/2)-1} f(x_{2i}, y_{2j-1}) + 16 \sum_{j=1}^{m/2} \sum_{i=1}^{n/2} f(x_{2i-1}, y_{2j-1}) + 4 \sum_{j=1}^{m/2} f(x_n, y_{2j-1}) \right] \quad (69)$$

To sum up Eq. (63) becomes

$$\int \int f(x, y) dx dy = \frac{h_x h_y}{9} (I_1 + I_2 + I_3 + I_4), \quad (70)$$

where

$$I_1 = f(x_0, y_0) + 2 \sum_{i=1}^{(n/2)-1} f(x_{2i}, y_0) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}, y_0) + f(x_n, y_0), \quad (71)$$

$$I_2 = f(x_0, y_m) + 2 \sum_{i=1}^{(n/2)-1} f(x_{2i}, y_m) + 4 \sum_{i=1}^{n/2} f(x_{2i-1}, y_m) + f(x_n, y_m), \quad (72)$$

$$I_3 = 2 \sum_{j=1}^{(m/2)-1} f(x_0, y_{2j}) + 4 \sum_{j=1}^{(m/2)-1} \sum_{i=1}^{(n/2)-1} f(x_{2i}, y_{2j}) + 8 \sum_{j=1}^{(m/2)-1} \sum_{i=1}^{n/2} f(x_{2i-1}, y_{2j}) + 2 \sum_{j=1}^{(m/2)-1} f(x_n, y_{2j}), \quad (73)$$

and

$$I_4 = 4 \sum_{j=1}^{m/2} f(x_0, y_{2j-1}) + 8 \sum_{j=1}^{m/2} \sum_{i=1}^{(n/2)-1} f(x_{2i}, y_{2j-1}) + 16 \sum_{j=1}^{m/2} \sum_{i=1}^{n/2} f(x_{2i-1}, y_{2j-1}) + 4 \sum_{j=1}^{m/2} f(x_n, y_{2j-1}). \quad (74)$$

The same principles can be applied to perform the double integral using the composite trapezoidal rule or Gaussian integration, but we will not expand on the latter further. The former is an homework exercise.

D. Non-rectangular integration domains

We can also have integration in non-rectangular domains, e.g.,

$$\int_a^b \int_{c(x)}^{d(x)} f(x, y) dx dy \text{ or } \int_c^d \int_{a(y)}^{b(y)} f(x, y) dx dy \quad (75)$$

Let 's consider the left integral, as the right one can be treated in a similar way. Simpson's 3-point rule for

$$\int_a^b \int_{c(x)}^{d(x)} f(x, y) dx dy \quad (76)$$

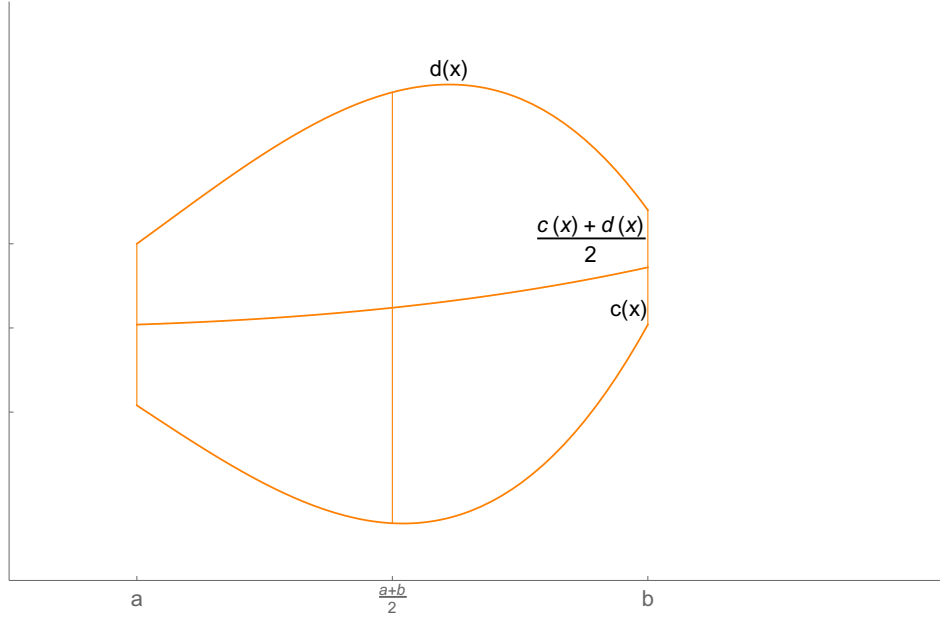


FIG. 8. Depiction of Simpson's method for a double integral in a non-rectangular integration domain. In this case the step size of the integration over y depends on x , i.e., it is variable.

will have a fixed step size for x , i.e., $h_x = (b - a)/2$, but the step size for the integration over y , will have a variable step size $h_y(x) = (d(x) - c(x))/2$. As depicted in Fig. 8

We will again use the multiple integral property

$$\int_a^b \int_{c(x)}^{d(x)} f(x, y) dx dy = \int_a^b \left(\int_{c(x)}^{d(x)} f(x, y) dy \right) dx \quad (77)$$

Simpson's method for the interior integral gives

$$\int_{c(x)}^{d(x)} f(x, y) dy = \frac{h_y(x)}{3} [f(x, c(x)) + 4f(x, c(x) + h_y(x)) + f(x, d(x))] \quad (78)$$

Substituting the last expression in Eq. (77) and applying Simpson's rule yields

$$\begin{aligned} \int_a^b \int_{c(x)}^{d(x)} f(x, y) dx dy &= \int_a^b \left(\frac{h_y(x)}{3} [f(x, c(x)) + 4f(x, c(x) + h_y(x)) + f(x, d(x))] \right) dx \\ &= \frac{1}{3} \int_a^b h_y(x) f(x, c(x)) dx + \frac{4}{3} \int_a^b h_y(x) f(x, c(x) + h_y(x)) dx + \frac{1}{3} \int_a^b h_y(x) f(x, d(x)) dx \\ &= \frac{1}{3} I_1 + \frac{4}{3} I_2 + \frac{1}{3} I_3, \end{aligned} \quad (79)$$

where

$$I_1 = \int_a^b h_y(x) f(x, c(x)) dx = \frac{h_x}{3} \left[h_y(a) f(a, c(a)) + 4h_y\left(\frac{a+b}{2}\right) f\left(\frac{a+b}{2}, c\left(\frac{a+b}{2}\right)\right) + h_y(b) f(b, c(b)) \right], \quad (80)$$

$$\begin{aligned} I_2 &= \int_a^b h_y(x) f(x, c(x) + h_y(x)) dx \\ &= \frac{h_x}{3} \left[h_y(a) f\left(a, c(a) + h_y(a)\right) + 4h_y\left(\frac{a+b}{2}\right) f\left(\frac{a+b}{2}, c\left(\frac{a+b}{2}\right) + h_y\left(\frac{a+b}{2}\right)\right) + h_y(b) f(b, c(b) + h_y(b)) \right], \end{aligned} \quad (81)$$

$$I_3 = \int_a^b h_y(x) f(x, d(x)) dx = \frac{h_x}{3} \left[h_y(a) f(a, d(a)) + 4h_y\left(\frac{a+b}{2}\right) f\left(\frac{a+b}{2}, d\left(\frac{a+b}{2}\right)\right) + h_y(b) f(b, d(b)) \right]. \quad (82)$$

Completely analogously to double integrals we can compute triple integrals or higher dimensions. However, when the dimensionality of the space is very high it becomes advantageous to employ Monte-Carlo methods to perform integration.

IV. MONTE CARLO INTEGRATION

Monte Carlo integration is based on a statistical interpretation of an integral. In particular when we have a probability distribution function $g(x)$ of a random variable (or set of variables) x , the average value of a quantity $f(x)$ that depends on the random variable x is given by

$$\langle f(x) \rangle = \frac{\int f(x)g(x)dx}{\int gdx} \quad (83)$$

If we assume a uniform distribution function $g(x) = c$, then the last expression becomes

$$\langle f(x) \rangle = \frac{\int f(x)dx}{\int dx} = \frac{\int f(x)dx}{V}, \quad (84)$$

where V is the “volume” of the integration domain. This last equation implies that the integral of a function in a given domain $\mathcal{V}$ is given by

$$I = \int_{\mathcal{V}} f(x)dx = V \langle f(x) \rangle \quad (85)$$

Hence, the integral I equal the product of the volume of the integration domain times the average of f assuming a uniformly distributed “random” variable x . Thus, in a statistical sense we have

$$I = \int_{\mathcal{V}} f(x)dx = V \sum_{i=1}^N \frac{f(x_i)}{N}, \quad N \rightarrow \infty \quad (86)$$

The central limit theorem guarantees that the error in computing the integral in this way goes down as $1/\sqrt{N}$ independently of the dimensionality of the integral. But, e.g., in Simpson’s method the error goes down as $h^4 \sim 1/n^4$, where n is the number of points in each direction of a multi-D integral. The total number of points however is $N = n^d$, where d is the dimensionality of the integral. Thus, error scales as $\sim 1/N^{4/d}$. If we want to determine the dimensionality at which the Monte-Carlo method becomes advantageous we simply equate the two errors, i.e.,

$$\frac{1}{N^{4/d}} \sim \frac{1}{N^{1/2}} \Rightarrow d \sim 8. \quad (87)$$

Thus, for $d \gtrsim 8$ the Monte-Carlo method becomes more efficient than standard numerical methods.

Algorithm for Monte-Carlo method: Goal to approximate $I = \int_a^b f(x)dx$

1. INPUT: $f(x)$, end points a, b and a large positive integer n .
OUTPUT: approximation to I .
2. Set $V = (b - a)$.
3. Set $xI = 0$ (sum over i)
4. For $i = 1, \dots, N$ do steps 5 and 6.
5. Generate a random number x in the interval $[a, b]$ and compute $f(x)$. In C++ (using the C++ 2011 standard) generating random numbers can be achieved as follows

```
#include <random>
#include <iostream>
#include <iomanip>
#include <cstdlib>

using namespace std;
```



```

int main()
{
    double a=0.;
    double b=1.;
    double lower_bound = a;
    double upper_bound = b;
    // Seed with a real random value, if available
    std::random_device rd;
    std::mt19937 gen(rd()); // seed
    std::uniform_real_distribution<double> unif(lower_bound,upper_bound); //uniform distribution between

    // Choose a random mean between a and b
    for (int i=0;i<100;i++){
        double a_random_double = unif(gen); //a_random_double is the random number you desire

        cout << setprecision(16) << a_random_double << endl;
    }

    return 0;
}

```

Note: To compile the code in the C++ 2011 standard you need to add `-std=c++11` in the compilation flag, i.e., `g++ -std=c++11`.

6. Set $xI = xI + f(x)$
7. Set $xI = V * xI / N$.
8. Return (xI)

Exercise 5: Use the Monte Carlo method to compute the integral $I = \int_{-1}^1 \cos(x) dx$. Run your code for $N = 10^2, 10^4, 10^6, 10^8$. Plot the relative error as a function of N . Does it scale roughly as $1/\sqrt{N}$?

We point out here, that there are smarter ways to sample a function, rather than the uniform sampling discussed here. By sampling the function where the contribution to the integral is larger leads to faster convergence of the numerical integration. However, we will not be discussing other sampling methods. Also, there exist better random number generators than the intrinsic C++ one.

MTH6150: Ordinary differential equations (ODEs)

Charalampos Markakis¹ & Vasileios Paschalidis²

¹ *School of Mathematical Sciences, Queen Mary University of London and*

² *Departments of Astronomy & Physics, University of Arizona, Tucson*

In this note we discuss methods for numerically solving ODEs.

I. ODES IN PHYSICS

Physics equations of motion require the solution of ordinary differential equations. While the vast majority of equations in Physics are partial differential equations, ordinary differential equations pop-up in scenarios where a high degree of symmetry and/or stationarity (time-independence) is imposed (e.g., stationary and spherically symmetric). For example, in Newtonian gravity the equation for the gravitational potential is

$$\nabla^2 \Phi = 4\pi G \rho, \quad (1)$$

but in spherical symmetry this becomes an ODE

$$\frac{1}{r^2} \frac{d}{dr} \left(r^2 \frac{d}{dr} \Phi \right) = 4\pi G \rho. \quad (2)$$

The Newtonian equation of motion is also a second-order ODE in time (acceleration=force/mass),

$$\frac{d^2 x}{dt^2} = F/m \quad (3)$$

For a very complex density distribution or complicated (time-dependent) forces, these last equations are not easily solved analytically. So, in such cases we need to solve the ODEs numerically. There are multiple ways one can numerically solve ODEs, and we will review here some of these methods including the Euler, and Runge-Kutta methods. We will also discuss the distinction between initial value and boundary value problems.

II. INITIAL VALUE PROBLEMS: THEORETICAL BACKGROUND

An initial value problem (IVP) for ODEs is described by the following equations

$$\frac{dy}{dt} = f(t, y), a \leq t \leq b, \text{ Initial conditions: } y(a) = \alpha \quad (4)$$

for a single ODE, and

$$\frac{d\vec{y}}{dt} = \vec{F}(t, \vec{y}), a \leq t \leq b, \text{ Initial conditions: } \vec{y}(a) = \vec{\alpha}, \quad (5)$$

where

$$\vec{y} = (y_1, y_2, \dots, y_n), \quad \vec{F} = (f_1(t, \vec{y}), f_2(t, \vec{y}) \dots f_n(t, \vec{y})). \quad (6)$$

for a system of coupled ODEs.

Before we decide to code any differential equation in a computer it is always important to be able to answer the following questions: 1) Does the ODE have a solution? 2) Is the ODE solution unique for a given initial conditions? 3) Does the solution depend continuously on the initial conditions? All three questions point to the concept of well-posedness of the IVP. If the answer is NO to any of these 3 questions, the IVP is not well-posed, and it is redundant to try to solve the ODE as an IVP, as the numerical algorithm will fail. In this first section we will introduce the mathematical background for answering the above questions.

DEFINITIONS

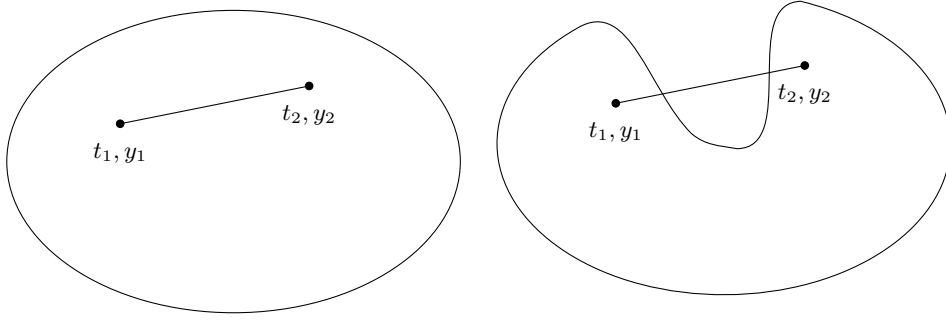


FIG. 1. Left: depiction of a convex set. Right: depiction of a non-convex set.

1. We say that $f(t, y)$ satisfies a Lipschitz condition (or is Lipschitz continuous) in the variable y on a set $D \subset \mathbb{R}^2$, if a constant $L > 0$ exists with $|f(t, y_1) - f(t, y_2)| \leq L|y_1 - y_2|$.

For example, $f(t, y) = ty$, $D = \{(t, y) | 1 \leq t \leq 2, -3 \leq y \leq 4\}$, then for $(t, y_1), (t, y_2)$ we have $|f(t, y_1) - f(t, y_2)| = |t||y_1 - y_2| \leq 2|y_1 - y_2|$. Thus, $L = 2$ and $f(t, y) = ty$ is Lipschitz continuous.

2. A set $D \subset \mathbb{R}$ is called convex, if for (t_1, y_1) and $(t_2, y_2) \in D$, then $[(1 - \lambda)t_1 + \lambda t_2, (1 - \lambda)y_1 + \lambda y_2] \in D \forall \lambda \in [0, 1]$

Theorem 1: If $f(t, y)$ is defined on a convex set $D \subset \mathbb{R}^2$, and there exists a positive $L > 0$ such that $\left| \frac{\partial f}{\partial y}(t, y) \right| \leq L$ $\forall (t, y) \in D$, then f is Lipschitz continuous on D , with Lipschitz constant L .

Theorem 2: If $D = \{(t, y) | a \leq t \leq b, -\infty < y < \infty\}$, and $f(t, y)$ is continuous on D , and $f(t, y)$ satisfies a Lipschitz condition on D in y , then the IVP

$$\frac{dy}{dt} = f(t, y), a \leq t \leq b, \text{ Initial conditions: } y(a) = \alpha \quad (7)$$

has a *unique* solution $y(t)$ for $a \leq t \leq b$.

For example, the IVP

$$\frac{dy}{dt} = f(t, y) = 1 + t \sin(ty), 0 \leq t \leq 2, \text{ Initial conditions: } y(a) = \alpha \quad (8)$$

satisfies

$$\frac{\partial f}{\partial y} = t^2 \cos(ty) \leq 4, \quad (9)$$

Thus, by theorems 1 and 2, the IVP with $f(t, y) = 1 + t \sin(ty)$ has a unique solution.

Well-posedness of the IVP

The IVP

$$\frac{dy}{dt} = f(t, y), a \leq t \leq b, \text{ Initial conditions: } y(a) = \alpha \quad (10)$$

is *well-posed* if

- A unique solution $y(t)$ exists
- There exist constants $\epsilon_0 > 0, k > 0$ such that $\forall \epsilon > 0$ with $\epsilon_0 > \epsilon > 0$, whenever $\delta(t)$ is continuous with $|\delta(t)| < \epsilon$ $\forall t \in [a, b]$, and $\delta_0 < \epsilon$, the IVP

$$\frac{dz}{dt} = f(t, z) + \delta(t), a \leq t \leq b, \text{ Initial conditions: } z(a) = \alpha + \delta_0 \quad (11)$$

has a unique solution $z(t)$ that satisfies

$$|z(t) - y(t)| < k\epsilon_0, \forall t \in [a, b]. \quad (12)$$

Theorem 3: Assume $D = \{(t, y) \mid a \leq t \leq b, -\infty < y < \infty\}$, where $f(t, y)$ is continuous and is Lipschitz continuous in y on D , then the IVP

$$\frac{dy}{dt} = f(t, y), a \leq t \leq b, \text{ Initial conditions: } y(a) = \alpha \quad (13)$$

is well-posed.

Well-posedness is a fundamental property of initial value problems for ordinary and partial differential equations. If a differential equation does not admit a well-posed initial value problem, then we cannot integrate it numerically. This is because, well-posedness is intimately related with the “stability” of the solution as stated in Eq. (12). This equation implies that the perturbed initial value problem must have a solution that is very close to the unperturbed one, i.e., the absolute difference between the solutions $z(t)$ and $y(t)$ is bounded. If a small perturbation either in initial conditions or in the right-hand-side of Eq. (11) leads the difference between the solution $z(t)$ and $y(t)$ to grow without bound, then this problem is ill-posed and cannot be integrated in a computer. The reason is that computers will always have finite error, i.e., will always apply small perturbations in the form of either truncation or round-off error. More specifically well-posedness implies continuous dependence on the initial data.

III. INITIAL VALUE PROBLEMS: NUMERICAL ALGORITHMS

We will now investigate numerical algorithms for solving IVPs for ODEs using first-order and higher-order accurate methods.

A. Euler’s Method

For an IVP

$$\frac{dy}{dt} = f(t, y), a \leq t \leq b, y(a) = \alpha \quad (14)$$

We seek a solution $y(t)$ at various values of $t \in [a, b]$ called the mesh points. The solution at other t can be obtained by interpolation. Consider the following mesh points

$$t_i = a + ih, i = 0, 1, 2, \dots, N, \quad (15)$$

where $h = (b - a)/N = t_{i+1} - t_i$ is called the step size. The simplest possible method for solving an IVP is the forward Euler method, which is derived from Taylor’s theorem

$$y(t_{i+1}) = y(t_i) + h \left. \frac{dy}{dt} \right|_{t_i} + \frac{h^2}{2} y''(\xi) \Rightarrow \quad (16)$$

$$y(t_{i+1}) = y(t_i) + hf(t_i, y(t_i)) + \frac{h^2}{2} y''(\xi) \quad (17)$$

Dropping the 2nd-order term, we have then the following algorithm

$$\begin{aligned} y_0 &= \alpha \\ y_{n+1} &= y_n + hf(t_n, y_n), n = 0, 1, 2, \dots, N - 1. \end{aligned} \quad (18)$$

The last equation is called a difference equation, and we denote $y_n = y(t_n)$.

Exercise 1: Consider the following IVP

$$\frac{dy}{dt} = -y, 0 \leq t \leq 3, y(0) = 1. \quad (19)$$

Is the problem well-posed? Use Euler’s method to solve the IVP for $N = 100, N = 500, N = 1000, N = 5000, N = 10000$ and compare $y(3)$ with the analytic (exact) solution. How quickly does the relative error decay as a function of h ?

B. Higher-order Taylor methods

Euler is only a first-order accurate method and is not used in practise. There are higher-order accurate methods that are preferred instead. Before we discuss standard methods we will first overview higher-order Taylor methods.

Consider the IVP

$$y' = f(t, y), a \leq t \leq b, y(a) = \alpha \quad (20)$$

where a prime ' implies differentiation with respect to t . Expand

$$y(t_{n+1}) = y(t_n) + hy'(t_n) + \frac{h^2}{2}y''(t_n) + \dots + \frac{h^k}{n!}y^{(k)}(t_n) + \frac{h^{k+1}}{(k+1)!}y^{(k+1)}(\xi_n), \xi_n \in (t_n, t_{n+1}). \quad (21)$$

But, from the ODE

$$y'(t) = f(t, y(t)), y''(t) = f'(t, y(t)), \dots, y^{(k)}(t) = f^{(k-1)}(t, y(t)) \quad (22)$$

By use of Eq. (22), Eq. (21) becomes

$$y(t_{n+1}) = y(t_n) + hf(t_n, y(t_n)) + \frac{h^2}{2}f'(t_n, y(t_n)) + \dots + \frac{h^k}{n!}f^{(k-1)}(t_n, y(t_n)) + \frac{h^{k+1}}{(k+1)!}f^{(k)}(\xi_n, y(\xi_n)). \quad (23)$$

Equation (23) motivates the algorithm of the Taylor method of order m , i.e.,

$$\begin{aligned} y_0 &= \alpha \\ y_{n+1} &= y_n + hT^{(m)}(t_n, y_n), \quad n = 0, 1, \dots, N-1, \end{aligned} \quad (24)$$

where

$$T^{(m)}(t_n, y_n) = f(t_n, y_n) + \frac{h}{2}f'(t_n, y_n) + \dots + \frac{h^{(m-1)}}{m!}f^{(m-1)}(t_n, y_n) \quad (25)$$

Euler is a Taylor method of order 1.

C. Runge-Kutta Methods

Runge-Kutta (RK) methods are widespread, efficient, and almost the standard. We will derive the second-order accurate RK2 method, and simply present the 3rd (RK3) and 4th-order accurate RK4 methods.

First, let us consider Taylor's theorem in 2 variables. Let $f(t, y)$ be C^{n+1} on $D = \{(t, y) \mid a \leq t \leq b, c \leq y \leq d\}$ and $(t_0, y_0) \in D$. Then $\forall (t, y) \in D$, there exists ξ between (t, t_0) and (y, y_0) such that

$$f(t, y) = P_n(t, y) + R_n(t, y), \quad (26)$$

where the Taylor polynomial of degree n is

$$\begin{aligned} P_n(t) &= f(t_0, y_0) + \left[(t - t_0) \frac{\partial f}{\partial t}(t_0, y_0) + (y - y_0) \frac{\partial f}{\partial y}(t_0, y_0) \right] \\ &+ \left[\frac{(t - t_0)^2}{2} \frac{\partial^2 f}{\partial t^2}(t_0, y_0) + (t - t_0)(y - y_0) \frac{\partial^2 f}{\partial t \partial y}(t_0, y_0) + \frac{(y - y_0)^2}{2} \frac{\partial^2 f}{\partial y^2}(t_0, y_0) \right] \\ &+ \dots \left[\frac{1}{n!} \sum_{j=0}^n \binom{n}{j} (t - t_0)^{n-j} (y - y_0)^j \frac{\partial^n f}{\partial t^{n-1} \partial y^j}(t_0, y_0) \right] \end{aligned} \quad (27)$$

and the remainder is

$$R_n(t, y) = \frac{1}{(n+1)!} \sum_{j=0}^{n+1} \binom{n+1}{j} (t - t_0)^{n+1-j} (y - y_0)^j \frac{\partial^{(n+1)} f}{\partial t^{n+1-j} \partial y^j}(\xi, \mu), \quad (28)$$

for some $\xi \in (t_0, t)$, and $\mu \in (y_0, y)$.

RK2

We wish to find a_1, α_1, β_1 such that $a_1 f(t + \alpha_1, y + \beta_1)$ approximates $T^{(2)}$, i.e.,

$$T^{(2)}(t, y) = f(t, y) + \frac{h}{2} f'(t, y) \quad (29)$$

with no error greater than $O(h^2)$. Since,

$$f'(t, y) = \frac{df}{dt} = \frac{\partial f}{\partial t} + \frac{\partial f}{\partial y} \frac{dy}{dt}, \text{ and } \frac{dy}{dt} = f, \quad (30)$$

we have

$$T^{(2)}(t, y) = f(t, y) + \frac{h}{2} \frac{\partial f}{\partial t} + \frac{h}{2} \frac{\partial f}{\partial y} \cdot f. \quad (31)$$

We also have from the Taylor expansion in 2 variables

$$a_1 f(t + \alpha_1, y + \beta_1) = a_1 f(t, y) + a_1 \alpha_1 \frac{\partial f}{\partial t} + a_1 \beta_1 \frac{\partial f}{\partial y} + a_1 \underbrace{R_1(t + \alpha_1, y + \beta_1)}_{\text{residual}} \quad (32)$$

where the residual is $O(h^2)$ if 2nd-order derivatives of f are bounded. If we match Eq. (32) (without its residual) to Eq. (31) we obtain the following set of algebraic equations

$$\begin{aligned} a_1 &= 1 \\ a_1 \alpha_1 &= \frac{h}{2} \\ a_1 \beta_1 &= \frac{h}{2} f(t, y) \end{aligned} \quad (33)$$

that is

$$a_1 = 1, \quad \alpha_1 = \frac{h}{2}, \quad \beta_1 = \frac{h}{2} f(t, y). \quad (34)$$

Thus,

$$T^{(2)}(t, y) = f\left(t + \frac{h}{2}, y + \frac{h}{2} f(t, y)\right) - R_1 \quad (35)$$

We have therefore arrived at the RK2 algorithm

$$\begin{aligned} y_0 &= \alpha \\ y_{n+1} &= y_n + h f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2} f(t_n, y_n)\right), \quad n = 0, 1, \dots, N-1, \end{aligned} \quad (36)$$

To avoid the nested functioning we can rewrite this as follows

$$\begin{aligned} y_0 &= \alpha \\ k_1 &= h f(t_n, y_n) \\ k_2 &= h f\left(t_n + \frac{h}{2}, y_n + \frac{1}{2} k_1\right) \\ y_{n+1} &= y_n + k_2, \quad n = 0, 1, \dots, N-1, \end{aligned} \quad (37)$$

RK3

The RK3 (Heun's) algorithm is the following

$$\begin{aligned}
 y_0 &= \alpha \\
 k_1 &= hf(t_n, y_n) \\
 k_2 &= hf(t_n + \frac{h}{3}, y_n + \frac{1}{3}k_1) \\
 k_3 &= hf(t_n + \frac{2h}{3}, y_n + \frac{2}{3}k_2) \\
 y_{n+1} &= y_n + \frac{1}{4}(k_1 + 3k_3)
 \end{aligned} \tag{38}$$

for each $n = 0, 1, \dots, N-1$. The method has local truncation error $O(h^3)$, as long as $y(t)$ is at least C^4 .

RK4

The RK4 algorithm is the following

$$\begin{aligned}
 y_0 &= \alpha \\
 k_1 &= hf(t_n, y_n) \\
 k_2 &= hf(t_n + \frac{h}{2}, y_n + \frac{1}{2}k_1) \\
 k_3 &= hf(t_n + \frac{h}{2}, y_n + \frac{1}{2}k_2) \\
 k_4 &= hf(t_{n+1}, y_n + k_3) \\
 y_{n+1} &= y_n + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)
 \end{aligned} \tag{39}$$

for each $n = 0, 1, \dots, N-1$. The method has local truncation error $O(h^4)$, as long as $y(t)$ is at least C^5 .

Algorithm for RK4:

Input: a, b, N , initial condition α , $f(t, y)$

- Step 1: Set $h = (b - a)/N$

$t = a$

$y = \alpha$

output(t, y)

- Step 2: For $n = 1, 2, \dots, N$ do Steps 3-5
- Step 3:

$$\begin{aligned}
 k_1 &= hf(t, y) \\
 k_2 &= hf(t + h/2, y + k_1/2) \\
 k_3 &= hf(t + h/2, y + k_2/2) \\
 k_4 &= hf(t + h, y + k_3)
 \end{aligned} \tag{40}$$

- Step 4:

$$y = y + (k_1 + 2k_2 + 2k_3 + k_4)/6 \tag{41}$$

$$t = a + n \cdot h \tag{42}$$

- Step 5:
output(t, y)

Exercise 2: Consider the following IVP

$$\frac{dy}{dt} = -2ty, 0 \leq t \leq 2, y(0) = 1. \quad (43)$$

Solve the ODE analytically. Follow the RK4 algorithm to implement instead the RK2 method and solve the IVP for $N = 100, N = 200, N = 400, N = 800, N = 1600$ and compare $y(3)$ with the analytic (exact) solution and the Euler solution. How quickly does the relative error decay as a function of h for RK2? Is RK2 more accurate than Euler? To make the code modular define a function for the right-hand-side of the equation, i.e., $f(t, y)$. Define a function RK2 that does the integration from t to $t + h$ and to which you will pass a function pointer that will point to $f(t, y)$. This way if you change $f(t, y)$, the code is invariant. The RK2 function will take as input t, h , the previous value of y , the pointer to the function and will return the new value of y , at $t + h$.

D. Higher order ODEs and systems of ODEs

Many ODEs in physics are of higher than first-order for which the methods we discussed above apply. For example, a simple harmonic oscillator satisfies

$$m \frac{d^2 x}{dt^2} = -kx \quad (44)$$

Any higher order ODE which can be solved for the highest order derivative can be written as a system of first order ODEs. For example, to obtain a first-order form for Eq. (44) we let $y = dx/dt$, and then the equation becomes the system of two first-order ODEs for the functions $y(t)$ and $x(t)$

$$\begin{aligned} \frac{dx}{dt} &= y \\ \frac{dy}{dt} &= -\frac{k}{m}x \end{aligned} \quad (45)$$

Thus, a large set of higher-order ODEs can be cast to first-order form as a system of first-order ODEs. This is why we will discuss here how we can solve numerical systems of coupled first-order ODEs. Thus, we will here concern ourselves primarily with generalizing our methods to systems of first order ODEs. The IVP for a general first-order system of m equations can be written as

$$\frac{d\vec{u}}{dt} = \vec{F}(t, \vec{u}), \quad a \leq t \leq b, \quad \vec{u}(t=a) = \vec{\alpha}, \quad (46)$$

where the column vector of unknown functions $\vec{u}(t) = (u_1(t), u_2(t), u_3(t), \dots, u_m(t))$, the column vector of “sources” $\vec{F}(t, \vec{u}) = (f_1(t, u_1, u_2, \dots, u_m), f_2(t, u_1, u_2, \dots, u_m), \dots, f_m(t, u_1, u_2, \dots, u_m))$, and the m initial values $\vec{\alpha} = (\alpha_1, \alpha_2, \dots, \alpha_m)$. The goal is to find $\vec{u}(t)$. First, we need to generalize our definitions for *Lipschitz condition* and our theorems for uniqueness of solutions.

Definition: For a function $f(t, u_1, \dots, u_m)$ defined on $D = \{(t, u_1, \dots, u_m) \mid a \leq t \leq b, -\infty < u_i < \infty, i = 1, 2, \dots, m\}$, we say that f satisfies a Lipschitz condition on D in the variables $u_1, \dots, u_m$, if there exists a constant $L > 0$ such that

$$|f(t, u_1, \dots, u_m) - f(t, z_1, \dots, z_m)| \leq \sum_{i=1}^m |u_i - z_i|, \quad \text{for all } (t, u_1, \dots, u_m) \text{ and } (t, z_1, \dots, z_m) \text{ in } D. \quad (47)$$

By virtue of the mean value theorem it can be shown that if f and its first partial derivatives are continuous in D , and if

$$\left| \frac{\partial f(t, u_1, u_2, \dots, u_m)}{\partial (u_1, u_2, \dots, u_m)} \right| \leq L, \quad \text{for } i = 1, 2, \dots, m \text{ and all } (t, u_i) \in D, \quad (48)$$

then f is Lipschitz continuous on D .

Theorem:

Let $f_i(t, u_1, \dots, u_m)$, $i = 1, \dots, m$ be defined on $D = \{(t, u_1, \dots, u_m) \mid a \leq t \leq b, -\infty < u_i < \infty, i = 1, 2, \dots, m\}$. If all $f_i(t, u_1, \dots, u_m)$ are Lipschitz continuous on D , then the IVP of Eq. (46) has a unique solution $u_1(t), u_2(t), \dots, u_m(t)$ for $a \leq t \leq b$.

1. RK4 for systems of ODEs

Here we will simply generalize RK4 for systems of ODEs of the type of Eq. (46). Let $N > 0$, such that $h = (b-a)/N$ is the integration step size. As usually we will define the mesh points

$$t_n = a + n * h, \quad n = 0, 1, 2, \dots, N. \quad (49)$$

We will use the notation $u_{i,n}$ to designate the function i at time t_n , i.e., $u_{i,n} = u_i(t_n)$. We have the initial conditions

$$u_{1,0} = \alpha_1, u_{2,0} = \alpha_2, \dots, u_{m,0} = \alpha_m, \quad (50)$$

The RK4 algorithm then becomes as follows.

Algorithm for RK4:

Input: a, b, N , initial condition α , $f(t, y)$

- Step 1: Set $h = (b - a)/N$

$$t = a$$

$$u_1 = \alpha_1, u_2 = \alpha_2, \dots, u_m = \alpha_m,$$

$$\text{output}(t, u_i)$$

- Step 2: For $n = 1, 2, \dots, N - 1$ do Steps 3-4

- Step 3:

For $i = 1, 2, \dots, m$

$$k_{1,i} = h f_i(t_n, u_1, \dots, u_m)$$

For $i = 1, 2, \dots, m$

$$k_{2,i} = h f_i\left(t + \frac{h}{2}, u_1 + \frac{k_{1,1}}{2}, u_2 + \frac{k_{1,2}}{2}, \dots, u_m + \frac{k_{1,m}}{2}\right)$$

For $i = 1, 2, \dots, m$

$$k_{3,i} = h f_i\left(t + \frac{h}{2}, u_1 + \frac{k_{2,1}}{2}, u_2 + \frac{k_{2,2}}{2}, \dots, u_m + \frac{k_{2,m}}{2}\right)$$

For $i = 1, 2, \dots, m$

$$k_{4,i} = h f_i(t + h, u_1 + k_{3,1}, u_2 + k_{3,2}, \dots, u_m + k_{3,m})$$

For $i = 1, 2, \dots, m$

$$u_i = u_i + (k_{1,i} + 2k_{2,i} + 2k_{3,i} + k_{4,i})/6$$

- Step 4:

$$t = a + n \cdot h \quad (52)$$

- Step 5:

$$\text{output}(t, \vec{u})$$

Exercise 3: Generalize the RK3 algorithm to a system of m first-order ODEs.

Exercise 4: Non-dimensionalize the ODE of the simple harmonic oscillator, and cast it to first-order form.

$$m\ddot{x} = -k(x - x_0). \quad (53)$$

To achieve a general solution (independent of m and k) we want to non-dimensionalize the equation first. To achieve this let us perform a dimensional analysis. We will denote the unit of time by $[T]$, and the unit of length $[L]$. Then above equation becomes

$$m \frac{[L]}{[T]^2} = -k[L] \Rightarrow m \frac{1}{[T]^2} = -k \Rightarrow \sqrt{\frac{m}{k}} = [T], \quad (54)$$

in other words, $\sqrt{\frac{m}{k}}$ defines a timescale in our problem. So, we will introduce a new dimensionless time $\hat{t} = t/\sqrt{\frac{m}{k}}$. Then, Eq. (53) becomes

$$\frac{d^2 x}{d\hat{t}^2} = -x. \quad (55)$$

We can further introduce the length of the spring at the relaxed position x_0 as a lengthscale in the problem to non-dimensionalize x , i.e., $\hat{x} = x/x_0$. The fully non-dimensional ODE then becomes

$$\frac{d^2 \hat{x}}{d\hat{t}^2} = -(\hat{x} - 1). \quad (56)$$

The benefit of this form is that all we need is to solve this problem once, and then by choosing x_0 and $\sqrt{\frac{m}{k}}$ we can rescale our time and length to solve the problem for any value of x_0 and $\sqrt{\frac{m}{k}}$ (for our choice of initial conditions). Since we can rename our variables, we can remove the hats to obtain the final non-dimensional form of the harmonic oscillator

$$\frac{d^2 x}{dt^2} = -(x - 1). \quad (57)$$

E. Stability of numerical schemes for ODE IVP

We have so far seen that round-off error instabilities can arise that destroy the stability of a numerical integration scheme. Numerical schemes for ODEs can also be unstable, and they have to be analyzed for each ODE separately. Let us consider for example the simple IVP

$$\frac{du}{dt} = -\lambda u, \quad u(0) = u_0, \quad t > 0. \quad (58)$$

and let us consider the forward Euler method as the solution scheme, i.e.

$$u_{n+1} = u_n - \lambda u_n h = (1 - \lambda h) u_n \quad (59)$$

Let us now consider that we have an exact solution of the algebraic Eqs. (59), u_n , and that we perturbed it $u_n \rightarrow u_n + \delta u_n$ (essentially the perturbation is seeded by round off error), then the above equation becomes

$$\delta u_{n+1} = (1 - \lambda h) \delta u_n \quad (60)$$

which we can rewrite as

$$\delta u_{n+1} = (1 - \lambda h)^n \delta u_0 \quad (61)$$

from which we can write

$$\left| \frac{\delta u_{n+1}}{\delta u_0} \right| = |1 - \lambda h|^n \quad (62)$$

Thus, the term $|1 - \lambda h|$ is an amplification factor. If $|1 - \lambda h| > 1$, then the magnitude of the perturbation grows in time, if $|1 - \lambda h| \leq 1$, the perturbation does not grow. Thus, we have stability if

$$|1 - \lambda h| \leq 1 \Leftrightarrow -1 \leq 1 - \lambda h \leq 1 \quad (63)$$

The right side of the inequality is always satisfied for $\lambda > 0$ (h is defined to be > 0), but the left side of the inequality is satisfied only if $h \leq 2/\lambda$! Thus, if the step size is greater than $2/\lambda$, the forward Euler method is numerically unstable!

Let us now consider the RK2 algorithm for the same ODE (58), for which we have $f(t, u) = -\lambda u$. The RK2 algorithm is

$$u_{n+1} = u_n + hf \left(t_n + \frac{h}{2}, u_n + \frac{h}{2} f(t_n, u_n) \right) \quad (64)$$

Using $f(t, u) = -\lambda u$, we can rewrite this as

$$u_{n+1} = u_n + hf\left(t_n + \frac{h}{2}, u_n - \lambda \frac{h}{2} u_n\right) \quad (65)$$

and

$$u_{n+1} = u_n - \lambda h(u_n - \lambda \frac{h}{2} u_n) = u_n \left(1 - \lambda h(1 - \lambda \frac{h}{2})\right) \quad (66)$$

or

$$u_{n+1} = u_0 \left(1 - \lambda h + \frac{(\lambda h)^2}{2}\right)^n \quad (67)$$

and again we obtain

$$\left|\frac{u_{n+1}}{u_0}\right| = \left|1 - \lambda h + \frac{(\lambda h)^2}{2}\right|^n \quad (68)$$

with the amplification factor this time being $\left|1 - \lambda h + \frac{(\lambda h)^2}{2}\right|$, which for stability must satisfy

$$\left|1 - \lambda h + \frac{(\lambda h)^2}{2}\right| \leq 1. \quad (69)$$

or

$$-1 \leq 1 - \lambda h + \frac{(\lambda h)^2}{2} \leq 1. \quad (70)$$

The right inequality becomes

$$\lambda h\left(\frac{\lambda h}{2} - 1\right) \leq 0 \Rightarrow \frac{\lambda h}{2} \leq 1 \Rightarrow h \leq \frac{2}{\lambda}, \quad (71)$$

which is the same restriction on the step size as in the Euler method. The left inequality in (70) requires

$$0 \leq 2 - \lambda h + \frac{(\lambda h)^2}{2} \quad (72)$$

If we set $x = \lambda h$, the right-hand-side in the last equation becomes $\frac{1}{2}x^2 - x + 2$, whose discriminant is $\Delta = 1 - 4\frac{1}{2}2 = -3 < 0$. Thus, $\frac{1}{2}x^2 - x + 2 > 0$. Hence, inequality (72) is always satisfied. Thus, RK2 has the same restriction on the timestep as the Euler method, i.e.,

$$h \leq \frac{2}{\lambda}. \quad (73)$$

Thus, we cannot choose an arbitrarily large h , because the numerical scheme will become unstable!

As a rule of thumb, when solving IVP usually there is an associated time (or length) scale with the problem, that we must resolve, i.e., the step must be smaller than this scale. The ODE (58), has as characteristic time scale $1/\lambda$. Thus, to be numerically stable (and to have an accurate solution), we must resolve this time scale, i.e.,

$$h < 1/\lambda, \quad (74)$$

which is a restriction very close to the one we obtained with a much more elaborate perturbation analysis. So, always try to think about the problem you are trying to solve. Determine the characteristic scales, and these will determine what step size you should use. Thinking, can save us a lot of time from trying to figure out why our codes may fail/crash.

Exercise 5: Let us now consider a system of ODEs

$$\begin{aligned} \frac{dx}{dt} &= -\frac{2}{3}(x + y) \\ \frac{dy}{dt} &= \frac{1}{3}(2x - 7y) \end{aligned} \quad (75)$$

What is the numerical stability condition when employing the forward Euler method?

Solution: The forward Euler method becomes

$$\begin{aligned}x_{n+1} &= x_n - \frac{2}{3}(x_n + y_n)h \\ y_{n+1} &= y_n + \frac{1}{3}(2x_n - 7y_n)h\end{aligned}\tag{76}$$

Let consider Harmonic perturbations to our variables, i.e.

$$x_n = x_0 \exp(inh), \quad y_n = y_0 \exp(inh),\tag{77}$$

where i is the imaginary unit. If we plug Eq. (77) in Eq. (78) we obtain

$$\begin{aligned}x_0 \exp(ih) &= x_0 - \frac{2}{3}(x_0 + y_0)h \\ y_0 \exp(ih) &= y_0 + \frac{1}{3}(2x_0 - 7y_0)h\end{aligned}\tag{78}$$

This is the following eigenvalue problem

$$\begin{bmatrix} 1 - 2h/3 & -2h/3 \\ 2h/3 & 1 - 7h/3 \end{bmatrix} \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} = \lambda \begin{bmatrix} x_0 \\ y_0 \end{bmatrix}\tag{79}$$

where $\lambda = \exp(ih)$ is the eigenvalue. For the system to be numerically stable the eigen values of the matrix

$$M = \begin{bmatrix} 1 - 2h/3 & -2h/3 \\ 2h/3 & 1 - 7h/3 \end{bmatrix}\tag{80}$$

must be smaller than unity. To find the eigenvalue of M we need to solve the characteristic polynomial

$$\det[M - \lambda I] = 0 \Rightarrow (1 - 2h/3 - \lambda)(1 - 7h/3 - \lambda) + 4h^2/9 = 0.\tag{81}$$

This equation has the following solutions $\lambda_1 = 1 - h$, $\lambda_2 = 1 - 2h$. For the numerical integration system to be stable the **absolute value** of both eigenvalues must be smaller than unity, i.e., we obtain the following stability requirements $|1 - h| < 1$, and $|1 - 2h| < 1$, which yield $-1 < 1 - h < 1$ and $-1 < 1 - 2h < 1$, and $h < 2$ and $h < 1/2$, respectively.

Exercise 6: Implement the RK4 algorithm for the following system of ODEs

$$\begin{aligned}\frac{dx}{dt} &= -\frac{2}{3}(x + y) \\ \frac{dy}{dt} &= \frac{1}{3}(2x - 7y)\end{aligned}\tag{82}$$

This system is exactly solvable. The help you compare your algorithm the analytic solution of the system is the following

$$\begin{aligned}x &= \frac{2}{3}e^{-t}(2x_0 - y_0) - \frac{1}{3}e^{-2t}(x_0 - 2y_0) \\ y &= \frac{1}{3}e^{-t}(2x_0 - y_0) - \frac{2}{3}e^{-2t}(x_0 - 2y_0)\end{aligned}\tag{83}$$

where $x_0 = x(0)$ and $y_0 = y(0)$ are the initial conditions.

IV. BOUNDARY VALUE PROBLEMS AND STIFF ODES

In many occasions in physics we need to specify not only an initial condition for the ODE, but also a “final” condition. These types of problems are called *boundary value problems*. A first-order ODE cannot have a boundary value problem because all first-order ODEs require is the value of the variable at one point, e.g., the value at one “boundary”. Thus, second or higher order ODEs are the only ones that can have boundary value problems. For example, consider the electrostatic potential between two metal spheres of different radii. We can ask what is the potential between the two if the potential of the inner sphere is kept at potential V_{inner} and the outer one sphere is

kept at potential 0. The potential between the sphere is governed by Laplace's equation, which in spherical symmetry reduces to

$$\frac{d^2V}{dr^2} + \frac{2}{r} \frac{dV}{dr} = 0, \quad V(R_{\text{inner}}) = V_{\text{inner}}, V(R_{\text{outer}}) = 0. \quad (84)$$

The reason why higher order systems can be cast as boundary value problems is that a higher order system requires more than one initial condition to determine the solution. In the example above, the second order equation requires that we know the value of V and the value of dV/dr at a point to solve it completely. Thus, it requires two conditions. Instead of providing the derivative, one can provide the value of the function at another point, making it a boundary value problem.

The above discussion should have already prepared the ground for one of the most common methods for solving boundary value problems – *the shooting method*. In the shooting method one casts the boundary value problem as an initial value problem where one would set $V(R_{\text{inner}}) = V_{\text{inner}}$, and then set the derivative to some value solve the initial value problem, and if $V(R_{\text{outer}}) \neq 0$, go back change the value of the derivative and iterate until $V(R_{\text{outer}}) = 0$, to within a certain tolerance. This can be done for linear and non-linear equations, but we will not expand further on this topic here.

In addition to boundary value problems there exist *stiff* ordinary differential equations, which typically have part of their solution a transient solution (signal) changing on a rapid time scale, and a more “steady-state” part which changes on a slower time scale. These problems require special numerical schemes to be solved. We only mention stiff systems here, in case you ever run into a problem that has such behavior, and you need to resort to these special stiff solvers.

MTH6150: Partial differential equations (PDEs)

Charalampos Markakis¹ & Vasileios Paschalidis²

¹ *School of Mathematical Sciences, Queen Mary University of London and*

² *Departments of Astronomy & Physics, University of Arizona, Tucson*

In this note we discuss methods for numerically solving PDEs.

I. PDES IN PHYSICS

The vast majority of equations describing physical theories are partial differential equations, For example, in Newtonian gravity the equation for the gravitational potential is

$$\nabla^2 \Phi = 4\pi G \rho, \quad (1)$$

The propagation of a disturbance on a string, or electromagnetic perturbations satisfy a wave equation

$$\frac{\partial u}{\partial t} = c^2 \nabla^2 u, \quad (2)$$

where c is the speed of propagation of information.

Inhomogeneities in temperature T in a medium obey the heat equation

$$\frac{\partial T}{\partial t} = \kappa \nabla^2 T, \quad (3)$$

where $\kappa > 0$ is the diffusivity. Additional examples involve the Euler equations of (magneto)hydrodynamics, Einstein's theory of general relativity, and many more.

II. THEORETICAL BACKGROUND

Any course on numerically solving PDEs must start with the classification of the equations.

A. Types of PDEs based on second-order equations in two dimensions

The classification of PDEs is extremely important, because it dictates the numerical methods that one can adopt to solve the equations. We will start our classification of PDEs by considering the most general second-order PDE in two variables and constant coefficients

$$A \frac{\partial^2 u}{\partial x^2} + B \frac{\partial^2 u}{\partial x \partial y} + C \frac{\partial^2 u}{\partial y^2} + D \frac{\partial u}{\partial x} + E \frac{\partial u}{\partial y} + F = 0. \quad (4)$$

This equation is classified based on the discriminant $\Delta = B^2 - 4AC$.

- If $\Delta > 0$, the equation is of the *hyperbolic* type.
- If $\Delta = 0$, the equation is of the *parabolic* type.
- If $\Delta < 0$, the equation is of the *elliptic* type.

Thus, the classification of PDEs is based solely on the highest derivative operators.

We will later give general definitions of hyperbolic, parabolic and elliptic PDEs in any number of dimensions. For now let us consider a few examples.

In Cartesian coordinates Poisson's equation in 2D is written as

$$\frac{\partial^2 \Phi}{\partial x^2} + \frac{\partial^2 \Phi}{\partial y^2} = 4\pi G \rho. \quad (5)$$

This matches Eq. (4) for $A = 1$, $B = 0$, $C = 1$, $D = E = 0$, $F = -4\pi G\rho$. This implies that $\Delta = B^2 - 4AC = -4 < 0$. Thus, Poisson's equation is the archetypical example of an elliptic equation.

In Cartesian coordinates the wave equation in one spatial dimension is written as

$$\frac{\partial^2 u}{\partial t^2} - c^2 \frac{\partial^2 u}{\partial x^2} = 0 \quad (6)$$

If we identify t with y , this matches Eq. (4) for $A = -c^2$, $B = 0$, $C = 1$. This implies that $\Delta = B^2 - 4AC = 4c^2 > 0$. Thus, the wave equation is the archetypical example of a hyperbolic equation.

In Cartesian coordinates the heat equation in one spatial dimension is written as

$$\frac{\partial T}{\partial t} - \kappa \frac{\partial^2 T}{\partial x^2} = 0 \quad (7)$$

If we identify t with y , this matches Eq. (4) for $A = \kappa > 0$, $B = 0$, $C = 0$, $D = 0$, $E = 1$, $F = 0$. This implies that $\Delta = B^2 - 4AC = 0$. Thus, the heat equation is the archetypical example of a parabolic equation. Note that although the highest derivative operator determines the classification of the equation, it is the fact that $E \neq 0$ that makes the equation parabolic. If the only non-zero coefficient was A then the equation would be an ordinary differential equation.

One of the most fundamental physical differences between hyperbolic, parabolic, and elliptic PDEs is that among them only hyperbolic equations have finite speed of propagation of information, which is given by the wave speed c .

Elliptic equations have infinite speed of propagation of information, this can be handwaivingly be understood by writing the wave equation as

$$\nabla^2 u = \frac{1}{c^2} \frac{\partial^2 u}{\partial t^2}, \quad (8)$$

which for $c \rightarrow \infty$ becomes $\nabla^2 u = 0$, which is an elliptic equation.

Parabolic PDEs also have infinite speed of propagation of information, but also affect different scales (wavelengths) in different ways. We will later describe in more detail these properties. The fact that only hyperbolic equations have finite speed of propagation of information, implies that among the three types, only hyperbolic equations can be causal, and respect the fact that the maximum speed of propagation of information is the speed of light in vacuum.

Apart from these physical differences the different PDE types require different numerical methods for solving them. Hyperbolic and parabolic PDEs can be solved as initial value problems, but elliptic PDEs can be solved only as boundary value problems.

It is important to note that in general PDE does not have to be of a single type. In particular, a PDE or system of PDEs can change type. A typical example is the following PDE

$$\frac{\partial^2 u}{\partial t^2} = -t \nabla^2 u \quad (9)$$

This equation is hyperbolic for $t < 0$, but becomes elliptic for $t > 0$ and degenerate parabolic for $t = 0$. In addition, a *system* of PDEs can be of mixed type: we can have mixed hyperbolic-elliptic and mixed hyperbolic-parabolic (e.g. the Navier-Stokes equations). In fact, one can possibly imagine mixed hyperbolic-parabolic-elliptic systems of PDEs.

Finally, there can be PDEs that cannot be characterized using the above classification scheme.

B. PDE types and systems of PDEs in any dimensions

The world around us is 3-dimensional and equations in physical theories can also be in four or higher numbers of dimensions. Thus, to understand what numerical methods to employ for solving complex high-order PDEs or systems of PDEs we would like to be able to classify PDEs in any number of dimensions and in systems. Since, most PDEs in Physics are of second-order let us start with the classification of second-order PDEs in any number of dimensions.

1. Classification of second-order PDEs in d dimensions

A general second-order PDE with constant coefficients in d dimensions can be written as follows

$$\sum_{i=1}^d \sum_{j=1}^d A_{ij} \frac{\partial^2 u}{\partial x^i \partial x^j} + \text{lower order terms} = 0. \quad (10)$$

The PDE type is entirely determined by the matrix A_{ij} .

- If the matrix is positive (or negative) definite, i.e.,

$$\sum_{i=1}^d \sum_{j=1}^d A_{ij} k^i k^j > 0, \quad (11)$$

for any d -dimensional vector $\vec{k}$, then the PDE is *elliptic*. Since, a matrix is positive (or negative) definite when all its eigenvalues are positive (or negative), then the PDE is elliptic if **all** eigenvalues of A_{ij} are positive (or **all** are negative).

- If all, **but one** of the eigenvalues of A_{ij} are positive (or negative), and one eigenvalue is 0, then the PDE is *parabolic*.
- If A_{ij} has **one** negative (or positive) eigenvalue, and all other eigenvalues are positive (or negative), then the PDE is hyperbolic.
- If A_{ij} has more than one negative and more than one positive eigenvalue, and there are no zero eigenvalues, then the PDE is *ultrahyperbolic*.

Before we continue to systems of PDEs, let us consider a few examples. In Cartesian coordinates Poisson's equation in 3D is written as

$$\frac{\partial^2 \Phi}{\partial x^2} + \frac{\partial^2 \Phi}{\partial y^2} + \frac{\partial^2 \Phi}{\partial z^2} = 4\pi G\rho. \quad (12)$$

The matrix A_{ij} for this PDE is $A = \text{diag}(1, 1, 1)$, the Eigenvalues are all therefore $\lambda_i = 1 > 0$, $i = 1, 2, 3$, and hence the equation is elliptic. This is true for any number of dimensions.

In Cartesian coordinates the wave equation in 3 spatial dimensions is written as

$$-\frac{\partial^2 u}{\partial t^2} + c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right) = 0 \quad (13)$$

The matrix A_{ij} for this PDE is $A = \text{diag}(-1, c^2, c^2, c^2)$, the Eigenvalues are $\lambda_1 = -1 < 0$ and $\lambda_i = c^2 > 0$, $i = 2, 3, 4$. There is only **one** negative eigenvalue and all others are positive, hence the equation is hyperbolic. This is true for any number of *spatial* dimensions.

In Cartesian coordinates the heat equation in 3 spatial dimensions is written as

$$\frac{\partial T}{\partial t} - \kappa \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} + \frac{\partial^2 T}{\partial z^2} \right) = 0. \quad (14)$$

Since we have a first-order partial derivative with respect to t , but no second-order derivative with respect to t , the matrix A_{ij} for this PDE is $A = \text{diag}(0, \kappa, \kappa, \kappa)$, the Eigenvalues are therefore $\lambda_1 = 0$ and $\lambda_i = \kappa > 0$, $i = 2, 3, 4$. There is only **one** zero eigenvalue, and all others are positive, hence the equation is parabolic. This is true for any number of *spatial* dimensions.

An example of an ultrahyperbolic equation would be

$$-\frac{\partial^2 u}{\partial t^2} - \frac{\partial^2 u}{\partial v^2} + c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right) = 0 \quad (15)$$

where $A = \text{diag}(-1, -1, c^2, c^2, c^2)$, whose eigenvalues are $\lambda_1 = -1$, $\lambda_2 = -1$, and $\lambda_i = c^2 > 0$, $i = 3, 4, 5$, i.e., we have two negative and 3 positive eigenvalues. Ultrahyperbolic equations rarely arise in Physics. So, we will not be considering them further in this class.

2. Classification of first-order PDEs in d dimensions

Higher-order systems of PDEs can often be cast to completely first-order form.

This is the case for elliptic and hyperbolic PDEs, but parabolic PDEs cannot be cast to pure first-order form. Let us consider for example the wave equation in two spatial dimensions

$$\frac{\partial^2 u}{\partial t^2} = c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) \quad (16)$$

To cast it to first-order form first introduce a new variable $K = \frac{\partial u}{\partial t}$, then the wave equation becomes

$$\begin{aligned}\frac{\partial u}{\partial t} &= K, \\ \frac{\partial K}{\partial t} &= c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)\end{aligned}\tag{17}$$

But, we to cast this to complete first order form we still have reduce the second-order derivatives to first-order derivative. This can be achieved by introducing two new variables $D_x = \frac{\partial u}{\partial x}$ and $D_y = \frac{\partial u}{\partial y}$, then the system becomes

$$\begin{aligned}\frac{\partial u}{\partial t} &= K, \\ \frac{\partial K}{\partial t} &= c^2 \left(\frac{\partial D_x}{\partial x} + \frac{\partial D_y}{\partial y} \right)\end{aligned}\tag{18}$$

But, now the system has 4 variables u, K, D_x, D_y and 2 equations. Thus, it is underdetermined. To close the system we must promote the variables D_x, D_y to dynamical (evolved variables). This can be achieved by taking a time derivative using their definition and commuting the time and spatial derivative. For example, from $D_x = \frac{\partial u}{\partial x}$ we have

$$\frac{\partial D_x}{\partial t} = \frac{\partial}{\partial t} \frac{\partial u}{\partial x} = \frac{\partial}{\partial x} \frac{\partial u}{\partial t} \xrightarrow{\frac{\partial u}{\partial t} = K} \frac{\partial D_x}{\partial t} = \frac{\partial K}{\partial x}.$$

Similarly it can be shown that

$$\frac{\partial D_y}{\partial t} = \frac{\partial K}{\partial y}.\tag{20}$$

Thus, the complete reduction of the wave equation to first-order form is

$$\begin{aligned}\frac{\partial u}{\partial t} &= K, \\ \frac{\partial K}{\partial t} &= c^2 \left(\frac{\partial D_x}{\partial x} + \frac{\partial D_y}{\partial y} \right), \\ \frac{\partial D_x}{\partial t} &= \frac{\partial K}{\partial x}, \\ \frac{\partial D_y}{\partial t} &= \frac{\partial K}{\partial y},\end{aligned}\tag{21}$$

which has 4 equations for the 4 unknown variables, but is subject to two constraints

$$D_x - \frac{\partial u}{\partial x} = 0, \quad D_y - \frac{\partial u}{\partial y} = 0.\tag{22}$$

The system (21) has 4 degrees of freedom and as such the totality of solutions of this system is larger than the totality of solutions of the original wave equation (which has 2 degrees of freedom). The two systems are equivalent, if and only if the constraints (22) are satisfied.

Given that we can reduce higher-order PDEs to first order, first we will discuss ellipticity and hyperbolicity of first-order PDEs, and we will consider general parabolic PDEs subsequently.

A general system of m first-order PDE in d dimensions for m unknown dependent variables u can be written as follows (not all systems of PDEs can be written in this form, but in general equations in Physics can).

$$\frac{\partial \vec{u}}{\partial t} = \sum_{k=1}^d A^k \frac{\partial u}{\partial x^k} + \vec{B},\tag{23}$$

where $\vec{u} = (u_1, u_2, \dots, u_m)$ are the m unknown variables, and the d in number $m \times m$ matrices A_{ij}^k , and the vector $\vec{B}$ characterize the linear/non-linear nature of the system. In particular we have:

- If A_{ij}^k and the vector $\vec{B}$ depend at most on $\vec{u}$ and the coordinates (or independent variables) $\vec{x}$, the system is called *quasilinear*.

- If A_{ij}^k were to depend on derivatives of $\vec{u}$, then the system would be classified as *genuinely nonlinear*.
- If A_{ij}^k and the vector $\vec{B}$ depend only on the coordinates $\vec{x}$ or are constant, then the system is called linear and the superposition principle holds.

Equations in physics tend to be of the quasilinear type. Regarding the classification of the PDEs as elliptic, hyperbolic, parabolic as in the case of general second-order PDEs, these are determined by the highest derivative operator appearing. In the context of PDEs that can be written in the form

$$\frac{\partial \vec{u}}{\partial t} = L\vec{u} + \vec{B}, \quad (24)$$

where L is some differential operator, the highest-order derivative part of the equations is called the *principal part*. The principal part determines the character of the equations and the fundamental physics described by the equations. In the context of Eq. (23), the operator L equals

$$L = \sum_{k=1}^d A^k \frac{\partial}{\partial x^k} \quad (25)$$

and the principal part of the equations is $\sum_{k=1}^d A^k \frac{\partial u}{\partial x^k}$, because the highest order derivatives that appear are first-order. We will later see that this is not the case for parabolic PDEs. The matrices appearing in the principal part of first-order PDEs can be used to find the *characteristic matrix* of the system which is defined as

$$M = \sum_{i=1}^d A^i \hat{k}_i, \quad (26)$$

where $\hat{k}$ is a **unit** vector in some norm. In Euclidean space the norm of $\hat{k}$ is $||k|| = \sqrt{\sum_{i=1}^d k_i^2}$. The eigen-structure of the characteristic matrix M determines the classification of the PDE (23). We have the following cases:

- If there are any eigenvalues of the characteristic matrix M that are purely imaginary, the system of PDEs is elliptic.
- If all eigenvalues of the characteristic matrix M are purely real, the system of PDEs is hyperbolic. For hyperbolic systems we have additional classifications
 1. If a system of PDEs is hyperbolic, and the principal matrix does not have a complete set of eigenvectors, the system is called *weakly hyperbolic*
 2. If a system of PDEs is hyperbolic, and the principal matrix has a complete set of eigenvectors, the system is called *strongly hyperbolic*
 3. If the principal matrix of a hyperbolic system of PDEs has all of its eigenvalues distinct, then the system is called *strictly hyperbolic*. A matrix which has distinct eigenvalues also has a complete set of eigenvectors, and hence strictly hyperbolic systems are strongly hyperbolic.
 4. If the principal matrix of the system of PDEs is hermitian, i.e., $M^\dagger = M$, then the system is called *symmetric hyperbolic*. Hermitian matrices have real eigenvalues and a complete set of eigenvectors. Thus, symmetric hyperbolic systems are strongly hyperbolic.

In general the existence of complex eigenvalues for the characteristic matrix M implies at least partial elliptic behavior.

Let us now apply the above to the Poisson equation in 2D in Cartesian coordinates.

$$\frac{\partial^2 \Phi}{\partial x^2} + \frac{\partial^2 \Phi}{\partial y^2} = 4\pi G\rho. \quad (27)$$

First, introduce new variables $D_x = \frac{\partial \Phi}{\partial x}$, $D_y = \frac{\partial \Phi}{\partial y}$, then the above equation becomes

$$\begin{aligned} \frac{\partial \Phi}{\partial x} &= D_x \\ \frac{\partial D_x}{\partial x} &= -\frac{\partial D_y}{\partial y} + 4\pi G\rho \end{aligned} \quad (28)$$

and we need an equation for how D_y varies along x , which we obtain from $D_y = \frac{\partial \Phi}{\partial y}$ as follows

$$\frac{\partial D_y}{\partial x} = \frac{\partial}{\partial x} \frac{\partial \Phi}{\partial y} = \frac{\partial}{\partial y} \frac{\partial \Phi}{\partial x} = \frac{\partial D_x}{\partial y}. \quad (29)$$

Using this last equation the original Poisson equation can be cast to complete first-order form as

$$\begin{aligned} \frac{\partial \Phi}{\partial x} &= D_x, \\ \frac{\partial D_x}{\partial x} &= -\frac{\partial D_y}{\partial y} + 4\pi G\rho, \\ \frac{\partial D_y}{\partial x} &= \frac{\partial D_x}{\partial y}. \end{aligned} \quad (30)$$

In the language of Eq. (23) we have the following identifications $t \rightarrow x$, $x \rightarrow y$, $\vec{u} = (\Phi, D_x, D_y)$. Since the right-hand-side of Eq. (30) is in 1D, there is only one matrix A^i , which is given by

$$M = A^y = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \quad (31)$$

Note that since there is only one A^i matrix the characteristic matrix is the same as the A^y matrix. To find the eigenvalues of this matrix we need to solve the characteristic polynomial, i.e., $\det[M - \lambda I] = 0$, which for this matrix becomes

$$\lambda(\lambda^2 + 1) = 0. \quad (32)$$

Thus, there are 3 eigenvalues: $\lambda_0 = 0$, $\lambda_{1,2} = \pm i$. Since, there are purely imaginary eigenvalues, this implies that the system is of elliptic type. We should have anticipated this because the Poisson equation is elliptic.

Exercise 1: What type of system is the one of Eq. (21) when $c = 1$ (we can always set $c = 1$ by choice of units)?

3. Generalized parabolic equations

Consider a system of n partial differential equations for n unknown variables $\vec{u}$ of the form

$$\frac{\partial \vec{u}}{\partial t} = [P_{2m} \left(\frac{\partial}{\partial x} \right) + Q \left(\frac{\partial}{\partial x} \right)] \vec{u}, \quad (33)$$

where P_{2m} is a quasi-linear even-order operator given by

$$P_{2m} \left(\frac{\partial}{\partial x} \right) = \sum_{\nu=2m} A_\nu(\vec{u}) \frac{\partial^{\nu_1}}{\partial x_1^{\nu_1}} \frac{\partial^{\nu_2}}{\partial x_2^{\nu_2}} \cdots \frac{\partial^{\nu_d}}{\partial x_d^{\nu_d}} \quad (34)$$

here $\sum_{i=1}^d \nu_i = \nu = 2m$, d is the number of spatial dimensions, and Q is a quasi-linear operator of order less than $2m$

$$Q \left(\frac{\partial}{\partial x} \right) = \sum_{\nu \leq 2m-1} B_\nu(\vec{u}) \frac{\partial^{\nu_1}}{\partial x_1^{\nu_1}} \frac{\partial^{\nu_2}}{\partial x_2^{\nu_2}} \cdots \frac{\partial^{\nu_d}}{\partial x_d^{\nu_d}}, \quad (35)$$

where $\sum_{i=1}^d \nu_i = \nu \leq 2m-1$, and A_ν , B_ν are $n \times n$ matrix functions of the dynamical variables $\vec{u}$, but not of their derivatives.

Definition: The equation Eq. (33) is called 2m-order parabolic, if for all $k \in \mathbb{R}$, the eigenvalues $\omega_j(k)$, $j = 1, \dots, n$ of the $n \times n$ matrix

$$P_{2m}(ik) = - \sum_{\nu=2m} A_\nu k_1^{\nu_1} k_2^{\nu_2} \cdots k_d^{\nu_d}, \quad (36)$$

(in $P_{2m}(ik)$ the symbol i is the imaginary unit $i^2 = -1$) satisfy

$$\text{Re}(\omega_j(k)) \leq -\delta |k|^{2m}, \quad j = 1, 2, \dots, n, \quad (37)$$

with some $\delta > 0$ independent of k .

Let us see the definition in practice. Consider the heat equation in Cartesian coordinates in three spatial dimensions

$$\frac{\partial T}{\partial t} = \xi \left(\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2} \right) T, \quad (38)$$

The even order operator here has $m = 1$, i.e.,

$$P_2 \left(\frac{\partial}{\partial x} \right) = \xi \left(\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2} + \frac{\partial^2}{\partial z^2} \right) \quad (39)$$

Now we have to replace $\frac{\partial}{\partial x} \rightarrow ik^x$, $\frac{\partial}{\partial y} \rightarrow ik^y$, $\frac{\partial}{\partial z} \rightarrow ik^z$

$$P_2(ik) = \xi \left(i^2(k^x)^2 + i^2(k^y)^2 + i^2(k^z)^2 \right) = -\xi|k|^2. \quad (40)$$

Thus, if $\xi > 0$, then for $\delta > \xi$ we satisfy the definition of parabolicity

$$P_2(ik) = \xi \left(i^2(k^x)^2 + i^2(k^y)^2 + i^2(k^z)^2 \right) = -\xi|k|^2 < -\delta|k|^2 \quad (41)$$

But, if $\xi < 0$ the equation does not satisfy the definition of parabolicity.

Let us now consider a 4th-order system in three spatial dimensions

$$\frac{\partial u}{\partial t} = \theta \left(\frac{\partial^4}{\partial x^4} + \frac{\partial^4}{\partial y^4} + \frac{\partial^4}{\partial z^4} \right) u, \quad (42)$$

The even order operator here now has $m = 2$, i.e.,

$$P_4 \left(\frac{\partial}{\partial x} \right) = \theta \left(\frac{\partial^4}{\partial x^4} + \frac{\partial^4}{\partial y^4} + \frac{\partial^4}{\partial z^4} \right) \quad (43)$$

Now we have to replace $\frac{\partial}{\partial x} \rightarrow ik^x$, $\frac{\partial}{\partial y} \rightarrow ik^y$, $\frac{\partial}{\partial z} \rightarrow ik^z$

$$P_4(ik) = \theta \left(i^4(k^x)^4 + i^4(k^y)^4 + i^4(k^z)^4 \right) = \theta \left((k^x)^4 + (k^y)^4 + (k^z)^4 \right) \quad (44)$$

Thus, if $\theta < 0$, then since $(k^x)^4 + (k^y)^4 + (k^z)^4 < ((k^x)^2 + (k^y)^2 + (k^z)^2)^2 = k^4$ for $\delta > |\theta|$ we satisfy the definition of parabolicity

$$P_4(ik) = \theta \left(i^2(k^x)^2 + i^2(k^y)^2 + i^2(k^z)^2 \right) = -|\theta| \left((k^x)^4 + (k^y)^4 + (k^z)^4 \right) < -\delta|k|^4 \quad (45)$$

But, if $\theta > 0$ the equation does not satisfy the definition of parabolicity. For $\theta < 0$, Eq. (42) is called 4th-order parabolic.

C. Well-posedness of the initial value problem

As in the case of ordinary differential equations, certain types of PDEs can be solved as initial value problems. For PDEs the initial value problem is also referred to as the Cauchy problem. Given that when we have second-order time derivatives we will be reducing the system to first-order in time, consider the following initial value problem (IVP) in d dimensions

$$\frac{\partial \vec{u}}{\partial t} = P \left(\frac{\partial}{\partial x} \right), \quad a \leq t \leq b, \quad \text{Initial conditions: } \vec{u}(t=0, x^1, x^2, \dots, x^d) = \vec{\alpha}(x^1, x^2, \dots, x^d). \quad (46)$$

where P is a spatial quasi-linear differential operator. We will not give a precise definition of well-posedness for PDEs, but we will describe its properties. The IVP (46) is *well-posed* if

- A solution $\vec{u}(t, x^1, x^2, \dots, x^d)$ exists.

- The solution $\vec{u}(t, x^1, x^2, \dots, x^d)$ is unique for given initial data.
- The solution exhibits continuous dependence on initial data.

Theorems in mathematics guarantee the (local) well-posedness of (quasi-linear) linear strongly hyperbolic and parabolic PDEs.

Elliptic PDEs do not admit a well-posed IVP. It is instructive to see this. For both the classification and the well-posedness of the IVP the highest order operator is the most important part of a PDE. Let us consider the wrong-sign wave equation in Cartesian coordinates, i.e., (contrast the following equation with the correct sign Eq. (13))

$$\frac{\partial^2 u}{\partial t^2} + c^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right) = 0 \quad (47)$$

The matrix A_{ij} appearing in Eq. (10) here becomes $A_{ij} = \text{diag}(1, c^2, c^2, c^2)$, hence its eigenvalues are all positive and thus the wrong-sign wave equation is elliptic. Let us now consider that we try to solve this equation as an IVP, i.e., at $t = 0$ we provide initial conditions for $u(0, x, y, z)$, and its time derivative $\frac{\partial u}{\partial t}(0, x, y, z)$. In order for the problem to be well-posed small perturbation should not grow in time, they should be bounded so that the solution satisfies the last condition of well-posedness – continuous dependence on initial data. Thus, let us consider harmonic perturbations $u = u_0 \exp[i(\vec{k} \cdot \vec{x} - \omega t)]$. Plugging this equation in the wrong-sign wave equation we find

$$-\omega^2 u - c^2 k^2 u = 0 \quad (48)$$

which (for $c > 0$) implies that $\omega = \pm ic|k|$. But, the time part of the solution $u = u_0 \exp[i(\vec{k} \cdot \vec{x})] \exp[-i\omega t]$ behaves as $\exp[-i\omega t]$. For $\omega = -ic|k|$, the time part behaves as $\exp[-c|k|t]$, i.e., it is exponentially damped in time. But, when $\omega = ic|k|$ the time part behaves as $\exp[c|k|t]$, i.e., it exponentially blows up. Thus, any initial small perturbations will blow up exponentially in time at a rate that depends on the wavelength of the perturbation, i.e., $|k|$. The shorter the wavelength of the perturbation, the larger the wavenumber $|k|$ and the faster the solution blows up. This is the tell-tale sign of ill-posedness: **When in a correct numerical implementation one increases the resolution and the solution blows up faster, that usually implies an ill-posed initial value problem.**

To sum, for the solution of initial value problems we want to have either strongly hyperbolic (which includes strictly and symmetric hyperbolic PDEs) or parabolic equations, or mixed hyperbolic-parabolic equations.

III. NUMERICAL SOLUTION OF PDES

A. Solving hyperbolic PDEs

We will consider an initial value problem involving the wave equation in one spatial dimension in a box, which is given by

$$\frac{\partial^2 u}{\partial t^2} - c^2 \frac{\partial^2 u}{\partial x^2} = 0, \quad 0 \leq x \leq L, t \geq 0. \quad (49)$$

We can always redefine a new time-coordinate through $\tilde{t} = ct$, and the equation becomes

$$\frac{\partial^2 u}{\partial \tilde{t}^2} - \frac{\partial^2 u}{\partial x^2} = 0, \quad (50)$$

so we will drop the $\tilde{}$ from t and instead we will be dealing with the equation

$$\frac{\partial^2 u}{\partial t^2} = \frac{\partial^2 u}{\partial x^2}, \quad (51)$$

with boundary conditions

$$u(t, 0) = u(t, L) = 0, \quad (52)$$

and initial conditions

$$u(0, x) = f(x) \text{ and } \left. \frac{\partial u}{\partial t} \right|_{t=0} = g(x), \quad (53)$$

Similarly we can always refine our spatial coordinate $\tilde{x} = x/L$, so that the upper boundary $\tilde{L} = L/L = 1$. So, in what follows we will be considering $L = 1$.

We will be solving the wave equation using finite differences. So, first we will create a spatial grid and a time grid,

$$x_i = i\Delta x, \Delta x = 1/M, \text{ and } i = 0, \dots, M. \quad (54)$$

this choice warrants that the boundaries are always grid points, i.e., for $i = 0$, $x_0 = 0$, and for $i = M$, $x_M = 1$. and

$$t_n = n\Delta t, n = 1, \dots, N. \quad (55)$$

where Δt and N will be chosen such that we capture the basic timescale in the problem while we integrate long enough. Since the wave is traveling at speed 1, and the domain has length 1, then it takes unit time to propagate the wave from one end to the other. We would like to evolve for multiple such times, e.g., typically up to $t = 5$, and for the scheme to be stable $\Delta t/\Delta x < 1$. Typically we will chose $M = 100$ or so, which will determine Δx , and choose $\Delta t/\Delta x = 0.8$ which will determine Δt . On this grid we will denote the solution at $t = t_n$ and $x = x_i$ as u_i^n , i.e., $u_i^n = u(t_n, x_i)$.

The next step is to approximate the derivatives that appear in the wave equation. Starting with the right-hand-side of Eq. (51), we approximate the second-order derivative using a second-order accurate finite difference, i.e.,

$$\left. \frac{\partial^2 u}{\partial t^2} \right|_i^n = \frac{u_{i+1}^n + u_{i-1}^n - 2u_i^n}{\Delta x^2}. \quad (56)$$

This equation can be viewed as a set of M coupled ordinary differential equations. As in the case of ODEs we can define the first time derivative as a new variable and evolve a system of two ODEs at every single point of the spatial grid. The method of solving PDEs in this way is called the method of lines, and is the standard method. Before we use more sophisticated solvers for the time-integration (e.g. RK4), let us also finite difference the second-order time derivative using also a second-order accurate stencil. The difference equation Eq. (56) then becomes

$$\frac{u_i^{n+1} + u_i^{n-1} - 2u_i^n}{\Delta t^2} = \frac{u_{i+1}^n + u_{i-1}^n - 2u_i^n}{\Delta x^2}. \quad (57)$$

Solving this equation with respect to u_i^{n+1} we find

$$u_i^{n+1} = 2u_i^n - u_i^{n-1} + \lambda^2(u_{i+1}^n + u_{i-1}^n - 2u_i^n), i = 1, \dots, M-1, \quad (58)$$

and boundary conditions

$$u_0^{n+1} = 0 = u_M^{n+1} \quad (59)$$

where $\lambda = \Delta t/\Delta x$ is called the Courant-Friedrich-Lewy (CFL) number, and must be $\lambda < 1$ for numerical stability. Notice that this is a 3-time level numerical scheme, i.e., to obtain the solution at the $n+1$ step we need the solution at two previous time-levels, u^n and u^{n-1} . This, creates a problem initializing the algorithm described by Eq. (58), since we will have u_i^0 from the initial data, but we need also u_i^1 to start the algorithm. Fortunately, the initial data must also specify the time derivative of u . In other words at $t = 0$ we know

$$\frac{\partial u}{\partial t} = g(x). \quad (60)$$

The simplest way to start the algorithm then is to use a first-order approximation to the time derivative, i.e.,

$$\frac{u_i^1 - u_i^0}{\Delta t} = g(x_i) \Rightarrow \quad (61)$$

$$u_i^1 = u_i^0 + g(x_i)\Delta t. \quad (62)$$

note that if $g(x=0) = g(x=1) = 0$ and $u(0, x=0) = u(1, x=1) = 0$, then u_i^1 will respect the boundary conditions, i.e., $u_0^1 = u_M^1 = 0$. So, with $u_i^0 = f(x_i)$ and u_i^1 now given from Eq. (62) we can start the algorithm described by Eq. (58) for $n = 1$. The only problem is that this initialization will be only first-order accurate.

We can do better than that if $f(x)$ is twice differentiable. The Taylor expansion in t only of u is as follows

$$u(t_1, x_i) = u(0, x_i) + \Delta t \left. \frac{\partial u}{\partial t} \right|_{x_i}^0 + \frac{\Delta t^2}{2} \left. \frac{\partial^2 u}{\partial t^2} \right|_{x_i}^0 + O(\Delta t^3). \quad (63)$$

But, from the initial conditions and the differential equation we have

$$u(t_1, x_i) = u(0, x_i) + g(x_i)\Delta t + \frac{\Delta t^2}{2} \frac{\partial^2 f}{\partial x^2} \Big|_{x_i} + O(\Delta t^3). \quad (64)$$

Thus, the more accurate way to start the numerical integration is to set

$$u_i^1 = u_i^0 + g(x_i)\Delta t + \frac{\Delta t^2}{2} \frac{\partial^2 f}{\partial x^2} \Big|_{x_i}. \quad (65)$$

Note, that care must be taken such that $\frac{\partial^2 f}{\partial x^2} \Big|_{x_0} = 0 = \frac{\partial^2 f}{\partial x^2} \Big|_{x_M}$, so that $u(t_1, x_0) = 0 = u(t_1, x_M)$. We will be comparing the solutions obtained when initialized either with Eq. (62) or Eq. (65).

The algorithm for solving the initial value problem

$$\frac{\partial^2 u}{\partial t^2} = \frac{\partial^2 u}{\partial x^2}, \quad (66)$$

with boundary conditions

$$u(t, 0) = u(t, 1) = 0, \quad (67)$$

and initial conditions

$$u(0, x) = f(x) \text{ and } \frac{\partial u}{\partial t} \Big|_{t=0} = g(x), \quad (68)$$

is as follows

Input: number of timesteps to evolve N , CFL number λ , integer M , initial conditions $f(x)$ $g(x)$.

Output: solution u_i^n for each $i = 0, \dots, M$, and $n = 0, \dots, N$.

Step 1: Set $\Delta x = 1./M$, $\Delta t = \lambda \Delta x$, $t = \Delta t$

Step 2: Introduce arrays of length $M + 1$ called x (for the coordinates), u_{new} and u_{old} , u_{old2} for the new, the previous and the previous previous timestep solutions.

Step 3: For $i = 0, \dots, M$ set $x[i] = i\Delta x$, $u_{\text{old2}}[i] = f(x[i])$, and use Eq. (62) or (65) to set $u_{\text{old}}[i]$. Make sure that $u_{\text{old}}[0] = u_{\text{old}}[M] = 0$, so that the boundary conditions are respected.

Step 4: Main time integration loop;

```
for (int n=2; n<N; n++)
{
    // Update the interior spatial points, i.e., everything but the boundary points
    for (int i=1; i<M; i++)
    {
        unew[i]=2.*uold[i]-uold2[i]+lambda*lambda*(uold[i+1]+uold[i-1]-2.*uold[i]);
    }
    // Set the boundary conditions
    unew[0]=0.;
    unew[M]=0.;
    t+=DeltaT; // Update the time
    Output t, x[i], unew[i] to a file named based on the timestamp.
    A code snippet for this is given below.

    // Set the uold2=uold and uold=unew to set up for the next timestep
    for (int i=0; i<=M; i++)
    {
        uold2[i]=uold[i];
        uold[i]=unew[i];
    }
}
```

Step 5: The procedure is complete.

Recall that to output a file use the following construct in C++

```
#include <iostream>
#include <fstream>
using namespace std;

ofstream myfile ("example.txt");
if (myfile.is_open())
{
    myfile << "# Column 1: t Column 2: x, Column 3: u(x)\n";
    for (int i=0; i<=M; i++)
    {
        myfile << t << " " << x[i] << " " << unew[i] << endl;
    }
    myfile.close();
}
else cout << "Unable to open file";
```

However, what we really want is to change the name of the file as time changes, and add the timestamp to the filename. The following snippet of code achieves this

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main ()
{
    string filename;
    int out_every=1; ever how many timesteps to output solution files here set to 1
    double dt=0.1;
    for (int n=0;n<10;n++){

        if (i%out_every==0){ //

            filename="u_vs_x_"; // Filename prefix
            double t=n*dt;
            string s=to_string(t); // Convert double t to string
            filename+=(s+".dat"); // Concatenate filename prefix with t value as string and extension .dat
            ofstream myfile (filename); // open the output file stream with name that of the filename
            if (myfile.is_open())
            {
                myfile << "# Column 1: t Column 2: x, Column 3: u(x), Column 4: uexact(x)\n";
                for (int i=0; i<=M; i++)
                {
                    // output time x coordinate, numerical solution, and exact solution
                    myfile << t << " " << x[i] << " " << unew[i] " " << uexact(t,x[i]) << endl;
                    // Note that uexact is a function of the exact solution
                }
                myfile.close();
            }
            else cout << "Unable to open file";
        }
    }
    return 0;
}
```


This code snippet will create 10 files with names like

`u_vs_x_0.100000.dat, u_vs_x_0.200000.dat,`

where the number corresponds to the timestamp, and each of these files will have $M + 1$ lines with three columns having values for time, x , and the solution u , respectively. Note that for the above code to work you must compile with C++11 standard, i.e., the compilation command must be supplemented with the “-std=c++11” flag, i.e., “g++ -std=c++11”. Also, note that the above header files, include statements are those necessary for this code to work. You may need additional include statements for the rest of your code. **A beter coding approach is to include the above lines into a void function as follows**

```
void fileoutput(string filename,double t,double *x,double *unew,int M)
{
    // filename only enters with a prefix
    string s=to_string(t); // Convert double t to string
    filename+=(s+".dat");    // Concatenate filename prefix with t value as string and extension .dat

    ofstream myfile (filename); // open the output file stream with name that of the filename
    if (myfile.is_open())
    {
        myfile << "# Column 1: t Column 2: x, Column 3: u(x), Column 4: uexact(x)\n";
        for (int i=0; i<=M; i++)
        {
            myfile << setprecision(16) << setfill(' ') << setw(20) << t << " " << x[i] << " "
            << unew[i] << " " << uexact(t,x[i]) << endl; // output time, x coordinate and solution
        }
        myfile.close();
    }
    else cout << "Unable to open file";
}
```

After you have the file output function, you input the string filename using the filename prefix as in the previous lines, and then you can call the file output function anytime you want to output a file.

Exercise 2: Implement numerically the algorithm for the wave equation we just described. Use $\Delta x = 0.01$, $\lambda = 0.5$, $N = 200$. Use as initial conditions.

$$u(t, 0) = f(x) = \sin(\pi x), \quad 0 \leq x \leq 1. \quad (69)$$

$$\left. \frac{\partial u}{\partial t} \right|_{t=0} = g(x) = 0, \quad 0 \leq x \leq 1. \quad (70)$$

Implement the initialization in Step 3 above, using once Eq. (62) and once Eq. (65). After your code is complete compare the solution at the final time $T = 1$ with the exact solution which is given by $u = \sin(\pi x) \cos(\pi t)$. Make a plot of $u_{\text{numerical}}$ vs x and the exact solution $u(1, x)$ vs x . Then plot the difference between the numerical and analytic solution vs x . Run your code again with $dx = 0.005$, $N = 400$, $\lambda = 0.5$, and plot the difference between the numerical and analytic solution vs x at $T = 1$. By what factor do you have to multiply the difference found for $dx = 0.005$ in order for it to overlap the difference found for $dx = 0.01$? Do this exercise using once Eq. (62) and once Eq. (65). Is the factor different for the two methods?

B. Von Neumann Stability analysis and Lax's equivalence theorem

We have so far seen that round-off error instabilities can arise that destroy the stability of a numerical integration scheme. Numerical schemes for PDEs can also be unstable, and they have to be analyzed for each PDE and PDE scheme separately. The method we will consider here is called the Von Neumann stability analysis.

Let us consider for example the simple IVP of the wave equation, solved with the finite difference scheme we discussed in the previous subsection, i.e.,

$$u_i^{n+1} = 2u_i^n - u_i^{n-1} + \lambda^2(u_{i+1}^n + u_{i-1}^n - 2u_i^n), \quad i = 1, \dots, M-1, \quad (71)$$

Since the equation is linear, the same equation is satisfied by u and any perturbations δu in u . If the equation was non-linear we would have to linearize it about a background solution u to perform the Von Neumann stability analysis. Next consider harmonic perturbations of the form

$$u_i^n = \exp[\omega t_n - ikx_i] \quad (72)$$

Note that from this equation the amplification factor is

$$\Lambda = \frac{u_i^{n+1}}{u_i^n} = \frac{\exp[(\omega t_{n+1} - ikx_i)]}{\exp[(\omega t_n - ikx_i)]} = \frac{\exp[(\omega(t_n + \Delta t) - ikx_i)]}{\exp[(\omega t_n - ikx_i)]} = \exp(\omega \Delta t) \quad (73)$$

Thus, to find whether a numerical scheme is stable or not for a particular PDE (or set of PDEs) we have to use the PDEs to solve for $\exp(\omega \Delta t)$. If we plug Eq. (72) in Eq. (71) we obtain

$$\begin{aligned} & \exp[(\omega(t_n + \Delta t) - ikx_i)] = \\ & 2 \exp[(\omega t_n - ikx_i)] - \exp[(\omega(t_n - \Delta t) - ikx_i)] + \lambda^2 (\exp[(\omega t_n - ik(x_i + \Delta x))] + \exp[(\omega t_n - ik(x_i - \Delta x))] - 2 \exp[(\omega t_n - ikx_i)]) \end{aligned} \quad (74)$$

Diving out the last equation by $\exp[(\omega t_n - ikx_i)]$, yields

$$\exp(\omega \Delta t) = 2 - \exp(-\omega \Delta t) + \lambda^2 (\exp(-ik\Delta x) + \exp(ik\Delta x) - 2) \quad (75)$$

or after using Eq. (73)

$$\Lambda = 2 - \Lambda^{-1} + \lambda^2 (\exp(-ik\Delta x) + \exp(ik\Delta x) - 2), \quad (76)$$

which after using the Euler formula $\exp(ik\Delta x) = \cos(k\Delta x) + i \sin(k\Delta x)$ becomes

$$\Lambda = 2 - \Lambda^{-1} + 2\lambda^2 (\cos(k\Delta x) - 1) = 2 - \Lambda^{-1} - 4\lambda^2 \sin^2(k\Delta x/2) \Rightarrow \quad (77)$$

where we used the identity $\cos(2x) = 1 - 2\sin^2(x)$ in the last step. The equation then becomes

$$\Lambda^2 - 2[1 - 2\lambda^2 \sin^2(k\Delta x/2)]\Lambda + 1 = 0, \quad (78)$$

This is a quadratic equation for Λ , whose discriminant is given by

$$\Delta = 4\{[1 - 2\lambda^2 \sin^2(k\Delta x/2)]^2 - 1\} \quad (79)$$

Thus, the solutions are

$$\Lambda_{\pm} = \Sigma \pm \sqrt{\Sigma^2 - 1} \quad (80)$$

where $\Sigma = 1 - 2\lambda^2 \sin^2(k\Delta x/2)$. If $\Sigma > 1$, then the $+$ always satisfies $\Lambda_+ > 1$ and hence the scheme will be numerically unstable. Thus, for numerical stability we must have $|\Sigma| \leq 1$ which implies

$$|1 - 2\lambda^2 \sin^2(k\Delta x/2)| \leq 1 \Rightarrow \quad (81)$$

$$-1 \leq 1 - 2\lambda^2 \sin^2(k\Delta x/2) \leq 1. \quad (82)$$

The right inequality is satisfied always, but the left part is satisfied when $\lambda \leq 1$. But, does $|\Sigma| \leq 1$ imply $|\Lambda| \leq 1$? If $|\Sigma| \leq 1$ we have

$$\Lambda_{\pm} = \Sigma \pm i\sqrt{1 - \Sigma^2}, \quad (83)$$

Therefore,

$$|\Lambda_{\pm}| = \Sigma^2 + |1 - \Sigma^2| = 1. \quad (84)$$

Therefore, the scheme will always be stable if $\lambda \leq 1$, which means the CFL number must satisfy

$$\frac{\Delta t}{\Delta x} \leq 1. \quad (85)$$

Having the notion of stability, we also discuss the notion of consistency and convergence.

Definition 1: A consistent discrete numerical scheme for a PDE (system) is one when $\Delta t \rightarrow 0$ and $\Delta x \rightarrow 0$ the discrete system becomes equivalent to the underlying exact PDE (system).

Definition 2: A convergent numerical scheme for a PDE (system) is one for which

$$\lim_{\Delta x \rightarrow 0, \Delta t \rightarrow 0} u(t_n, x_i) = u_0(t_n, x_i), \quad (86)$$

where u_0 is the true solution at $t = t_n$ and $x = x_i$ of the PDE for given initial data and boundary conditions.

The Lax equivalence Theorem: Consider a well-posed initial value problem for a PDE (system) with constant coefficients. Then any two of the following 3 conditions imply the other:

- The numerical scheme is consistent.
- The numerical scheme is convergent.
- The numerical scheme is stable.

The most important application of the theorem is on proving convergence especially for applications where we do not have exact solutions. Typically demonstrating consistency is straightforward, and stability is typically easy to show by running the code and ensuring the scheme is stable. Thus, by the Lax equivalence theorem then one proves convergence. However, convergence and self-convergence studies are **always** necessary, especially in combination with Richardson extrapolation to evaluate errors.

C. Parabolic and elliptic PDEs

Before we discuss numerical schemes for solving parabolic and elliptic PDEs let us first try to understand a few properties of parabolic PDEs. As we mentioned earlier in these notes, the archetypical example of a parabolic PDE is the heat equation, which in Cartesian coordinates is given by

$$\frac{\partial u}{\partial t} = \sigma \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} \right), \quad (87)$$

with $\sigma > 0$ the diffusivity parameter. Let us now consider that we have some solution u , and with given boundary conditions we apply harmonic perturbations spatial to it of the form $u = u + \delta u$, $\delta u = \delta \tilde{u} \exp(i \sum_{i=1}^3 k^i x^i)$, with $x^1 = x$, $x^2 = y$, $x^3 = z$, and $k^1 = k^x$, $k^2 = k^y$, $k^3 = k^z$. Then since

$$\frac{\partial^2 \delta u}{\partial x^{i^2}} = -(k^i)^2 \delta u \quad (88)$$

the heat equation for these perturbations becomes

$$\frac{\partial \delta \tilde{u}}{\partial t} = -\sigma k^2 \delta \tilde{u}, \quad (89)$$

where $k^2 = (k^x)^2 + (k^y)^2 + (k^z)^2$ is the magnitude of the wavevector. For any value of k this last equation is a simple ordinary differential equation with exact solution given by

$$\delta \tilde{u} = \delta \tilde{u}_0 \exp(-\sigma k^2 t). \quad (90)$$

In other words, the shorter the wavelength of perturbation, i.e., the higher the wavenumber, the faster that perturbation mode decays to 0. Thus, short-wavelength modes decay extremely fast, this is why parabolic equations have smoothing properties. If you start with extremely noisy initial data and evolve them with a parabolic equation, these data will become smooth very very rapidly. That's what a diffusion (parabolic) equation does. Notice that associated with Eq. (90) there exists a characteristic damping timescale

$$\tau_k = \frac{1}{\sigma k^2} = \frac{\lambda^2}{4\pi^2 \sigma}. \quad (91)$$

This is important to understand qualitatively the stability conditions of numerical schemes for solving parabolic equations.

1. Solving parabolic PDEs

Let us now focus on one spatial dimension, so that the heat equation becomes

$$\frac{\partial u}{\partial t} = \sigma \frac{\partial^2 u}{\partial x^2}. \quad (92)$$

with boundary conditions

$$u(t, 0) = u(t, L) = 0, \quad (93)$$

and initial conditions

$$u(0, x) = f(x). \quad (94)$$

We can always redefine our spatial coordinate to be dimensionless $\tilde{x} = x/L$, so that the upper boundary $\tilde{L} = L/L = 1$, and then our evolution equation becomes

$$\frac{\partial u}{\partial t} = \frac{\sigma}{L^2} \frac{\partial^2 u}{\partial \tilde{x}^2}. \quad (95)$$

In addition, we can define a new dimensionless time through $\tilde{t} = t/(L^2/\sigma)$, so that we finally have to solve

$$\frac{\partial u}{\partial \tilde{t}} = \frac{\partial^2 u}{\partial \tilde{x}^2}. \quad (96)$$

Again, we can drop the $\tilde{}$ and consider the following IVP

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2}. \quad (97)$$

with boundary conditions

$$u(t, 0) = u(t, 1) = 0, \quad (98)$$

and initial conditions

$$u(0, x) = f(x). \quad (99)$$

Next, as in the case of the wave equation we will discretize space and time as

$$x_i = i\Delta x, \quad \Delta x = 1/M, \quad \text{and } i = 0, \dots, M. \quad (100)$$

this choice warrants that the boundaries are always grid points, i.e., for $i = 0$, $x_0 = 0$, and for $i = M$, $x_M = 1$. and

$$t_n = n\Delta t, \quad n = 1, \dots, N. \quad (101)$$

where Δt and N will be chosen such that we capture the basic timescale in the problem while we integrate long enough. Since the characteristic timescale in the problem is given by Eq. (91), the longest timescale corresponds to the longest lengthscale in the problem, and that is the length of the box 1. Thus, we would like to evolve for a few timescales

$$\tau_L = \frac{1}{4\pi^2}. \quad (102)$$

For the scheme to be stable we should resolve the smallest timescale in the problem, which corresponds to the shortest wavelength we can resolve on the grid, which has size $\lambda_{\text{shortest}} = 2\Delta x$ (this is called the Nyquist frequency for a grid). Thus, for stability

$$\Delta t \lesssim \frac{4\Delta x^2}{4\pi^2} = \frac{\Delta x^2}{\pi^2}. \quad (103)$$

Notice, that the higher the resolution (smaller Δx), the smaller the timestep must become for numerical stability. This is the case for hyperbolic equations, too, but in this case the timestep must decrease quadratically with Δx ,

while in the wave equation case it was linearly. This imposes a very stringent constraint on the timestep for parabolic equations. However, there are ways around this by choosing implicit integration schemes.

The next step is to approximate the derivatives that appear in the heat equation. Starting with the right-hand-side, we approximate the second-order derivative using a second-order accurate finite difference, i.e.,

$$\left. \frac{\partial u}{\partial t} \right|_i^n = \frac{u_{i+1}^n + u_{i-1}^n - 2u_i^n}{\Delta x^2}. \quad (104)$$

This equation can be viewed as a set of M coupled ordinary differential equations. As in the case of ODEs this could be integrated with, e.g. RK4. But, let us also finite difference the time derivative using a forward difference method. The difference equation Eq. (104) then becomes

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \frac{u_{i+1}^n + u_{i-1}^n - 2u_i^n}{\Delta x^2}. \quad (105)$$

Solving this equation with respect to u_i^{n+1} we find

$$u_i^{n+1} = u_i^n + \lambda(u_{i+1}^n + u_{i-1}^n - 2u_i^n), i = 1, \dots, M-1, \quad (106)$$

and boundary conditions

$$u_0^{n+1} = 0 = u_M^{n+1} \quad (107)$$

where $\lambda = \Delta t / \Delta x^2$ is again called the CFL number, and strictly speaking must be $\lambda < 1/2$ for numerical stability. Notice that this is a 2-time level numerical scheme, i.e., to obtain the solution at the $n+1$ step we need the solution at u^n . However, the time derivative is not as accurate as in the wave equation where we adopted a second-order accurate scheme, whereas here we adopted a first-order accurate one.

Let us next derive the stability condition for the scheme of Eq. (106). We will again consider harmonic perturbations

$$u_i^n = \exp[\omega t_n + i k x_i] \quad (108)$$

from which the amplification factor becomes

$$\Lambda = \frac{u_i^{n+1}}{u_i^n} = \exp(\omega \Delta t). \quad (109)$$

Plugging the harmonic perturbations in Eq. (106) yields

$$\exp[\omega t_n + i k x_i] \exp[\omega \Delta t] = \exp[\omega t_n + i k x_i] + \lambda \exp[\omega t_n] (\exp[i k (x_i + \Delta x)] + \exp[i k (x_i - \Delta x)] - 2 \exp[i k x_i]). \quad (110)$$

Diving out by $\exp[\omega t_n + i k x_i]$ then we obtain

$$\Lambda = \exp[\omega \Delta t] = 1 + \lambda (\exp[i k \Delta x] + \exp[-i k \Delta x] - 2) = 1 + 2\lambda (\cos(k \Delta x) - 1) = 1 - 4\lambda \sin^2(k \Delta x / 2). \quad (111)$$

For stability we must have $|\Lambda| \leq 1$, so

$$-1 \leq 1 - 4\lambda \sin^2(k \Delta x / 2) \leq 1. \quad (112)$$

The right-hand inequality is always satisfied, but the left one requires

$$\lambda \sin^2(k \Delta x / 2) < \frac{1}{2} \quad (113)$$

and since $\sin^2(k \Delta x / 2) \leq 1$, the inequality is satisfied for any λ if

$$\lambda = \frac{\Delta t}{\Delta x^2} \leq \frac{1}{2}. \quad (114)$$

This is close to our intuitive stability condition of Eq. (103), which states

$$\frac{\Delta t}{\Delta x^2} \lesssim \frac{1}{\pi^2}, \quad (115)$$

and is more stringent since it requires $\lambda \leq 1/\pi^2 \sim 0.1$, while Eq. (114) requires $\lambda \leq 0.5$. But, again physical/mathematical intuition can always do the job for us, even if we do not perform the careful stability analysis. Starting with our intuitive stability condition, we could use trial and error to find how high λ can be before the code becomes numerically unstable.

To sum, Eq. (106) subject to the boundary conditions (107) provide an algorithm for solving the heat equation.

Exercise 3: Implement numerically the scheme algorithm for the heat equation Eq. (106). Use $\Delta x = 0.1$, $\lambda = 0.45$, $N = 200$, and $\Delta x = 0.05$, $\lambda = 0.45$, $N = 800$. Use as initial conditions.

$$u(t, 0) = f(x) = \sin(\pi x), \quad 0 \leq x \leq 1. \quad (116)$$

and boundary conditions $u(0, t) = u(1, t) = 0$. Compare your numerical solution to the analytic solution

$$u(t, x) = \exp(-\pi^2 t) \sin(\pi x). \quad (117)$$

2. Elliptic PDEs

Elliptic PDE cannot be solved as initial value problems. Typically these equation are also finite differenced, and then a set of coupled algebraic PDEs has to be solved, the number of which equals the number of points on the spatial grid. However, there exists another method for solving elliptic PDEs that is called the relaxation method. The idea is to convert the elliptic PDE into a parabolic PDE in some fiducial time parameter. So, considering Poisson's equation

$$\nabla^2 \Phi - 4\pi G\rho = 0 \quad (118)$$

we can introduce a fiducial time t and convert this equation to

$$\frac{\partial \Phi}{\partial t} = \nabla^2 \Phi - 4\pi G\rho \quad (119)$$

set some guess initial conditions, and the correct boundary conditions that correspond to the elliptic equation and then solve this as an initial value problem for parabolic equations. The damping/smoothing properties of parabolic equations that we discussed guarantee that as time evolves the time variations of Φ will go to 0, i.e., $\lim_{t \rightarrow \infty} \frac{\partial \Phi}{\partial t} = 0$. Thus, when evolving for long times Eq. (119) will relax to $\frac{\partial \Phi}{\partial t} = 0$, and hence $\nabla^2 \Phi - 4\pi G\rho = 0$, i.e. the solution will satisfy the original elliptic equation. This is the essence of the numerical methods called relaxation methods for solving elliptic PDEs.